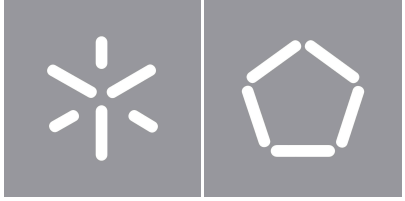
University of Minho
School of Engineering

Pedro Miguel Oliveira Carvalho

WAVE: A Workload-Aware Vessim Extension

July 2026



University of Minho
School of Engineering

Pedro Miguel Oliveira Carvalho

WAVE: A Workload-Aware Vessim Extension

Master's Dissertation in Informatics Engineering

Dissertation supervised by
André Martins Pereira

July 2026

Copyright and Terms of Use for Third Party Work

Acknowledgements

Statement of Integrity

I hereby declare having conducted this academic work with integrity.

I confirm that I have not used plagiarism or any form of undue use of information or falsification of results along the process leading to its elaboration.

I further declare that I have fully acknowledged the Code of Ethical Conduct of the University of Minho.

University of Minho, Braga, July 2026

Pedro Miguel Oliveira Carvalho

Abstract

Keywords keyword one, keyword two

Resumo

Palavras-chave palavra-chave um, palavra-chave dois

Contents

1 Introduction 1

1.1 Motivation and context 1

1.2 Problem statement 2

1.3 Objectives and research questions 3

1.4 Contributions 3

1.5 Dissertation structure 4

2 Background and Related Work 5

2.1 High-Performance Computing 5

2.2 Job scheduling in HPC 7

2.3 Energy and carbon in HPC 9

2.4 Related Work 14

2.5 Synthesis and Research Gap 18

3 WAVE: Workload-Aware Vessim Extension 20

3.1 Research approach and scope 20

3.2 Architecture and integration 21

3.3 Workload execution and resource management 23

3.4 Power demand and energy model 27

3.5 Energy scheduler 32

3.6 Job scheduler 33

3.7 Simulation timing and adaptive stepping 37

3.8 Monitoring and simulation output 38

4 The Puzzle Scheduler 40

4.1 Scheduling problem and overview 40

4.2 Representing jobs and available resources 41

4.3 Constructing the schedule 44

4.4 Jobs that run beyond the forecast 47

4.5 From the plan to execution . . . 50
4.6 Planning cost and limitations . . . 51
5 Evaluation . . . 53
5.1 Experimental setup . . . 53
5.2 Battery operation and frequency control . . . 58
5.3 Scheduling outcomes across energy profiles . . . 62
5.4 Effect of workload size . . . 66
5.5 Carbon threshold and energy-policy sensitivity . . . 67
5.6 Discussion and limitations . . . 70
6 Conclusions and Future Work . . . 73
6.1 Answers to the research questions . . . 73
6.2 Contributions . . . 74
6.3 Limitations . . . 74
6.4 Future work . . . 75
Bibliography . . . 76
A Additional evaluation material . . . 82
A.1 Reproducing the experiments . . . 82
A.2 Workload generation . . . 84
A.3 Scheduling-policy illustration . . . 85
A.4 Energy scheduling details . . . 86
A.5 Reading the execution histories . . . 87
A.6 Grid supply . . . 88
A.7 PV supply . . . 98
A.8 PV and wind supply . . . 108
A.9 Workload-size campaign . . . 117

List of Figures

Figure 1 Illustrative FIFO and backfilling schedules with the same jobs and protected start for job C. .8

Figure 2 Schematic example of equal energy use causing different operational emissions. 12

Figure 3 WAVE architecture, with arrows showing information and control exchanges, not electricity flows. 21

Figure 4 Compute budgets and storage controls coordinated by the energy scheduler. 22

Figure 5 Measurements and control updates through Vessim interfaces under a shared Mosaik clock. 22

Figure 6 A workload record becomes a pending job with the attributes of its configured queue. 24

Figure 7 Reduction in effective progress as partition utilization increases. 25

Figure 8 Illustrative execution progress through overhead, computation and a frequency change. . . 26

Figure 9 Resource-management operations and the resulting node-state transitions. 27

Figure 10 Node state and frequency determine the reported power demand. 28

Figure 11 Illustrative facility energy at different PUE values for a fixed IT demand of 100 kWh. 29

Figure 12 Example solar input: capacity factor before power scaling, 1 to 4 January 2023. 29

Figure 13 Battery exchange followed by grid import for a remaining deficit or grid export for a surplus in grid-connected operation. 30

Figure 14 The monitor combines grid emissions and a baseline term in timestamped records. 31

Figure 15 Example carbon-intensity input series, 1 to 4 January 2023. 32

Figure 16 From local scheduling policies and queue coordination to dispatch through the resource manager. 34

Figure 17 Illustrative prediction errors for a 10 h job, with constant frequency, no contention and immediate completion recognition. 36

Figure 18 Earlier resource release with a scheduling update aligned with job completion. 38

Figure 19 Illustrative 24-hour plan for jobs submitted at plan creation. Bars show reserved execution intervals. 41

Figure 20 Frequency alternatives for a compute-only job on one EPYC Rome node, with PUE 1.3 and idle demand reserved separately. Orange marks the job's grid demand. 42
Figure 21 Illustrative placements under the normal grid allowance. The dashed line includes permitted grid power. 45
Figure 22 Illustrative crossing of the saving rate and the carbon-regret threshold. 46
Figure 23 Illustrative placement before and after repair moves A to supply both jobs from green power. 47
Figure 24 Illustrative plan extended to 48 hours after adding job L. The dashed boundary at 24 hours separates the visible forecast from extrapolation. 48
Figure 25 Illustrative renewable estimates with a 20% margin in either direction for one extrapolated slot. 49
Figure 26 Applying the energy plan through compute budgets, dispatch checks and battery controls. 50
Figure 27 Calculated computation times at base frequency, with the sparse tail shown separately. . . 54
Figure 28 Available renewable power and grid carbon intensity over 160 h. 55
Figure 29 Battery state of charge with grid supply, PV, and PV with grid charging enabled. 59
Figure 30 Busiest-node frequency timelines with fixed control. 60
Figure 31 Busiest-node frequency timelines with adaptive Dynamic Voltage and Frequency Scaling (DVFS). 61
Figure 32 Operational emissions and P95 wait across the three supply profiles. 62
Figure 33 EFT (top) and Puzzle (bottom) execution in the x86 partition with PV and wind, on identical time and power axes. 65
Figure 34 Operational emissions and P95 wait across carbon thresholds under PV and wind. 68
Figure 35 Jobs by partition and requested-walltime queue. 85
Figure 36 Successive selections from a single queue at fixed frequency, with waiting-bound ordering disabled. 85
Figure 37 FIFO policy with grid supply: assigned budgets, planned sources and actor power. 89
Figure 38 FIFO policy with grid supply: cumulative IT energy and operational emissions. 90
Figure 39 Priority policy with grid supply: assigned budgets, planned sources and actor power. 91
Figure 40 Priority policy with grid supply: cumulative IT energy and operational emissions. 92
Figure 41 EFT policy with grid supply: assigned budgets, planned sources and actor power. 93
Figure 42 EFT policy with grid supply: cumulative IT energy and operational emissions. 94
Figure 43 Puzzle with grid supply: assigned budgets, planned sources and actor power. 95

Figure 44 Puzzle with grid supply: cumulative IT energy and operational emissions. 96
Figure 45 Puzzle execution in the ARM partition with grid supply. The power axis starts at the idle baseline. 97
Figure 46 Puzzle execution in the x86 partition with grid supply. The power axis starts at the idle baseline. 97
Figure 47 Puzzle execution in the GPU partition with grid supply. 98
Figure 48 FIFO policy with renewable supply (PV): assigned budgets, planned sources and actor power. 99
Figure 49 FIFO policy with renewable supply (PV): cumulative IT energy and operational emissions. 100
Figure 50 Priority policy with renewable supply (PV): assigned budgets, planned sources and actor power. 101
Figure 51 Priority policy with renewable supply (PV): cumulative IT energy and operational emissions. 102
Figure 52 EFT policy with renewable supply (PV): assigned budgets, planned sources and actor power. 103
Figure 53 EFT policy with renewable supply (PV): cumulative IT energy and operational emissions. . 104
Figure 54 Puzzle with renewable supply (PV): assigned budgets, planned sources and actor power. 105
Figure 55 Puzzle with renewable supply (PV): cumulative IT energy and operational emissions. 106
Figure 56 Puzzle execution in the ARM partition with renewable supply (PV). The power axis starts at the idle baseline. 107
Figure 57 Puzzle execution in the x86 partition with renewable supply (PV). The power axis starts at the idle baseline. 107
Figure 58 Puzzle execution in the GPU partition with renewable supply (PV). 108
Figure 59 FIFO policy with renewable supply (PV and wind): assigned budgets, planned sources and actor power. 109
Figure 60 FIFO policy with renewable supply (PV and wind): cumulative IT energy and operational emissions. 110
Figure 61 Priority policy with renewable supply (PV and wind): assigned budgets, planned sources and actor power. 111
Figure 62 Priority policy with renewable supply (PV and wind): cumulative IT energy and operational emissions. 112

Figure 63 EFT policy with renewable supply (PV and wind): assigned budgets, planned sources and actor power. 113
Figure 64 EFT policy with renewable supply (PV and wind): cumulative IT energy and operational emissions. 114
Figure 65 Puzzle with renewable supply (PV and wind): assigned budgets, planned sources and actor power. 115
Figure 66 Puzzle with renewable supply (PV and wind): cumulative IT energy and operational emissions. 116
Figure 67 Puzzle execution in the ARM partition with renewable supply (PV and wind). The power axis starts at the idle baseline. 117
Figure 68 Puzzle execution in the GPU partition with renewable supply (PV and wind). 117
Figure 69 Operational emissions and P95 wait by workload size under PV and wind. 118
Figure 70 FIFO with the 500-job small workload: carbon emissions, budget sources and actor power. 119
Figure 71 Priority with the 500-job small workload: carbon emissions, budget sources and actor power. 120
Figure 72 EFT with the 500-job small workload: carbon emissions, budget sources and actor power. 121
Figure 73 Puzzle with the 500-job small workload: carbon emissions, budget sources and actor power. 122
Figure 74 FIFO with the 1,250-job medium workload: carbon emissions, budget sources and actor power. 123
Figure 75 Priority with the 1,250-job medium workload: carbon emissions, budget sources and actor power. 124
Figure 76 EFT with the 1,250-job medium workload: carbon emissions, budget sources and actor power. 125
Figure 77 Puzzle with the 1,250-job medium workload: carbon emissions, budget sources and actor power. 126

List of Tables

Table 1 Compute partitions of Deucalion. Node counts and processor configurations are reported by EuroHPC [1]. 6

Table 2 Job and energy scheduling configurations in the main comparisons. 56

Table 3 Modeled queue limits and priorities used in the experiments. 57

Table 4 Verification of execution, energy and emissions. 58

Table 5 Service, energy and execution time for the main runs. Pale green and red mark the lowest and highest values within each profile. 62

Table 6 Observed plan-construction cost on the 2,523-job workload. 66

Table 7 Small and medium workload outcomes under PV and wind. 67

Table 8 Service and energy outcomes at each carbon threshold under PV and wind. 69

Table 9 Service and energy outcomes across the three energy-budget policies. 70

Table 10 Source selection for reproduction. Each directory contains `source_snapshot/`. 82

Table 11 Shared simulation parameters. 83

Table 12 Per-node IT power points used by DVFS. 83

Table 13 FIFO, Priority and EFT with grid supply, each with 2,523 jobs. 88

Table 14 FIFO, Priority and EFT with PV, each with 2,523 jobs. 98

Table 15 FIFO, Priority and EFT with PV and wind, each with 2,523 jobs. 108

Table 16 Full metrics for the small and medium workloads, with sampled excesses relative to each policy's own compute budget. 118

Acronyms

AMD Advanced Micro Devices.

API Application Programming Interface.

ARM Advanced RISC Machines.

ATLAS A Toroidal LHC Apparatus.

AVX-512 Advanced Vector Extensions 512.

CO_2 Carbon Dioxide.

CO_2e Carbon Dioxide Equivalent.

CPI Cycles Per Instruction.

CPU Central Processing Unit.

CSC IT Center for Science.

CUE Carbon Usage Effectiveness.

DC Direct Current.

DVFS Dynamic Voltage and Frequency Scaling.

EAR Energy Aware Runtime.

EARL Energy Aware Runtime Library.

EFLOPS Exaflops (10^{18} floating-point operations per second).

EFT Earliest Finish Time.

EuroHPC European High Performance Computing.

FIFO First-In-First-Out.

GFLOPS Gigaflops (10^9 floating-point operations per second).

GPU Graphics Processing Unit.

HPC High-Performance Computing.

IEC International Electrotechnical Commission.

ISO International Organization for Standardization.

IT Information Technology.

LUMI Large Unified Modern Infrastructure.

MPDATA Multidimensional Positive-Definite Advection Transport Algorithm.

MPI Message Passing Interface.

NAS NASA Advanced Supercomputing.

P95 95th percentile.

PanDA Production and Distributed Analysis.

PUE Power Usage Effectiveness.

PV Photovoltaic.

SLA Service Level Agreement.

SoC State of Charge.

WAVE Workload-Aware Vessim Extension.

Chapter 1

Introduction

1.1 Motivation and context

Throughout the years, High-Performance Computing (HPC) has played a fundamental role in setting the standard for computational power and performance. The TOP500 list highlights the most powerful supercomputers worldwide, showing the advances in this field. In the first list, published in June 1993, the leading system, the Thinking Machines CM-5, achieved a performance of 59.7 GFLOPS [2]. Since then, supercomputers have surpassed the exascale barrier, with El Capitan reaching 1.742 EFLOPS in November 2024 [3].

To achieve such performance levels, modern HPC systems operate under power budgets comparable to those of small cities [4], with operational costs approaching the initial capital investment of the system [5], making energy consumption a constraint in system design and operation. This high energy consumption has environmental implications as well, contributing to greenhouse gas emissions [6]. Energy-efficient techniques such as Dynamic Voltage and Frequency Scaling (DVFS) and power-aware scheduling have been developed to reduce this demand [5], [7], while the Green500 list ranks supercomputers by their energy efficiency [8].

Despite advances in energy management at both hardware and software levels, the analysis of HPC systems remains challenging. Their behavior emerges from the interactions between heterogeneous hardware, diverse workloads, scheduling and resource management policies, and operational constraints. These internal dynamics are further influenced by external factors, such as fluctuating renewable generation and grid carbon intensity, making it difficult to study and optimize system behavior under real-world conditions. Simulation frameworks allow researchers to explore different workloads, scheduling strategies

and energy-management techniques in a controlled environment, without the cost and practical limitations of large-scale experimental deployments.

As HPC infrastructures become more energy and carbon aware, energy-system simulators provide a foundation for studying these interactions. One such framework is Vessim, which models energy systems composed of power consumers, the electrical grid, storage, and renewable sources such as solar and wind [9]. Historical data can represent changes in renewable generation and grid carbon intensity, allowing researchers to study energy flows and the interaction between computing demand and energy availability [9].

1.2 Problem statement

Testing energy efficiency and carbon-aware scheduling directly on a production HPC system is difficult, as experimental policies can disrupt users, change resource allocation and compromise service commitments. Repeating an experiment under the same conditions is also difficult when workload arrivals, energy supply and storage state evolve during operation. A controlled environment is therefore needed to study how scheduling and energy-management decisions affect both computational performance and environmental impact.

Tools such as Green Algorithms provide a method to estimate the carbon footprint of individual computational tasks from their hardware requirements, runtime and execution location [6]. These estimates help researchers assess the impact of their computations, but do not simulate how queued jobs compete for resources or how scheduling decisions change the facility's power demand over time.

Vessim provides the energy-system model needed to study generation, storage and grid exchange, but does not provide an HPC workload model with jobs, queues and compute-resource allocation [9]. Supplying the energy simulation with a power trace leaves the decisions that produced that demand outside the model, so it cannot show how changing job order, node allocation or execution frequency affects the interaction with the energy system. The problem addressed by this dissertation is the coupling of HPC workload execution and energy-system simulation, so that power demand follows scheduling decisions and the resulting energy state can influence later decisions.

1.3 Objectives and research questions

The objective of this dissertation is to address these limitations by introducing workload-aware simulation capabilities into Vessim. We present Workload-Aware Vessim Extension (WAVE), which explicitly models computational jobs and their execution on an HPC system, together with resource management, configurable scheduling policies and hardware power consumption. By connecting these mechanisms to Vessim's energy simulation, we enable the evaluation of energy and carbon-aware strategies under controlled workload and energy conditions.

Within this framework, we also develop the Puzzle scheduler, which uses forecasts of renewable generation and grid carbon intensity, together with battery state, to plan job execution. It selects job start times and execution frequencies while considering available nodes and configured waiting limits, seeking to use renewable energy and reduce the emissions associated with grid imports. This allows us to examine the trade-offs between waiting for a more favorable energy supply and completing the workload sooner.

The work is guided by the following research questions:

1. How can HPC job execution, resource management and scheduling be integrated into an energy-system simulator so that workload decisions and energy availability influence each other?
2. How do scheduling and energy-management policies affect computational performance, energy consumption and carbon emissions under different energy profiles?
3. How does planning job start times and execution frequencies with energy forecasts, as implemented by the Puzzle scheduler, affect these trade-offs compared with the other scheduling policies?

1.4 Contributions

To address the problem described above, this dissertation presents the following contributions:

1. **Job dataset generation**: We developed a synthetic workload generator that represents job characteristics such as instruction count, Cycles Per Instruction (CPI), resource requirements and arrival time. Its configurable parameters allow workloads to be adapted to different system configurations and provide common inputs for controlled comparisons between policies.

2. **Integration of job-aware workload modeling**: WAVE represents computational jobs, tracks their progression from submission to completion, and derives execution progress from the work performed

by the allocated nodes. This connects power consumption to the jobs running and to the configuration of their computing resources.

3. **HPC resource management and scheduling**: We integrated resource management and configurable job-scheduling policies into Vessim. The resource manager maintains node availability and running allocations, while scheduling policies select jobs and their execution frequencies. Energy budgets connect these decisions to renewable availability, storage and grid carbon intensity.

4. **Metrics collection and visualization**: WAVE records job waiting and execution times, resource utilization and queue evolution alongside energy flows and carbon emissions. Gantt charts, node-frequency timelines and energy-system visualizations allow the effects of scheduling and energy-management decisions to be examined together.

5. **Development of the Puzzle scheduler**: We developed a scheduler that represents jobs as pieces with power and duration requirements and fits them into a forecast of available power, while considering node capacity and waiting limits. The plan combines job timing and frequency choices with renewable generation, battery availability and the emissions associated with grid supply, and is updated as jobs arrive and execution progresses.

6. **Evaluation of scheduling policies for carbon-aware computing**: We compare scheduling and energy-management policies using common workloads under different energy profiles. The evaluation includes the Puzzle scheduler and examines the trade-offs between job waiting times, workload completion, energy consumption and carbon emissions.

1.5 Dissertation structure

The remainder of this dissertation is organized as follows. In Chapter 2, we establish the background on HPC systems, workload management, energy efficiency and carbon-aware computing, then review related tools and their limitations. In Chapter 3, we present WAVE, describing its workload, execution, power and energy models, resource management, scheduling interfaces and simulation timing. In Chapter 4, we explain how the Puzzle scheduler represents jobs and available resources, constructs an energy plan and updates it during execution. Chapter 5 presents the experimental setup, compares the policies and discusses the results and their limitations, while Chapter 6 summarizes the conclusions and identifies directions for future work.

Chapter 2

Background and Related Work

2.1 High-Performance Computing

HPC systems combine compute nodes, shared storage, and a high-speed interconnect so that parallel applications can execute across many processors [10]. Modern machines may also combine general-purpose processors with accelerators. Using both can improve performance, energy efficiency, or both, but the benefit depends on how application work is divided between them [11]. Their resources therefore differ not only in capacity, but also in the workloads they execute efficiently.

2.1.1 Supercomputing infrastructure

A supercomputer integrates compute, storage, networking, and management software. Compute nodes contain processors and memory, while high-speed networks connect those nodes to one another and to storage. Resource managers and schedulers allocate nodes and manage queued work, while monitoring services observe jobs and resource use [5].

The facility surrounding that computing equipment must deliver electricity and remove the heat produced during operation. These supporting systems consume energy without executing jobs, so the energy delivered to the computing equipment does not describe the total demand of the facility [5].

2.1.2 The Deucalion supercomputer

Deucalion is a petascale EuroHPC supercomputer hosted in Guimaraes, Portugal [1]. Its heterogeneous architecture contains three compute partitions (Table 1). These partitions use different processor architectures and accelerator configurations, so they provide distinct execution environments rather than one uniform pool of nodes.

Partition	Nodes	Compute resources per node
ARM	1632	Fujitsu A64FX
x86	500	2 × AMD EPYC 7742
Accelerated	33	2 × AMD EPYC 7742 and 4 × NVIDIA A100

Table 1: Compute partitions of Deucalion. Node counts and processor configurations are reported by EuroHPC [1].

Deucalion uses Slurm to expose these resources through ten partitions organized by architecture and service class. Each partition sets a maximum node count and a wall-clock limit. Users therefore select a hardware class when submitting a job, and the request must fit the limits of the chosen partition [12].

2.1.3 Workloads and jobs

A workload is the stream of jobs submitted to a facility. In a batch system, each submission describes the required computing resources and sets a wall-clock limit [13]. These fields bound the allocation and its duration. The actual runtime instead follows from the work performed by the application and the hardware on which it executes. Job sizes and runtimes vary widely, so averages hide much of the distribution presented to a scheduler [14].

Within the Roofline model, the resource that limits execution is not the same for every job. Compute-bound work is constrained by processor throughput, whereas memory-bound work is constrained by memory bandwidth. Which of these limits applies depends on both the workload's operational intensity and the machine executing it [15]. The same application may therefore encounter a different bottleneck on a different machine.

A rigid job requests a fixed number of nodes and retains them throughout execution [16]. Malleable jobs can release or acquire resources while running, but doing so requires the application to redistribute its work and the surrounding runtime and scheduling system to coordinate the change [17]. For a rigid job, the scheduler can decide when it starts and which compatible nodes it uses, but it cannot change the job's size [17].

Because a scheduler works with whatever mix of jobs arrives, the workload used in an experiment is part of its result. Feitelson evaluated one processor-allocation scheme on an unchanged system under three different distributions of job size, and the utilization it could reach differed markedly between them, because each distribution left a different amount of fragmentation [14]. Results obtained on one queue therefore cannot be generalized to another without characterizing the workloads involved.

2.2 Job scheduling in HPC

2.2.1 Scheduling objectives and performance metrics

A job cannot always start when it arrives because rigid jobs require their full allocation before execution, and enough resources may not be available while other jobs are still running. Jobs that cannot start wait in a queue, where a scheduling policy decides which of them runs next as resources are released [18]. On a heterogeneous machine, the policy may also choose among compatible resources. These decisions affect both resource utilization and the time each job waits to run.

Utilization measures the fraction of available node capacity allocated to running jobs, whereas waiting time measures the interval between submission and the start of execution. Because the same wait has a different relative effect on a short and a long job, slowdown divides turnaround time by runtime. Bounded slowdown limits the influence of very short jobs by replacing runtimes below a chosen threshold τ with that threshold [19].

$$\mathrm{bsld} = \max\left(\frac{T_{\mathrm{wait}} + T_{\mathrm{run}}}{\max(T_{\mathrm{run}}, \tau)}, 1\right)$$

In the formulation used by Fan et al. [20], the outer maximum assigns a value of 1 to a job that experiences no effective slowdown. The threshold must be reported with the result because different studies choose different values. These metrics expose different parts of the same schedule, so improving utilization does not by itself establish how users experience it.

2.2.2 Queue ordering and backfilling

First-In-First-Out (FIFO) starts jobs in submission order. This preserves arrival order, but a wide job at the head of the queue can leave smaller sets of free nodes unused while it waits for its full allocation. This loss is a form of resource fragmentation [19].

Backfilling instead allows a later job to use such a gap when its resource and runtime estimates show that it will not delay a protected start time [21]. In Figure 1, job D fills otherwise idle nodes without delaying job C.

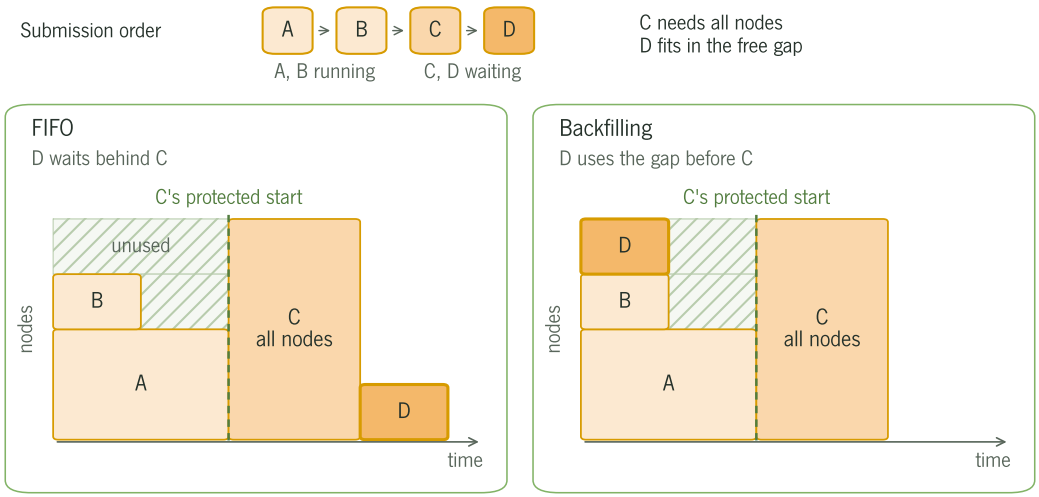


Figure 1: Illustrative FIFO and backfilling schedules with the same jobs and protected start for job C.

Queue order can also favor properties other than arrival time. Moving shorter jobs forward may improve responsiveness on some workloads, but repeatedly passing over long jobs can increase their delay [19]. The choice is therefore conditional on the workload and metric, and policies that use job duration depend on an estimate that is not known exactly at submission.

2.2.3 Runtime estimates and production backfilling

In Slurm's backfill scheduler, the expected completion time of a running job is derived solely from its configured time limit [21]. On Deucalion, the same configured limit also bounds execution: its partition guide specifies node and time limits and states that jobs exceeding their limit are terminated, but does not specify a maximum waiting time [12]. Estimation errors are asymmetric: an overestimate can make a job harder to fit into an available window, whereas an underestimate can terminate it before completion [20]. Analyses of production traces found that users tend to overestimate and repeatedly select a small set of round time limits [22].

Slurm assigns each eligible job a priority from configurable factors such as age, fair share, partition, quality of service, and job size. Its backfill plugin projects resource availability from running jobs and their configured limits, then starts a lower-priority job only if this does not delay any higher-priority job considered in the pass [21]. The schedule therefore depends on site policy and imperfect runtime information. By deciding when and where jobs execute, it also shapes the timing and location of energy demand.

2.2.4 Multi-objective scheduling

A batch scheduler may need to evaluate several properties of the same allocation. A constraint determines whether an allocation is admissible, while an objective ranks the admissible alternatives. When several objectives are retained separately, one allocation may improve one outcome while worsening another. A

Pareto set contains the non-dominated alternatives, in which improving one objective requires worsening at least one other. It exposes the available compromises, but does not itself decide which one should be executed [23].

Fan et al. apply this structure in BBSched. From a fixed window at the front of an existing scheduler's queue, it selects a subset of jobs whose combined node and shared burst-buffer requests fit the available capacities. The two objectives maximize the amounts of these resources allocated at each scheduling decision. A multi-objective genetic algorithm approximates the Pareto set, after which a separate decision rule selects one allocation. The authors evaluated the method in trace-driven simulations based on Cori and Theta logs. Under Theta-S4, a synthetic stress workload in which 75% of jobs were assigned burst-buffer demands sampled from the inferred requests larger than 20 TB, BBSched increased node usage by 20.03% and reduced average waiting time by 41% relative to the study's sequential baseline under the same queue-ordering policy [23]. Waiting time was an evaluation metric rather than a third objective. Both reported improvements came from the same severe modeled contention scenario rather than from observed scheduler outcomes on either production system.

Power-aware scheduling makes the distinction between objectives and constraints particularly important. Wallace et al. estimate power profiles from measurements of earlier jobs and their project groups, then use a window at the front of the queue to maximize the number of allocated nodes under a system-wide power cap. In a trace-driven simulation of 24,565 jobs from one year of operation on Mira, more than 99% of scheduling intervals remained within a 3 MW cap, equivalent to 62.5% of the machine's peak power. When the window was full and none of its jobs could fit under the cap, however, the scheduler allowed the oldest job to exceed it to prevent the queue from blocking [24]. A power cap restricts which jobs can run concurrently, but it does not identify the feasible schedule with the lowest emissions. That choice also depends on when electricity is consumed and on its carbon intensity [25].

2.3 Energy and carbon in HPC

The energy demand shaped by scheduling is described through related but non-interchangeable quantities. Power is the rate at which computing resources use energy, whereas energy accumulates that demand over time. Operational emissions additionally depend on how the electricity was produced.

2.3.1 Facility energy, cooling, and heat reuse

Computing equipment produces heat during operation, and the facility must remove it to keep the hardware within its operating conditions. Cooling is therefore one source of facility overhead: it consumes

energy without executing jobs [5]. Power Usage Effectiveness (PUE) does not measure cooling alone, but relates the energy used by the entire facility to the energy delivered to the Information Technology (IT) equipment [26].

$$\mathrm{PUE} = \frac{E_{\mathrm{facility}}}{E_{\mathrm{IT}}}$$

A PUE of 1 would imply no non-IT facility overhead within the measurement boundary. The difference between PUE and 1 expresses the supporting energy used per unit of IT energy.

PUE does not measure the facility's absolute energy use or the work produced by its computing equipment. Because IT energy is its denominator, the ratio can fall when IT consumption rises even if total facility consumption also rises. Its value also depends on the measurement boundary. For example, moving cooling work from room-level equipment into server fans shifts energy into the IT term and can improve PUE without reducing total consumption. The metric treats electricity from different sources alike and therefore does not measure operational emissions [27].

Cooling demand varies with both computation and the conditions surrounding the facility. Conficoni et al. model this relationship by connecting workload and ambient conditions to air and liquid cooling components. During validation on the Galileo supercomputer, the largest difference between predicted and measured monthly average PUE was approximately 3% [28]. The result shows how cooling overhead can be represented as a response to operating conditions rather than as a fixed addition to IT energy.

Once heat is removed from the computing equipment, part of it can be supplied to an external demand rather than rejected. In a modeled case study of the planned Lichtenberg II system, Oltmanns et al. found that the campus district heating route could use approximately 50% of the recovered heat, while direct use in nearby buildings could use approximately 20% [29]. Unlike that modeled analysis, LUMI shows the mechanism in operation: CSC reports that its recovered heat was supplied to Kajaani's district heating network throughout most of the year and was equivalent to approximately 10% of the city's demand in 2025 [30]. In both cases, the usable share depends on the temperature of the recovered heat and on the presence of a compatible external demand.

2.3.2 Computational energy efficiency and runtime control

Computational energy efficiency uses a different boundary by relating work to the power used to perform it. The Green500 expresses this as the maximum LINPACK performance achieved divided by the average power used during that benchmark [31]. This ratio distinguishes benchmark performance from benchmark

energy efficiency, but remains specific to one workload and measurement procedure. It does not account for whole-facility overhead in the way PUE does, and it ignores the source of the electricity, so it cannot establish the carbon emissions of a production workload.

Energy use within the computing stack can be changed before and during execution. Software optimization adapts an implementation to the installed hardware, while runtime controls modify how that hardware executes the implementation. These interventions act at different scales, so their reported energy gains depend on different baselines and cannot be compared directly.

Szustak et al. optimized the memory-bound Multidimensional Positive-Definite Advection Transport Algorithm (MPDATA) computational fluid dynamics application through code transformations that increased cache reuse, partitioned cores into work teams, and reduced synchronization overhead. On a dual-socket server with two 64-core AMD EPYC 7742 processors, the basic reference implementation consumed between 9.18 and 9.43 times as much total energy as the version combining these transformations across two problem sizes. Both versions used the same compiler optimization settings, so the measured difference comes from the application-level transformations evaluated for MPDATA [32].

DVFS instead changes the processor's operating point during execution while retaining both the hardware and application. Corbalan et al. apply this control through Energy Aware Runtime Library (EARL), the runtime library of the Energy Aware Runtime framework. EARL detects iterative regions in Message Passing Interface (MPI) applications and combines measurements of execution time, node power, CPI, and memory activity with node-specific models to select a Central Processing Unit (CPU) frequency online. Its `min_time_to_solution` policy raises the frequency only when the projected performance gain is large enough relative to the frequency increase. The authors evaluated seven scientific applications and one NAS benchmark using between 4 and 26 nodes, reaching 1,024 MPI processes. Relative to executions at the fixed nominal frequency of 2.4 GHz, the four memory-intensive applications reduced aggregate Direct Current (DC) node energy by 7% to 14% while increasing execution time by 1% to 4%. Three CPU-intensive applications reduced energy by only 1% to 2%, whereas the AVX-512-intensive GROMACS reduced it by 10% with a 3% increase in execution time [33]. EARL therefore manages a workload-dependent energy-runtime tradeoff within each running job, while queue order and resource allocation remain separate scheduling decisions.

2.3.3 Operational and embodied carbon

Operational emissions depend on the energy consumed and the carbon intensity of the electricity at the time and location of consumption. Carbon intensity expresses greenhouse gas emissions per unit

of electricity, commonly in gCO2e/kWh, and can vary across both dimensions [25]. Consequently, a job can consume less energy yet cause more emissions when it runs against a more carbon-intensive supply. Figure 2 isolates the timing effect: equal energy use produces different operational emissions in two carbon-intensity windows. An energy objective therefore cannot generally stand in for a carbon objective [34].

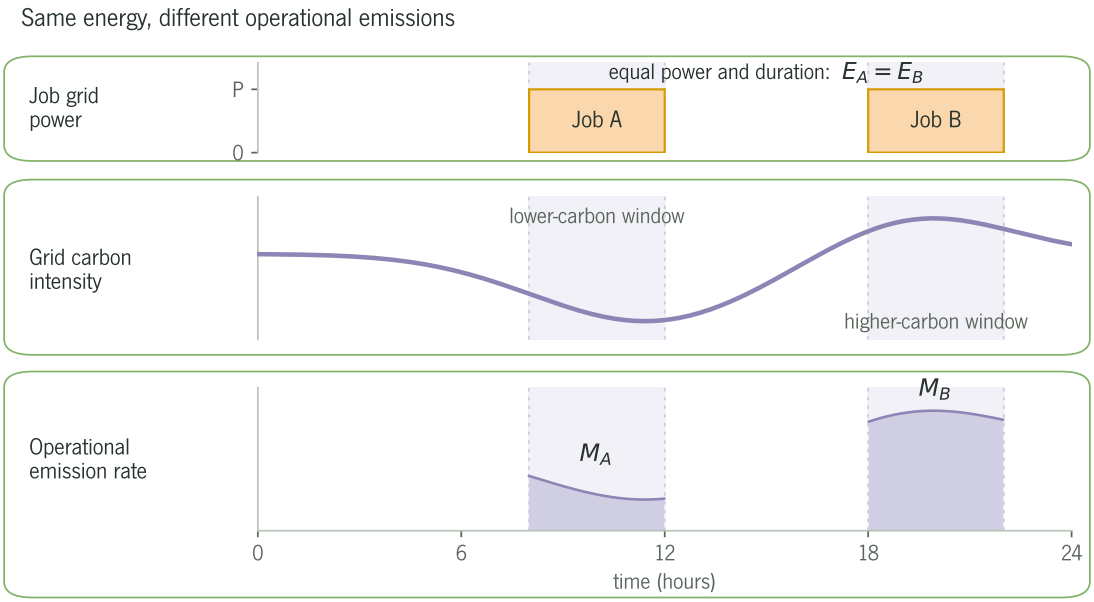


Figure 2: Schematic example of equal energy use causing different operational emissions.

Carbon Usage Effectiveness (CUE) relates those operational emissions to the energy consumed by IT equipment [35]. Let M_{facility} denote the mass of CO2-equivalent emissions caused by the energy supplied to the facility, and let E_{IT} denote the annual energy consumed by its IT equipment. The metric is

$$\text{CUE} = \frac{M_{\text{facility}}}{E_{\text{IT}}}$$

CUE is reported in kg of CO2-equivalent per kWh. The energy covered by its emissions term follows the same facility boundary as E_{facility}, the numerator of PUE. For a single grid source, CUE can also be expressed as the carbon emission factor multiplied by PUE. International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) 30134-8:2022 standardizes the metric for the use phase of a data center's lifecycle [36].

That use-phase boundary excludes the emissions embodied in the facility and its equipment. A data-center lifecycle assessment extends beyond operation to the facility's structural, mechanical, electrical, and IT infrastructure across manufacturing, transport, replacement, and end of life [37].

Hardware replacement can lower operational emissions, but manufacturing the new equipment adds embodied carbon. Li et al. modeled three upgrade paths among four-GPU systems based on NVIDIA P100, V100, and A100 accelerators. In their scenario with 40% average GPU usage, operational savings offset the embodied carbon added by an upgrade in less than six months at an average grid carbon intensity of 400 g of CO_2 per kWh, but required approximately five years or more at 20 g of CO_2 per kWh, with further variation across workloads and upgrade paths [38]. Runtime control does not replace the equipment and instead changes the operational energy and emissions of its use.

2.3.4 On-site generation and energy storage

On-site renewable generation changes the source of the electricity available to the facility, but its production does not necessarily coincide with computing demand. Storage can retain part of that generation for later use. It changes when the electricity is available without reducing the energy required by the computation itself [39].

GreenSwitch coordinates on-site solar generation, storage, and the grid. The authors evaluated it on the 64-server Parasol prototype with a 32 kWh battery bank using a day-long, non-deferrable MapReduce trace derived from Facebook. For the reported comparison, they did not run a second execution without battery use. Instead, they reconstructed that baseline from the same workload execution and solar conditions. Against this baseline, GreenSwitch reduced total grid electricity cost by 75% [39]. The evaluation measured electricity cost rather than carbon emissions, and the prototype did not include a conventional HPC batch scheduler.

2.3.5 Carbon-aware scheduling

Carbon-aware scheduling changes when, where, or how computation executes using information about the emissions associated with its electricity supply. It applies the distinction between energy use and operational carbon established in Section 2.3.3 to the scheduling problem [34]. Carbon awareness therefore adds the state of the energy supply to the objectives, metrics, and constraints introduced in Section 2.2.

Temporal workload shifting delays or reschedules flexible work to a period with lower expected carbon intensity. Its feasible window depends on submission time, expected duration, time constraints, and interruptibility, while choosing within that window also depends on carbon-intensity forecasts [40]. Spatial shifting instead places or moves work to a compatible site with a less carbon-intensive electricity supply [41]. It requires available capacity outside the original facility. Within one facility, temporal decisions can be combined with resource allocation or DVFS, whose energy-runtime trade-off becomes carbon-aware only when selected using information about the energy supply.

On-site generation and storage make these decisions dependent on the facility's energy state. Scheduling shapes electricity demand over time, while generation, stored energy, and the grid determine the emissions associated with serving that demand. The same policy can therefore produce different outcomes under a different workload, energy trace, storage state, or service constraint. Evaluating carbon-aware scheduling requires a model of the workload and scheduler together with a model of the energy system. The following related work compares tools that represent either side of this interaction and approaches that connect them.

2.4 Related Work

The reviewed tools are therefore organized by model boundary. Carbon-footprint calculators derive aggregate estimates from execution inputs. HPC workload simulators represent jobs, resources, and scheduling, whereas energy-system simulators represent generation, storage, and grid exchange. Coupled approaches connect parts of these domains. Each work is compared through its mechanism, workload abstraction, energy and carbon models, evaluation evidence, and the feedback it permits between computational demand and energy-system state.

2.4.1 Carbon-footprint estimation tools

Green Algorithms estimates the operational carbon footprint of a computation without requiring direct power measurements. Processor energy is derived from runtime, the number and type of CPU or GPU cores, their per-core power and usage, while memory energy follows from the allocated capacity. The sum is multiplied by PUE and by the carbon intensity associated with the execution location. A pragmatic scaling factor can also represent repeated runs such as parameter tuning [6]. Processor power is normally approximated from thermal design power, and full core utilization is assumed when the usage factor is unknown, so the estimate depends on the supplied values and defaults.

One trade-off exposed by the calculator concerns resource allocation. Lannelongue et al. applied it to published execution times for the GEANT4 TestEm12 example on a Xeon Phi processor. Increasing the allocation from 15 to 60 cores halved runtime. Under the study's worldwide-average PUE and carbon-intensity assumptions, however, estimated emissions rose from 238 to 481 g of CO_2-equivalent [6]. The example shows that faster execution need not lower emissions when the additional resources do not produce a proportional speedup. Although Green Algorithms can compare predefined execution scenarios, its fixed-input model cannot represent feedback between HPC scheduling and an evolving energy system, making it unsuitable for evaluating carbon-aware scheduling policies that adapt to energy-system state.

2.4.2 HPC workload simulators

HPC workload simulators differ in where they draw the boundary between application execution and workload-management decisions. SimGrid provides an execution substrate rather than a complete workload manager. Applications are expressed as concurrent processes, existing MPI programs, or task graphs, while the simulated platform specifies compute capacity, network topology, bandwidth, and latency. In an HPC case study with BigDFT on up to 128 nodes, SimGrid reproduced the parallel-speedup trend within 8% of the real measurements, whereas the LogGPS model reached errors of up to 40% [42]. This comparison provides application-level evidence for the computation and communication models used by tools built on SimGrid, but job submission, queue state, allocation, and scheduling must be implemented above them.

WRENCH and CGSim build higher-level abstractions on this substrate for workflows and multi-site computing. WRENCH represents scientific workflows through tasks, dependencies, data, and the services that execute them. Across three Pegasus scenarios repeated five times, with workflows from 71 to 1,240 tasks, its simulated makespan differed from real executions by less than 5% [43]. CGSim instead models a distributed computing grid in which a plugin assigns jobs to sites with their own resources and local queues. Using six months of ATLAS PanDA records, its calibration reduced the geometric mean of the relative mean absolute errors for job wall time across 50 sites from 76% to 17% [44]. WRENCH therefore centers workflow dependencies and data services, while CGSim centers allocation between sites. Neither evaluation represents the queue priorities, wall-clock limits, and backfilling decisions of one HPC resource manager, which limits their use for the local HPC batch-scheduling problem considered in this dissertation.

Batch-scheduling simulators choose the queue as their main abstraction. Batsim runs the SimGrid platform and workload model in one process and delegates decisions to a separate scheduler process through a synchronous event interface. When a job arrives or completes, the simulation pauses, reports the event, and applies the scheduler's reply. Jobs can use fixed durations or computation and communication profiles, while decisions can place or reject a job, change DVFS mode, or switch a machine on or off. The authors compared Batsim with the OAR resource manager on a homogeneous Grid'5000 platform and obtained similar turnaround-time distributions. They also state that the experiment did not exercise contention or heterogeneity and that the energy model had not been validated on real machines [45]. Batsim therefore exposes the control point required to test schedulers, but its energy state concerns the simulated machines rather than the generation, storage, grid exchange, and carbon intensity that can influence carbon-aware scheduling decisions.

AccaSim keeps the event manager and a customizable dispatcher in one Python library, reducing trace-processing overhead through incremental job loading. The event manager uses the actual duration to complete a job, while the dispatcher sees the queued and running jobs, available resources, and any supplied duration estimate. Galleguillos et al. compared simulator overhead by rejecting every submitted job across three production traces. In this benchmark, AccaSim performed trace preprocessing during the measured run, while Batsim's preprocessing was excluded. On the 5,731,100-job MetaCentrum trace, AccaSim completed the experiment in 6 min 23 s with an average memory use of 19 MB, compared with 29 min 29 s and 12,647 MB for Batsim [46]. Since no job was allocated or executed, the comparison establishes trace-processing scalability rather than scheduling or execution accuracy. AccaSim permits custom attributes and behaviors for power and energy, but the paper presents them as extension points rather than a validated energy model, leaving energy-system behavior outside the simulation.

SPARS moves closer to this dissertation's scope by processing batch scheduling, node power-state transitions, and energy accounting in the same discrete-event engine. Its workload describes submission time, requested nodes, requested wall-clock time, and actual runtime, while the platform assigns power and transition delays to active, idle, sleeping, and switching states. Scheduling policies can control both job dispatch and node state, either directly or through a reinforcement-learning interface. Under matched workloads, platform settings, and scheduling policies, total energy differed from Batsim by 1% for the Ames iPSC/860 trace and by 0.75% on average across the tested timeout settings for the Euler trace [47]. The evaluation nevertheless uses a simplified static node-power model, and the configurable DVFS model is not evaluated. Because its energy accounting is derived from node states rather than a time-varying facility energy system, SPARS cannot represent feedback from renewable generation, storage, grid exchange, or carbon intensity into scheduling, making it unsuitable for evaluating the interaction between HPC scheduling and a time-varying facility energy system.

2.4.3 Energy-system simulators

The facility-side behavior left outside these workload simulators can be assembled through Mosaik, which coordinates independently developed simulators without providing their domain models. Its sim-manager establishes connections, while a co-simulation scheduler exchanges data at discrete steps according to a directed dependency graph. The component Application Programming Interface (API) exposes each simulator, and the scenario API selects, parameterizes, and connects the participating simulators. Steinbrink et al. demonstrate the architecture in a virtual power plant where connected models shift industrial demand to keep the aggregate schedule within transformer limits [48]. Mosaik contains no simulation

models, so it supplies execution and synchronization without defining component behavior. It therefore does not define an HPC workload, resource manager, or job-scheduling policy.

Vessim builds on this orchestration layer with a Python interface for composing microgrids around computing systems. Actors report power production or consumption, the grid calculates the resulting balance, controllers observe and modify state, and optional storage settles the remaining energy between simulation steps [9]. Renewable-generation and carbon-intensity traces can drive these components, while software-in-the-loop interfaces connect real or emulated computing systems as power-consuming actors. Wiesner et al. demonstrate these interactions in a 24-hour scenario that connects a power-metered NVIDIA Jetson TX2 to simulated solar generation, a battery, and grid carbon-intensity data. The controller activates a power-saving mode for overnight operation, reducing the device's demand from approximately 15 W to 7.5 W [9]. Even in this closed-loop scenario, the published testbed exposes computing through actors and application-facing interfaces, not through an HPC resource manager with queued jobs, heterogeneous partitions, or backfilling.

2.4.4 Coupling computing and energy management

Where Vessim exposes computing through actors, DATAZERO gives the computing and electrical sides separate decision modules that negotiate a common power profile. The IT Decision Module derives candidate profiles from schedules of batch tasks and services, the Power Decision Module evaluates renewable-generation and storage constraints, and a Negotiation Module exchanges utility-scored proposals until it reaches a feasible compromise [49]. Changes on the electrical side can trigger IT rescheduling, while the agreed profile determines which power sources and storage devices to use [49].

In a proof-of-concept run driven by real workload and weather data, the profiles stabilized from negotiation round 13 [49]. In a separate comparison over 312 batch and service jobs, its two distance thresholds reduced average overall Service Level Agreement (SLA) violations from 2.89% for Best-Fit and 2.95% for GreenSlot to 0.81% and 0.23% [49]. The corresponding execution times were 10.69 and 16.31 minutes, compared with 3.16 minutes for Best-Fit and 0.43 minutes for GreenSlot [49]. Since the comparison assumes perfect renewable forecasts, predefined task durations, continued scheduling after due dates, and concurrent jobs per machine, it validates the negotiation mechanism under controlled conditions rather than the behavior of a production HPC resource manager [49].

Landré et al. minimize the power needed to process a forecast workload by choosing on/off and DVFS states for heterogeneous machines [50]. They represent the workload as parts with release times, operation counts and deadlines, allowing work to be processed across successive steps. Service constraints

limit the share of operations that miss their deadlines, while power commands remain constant for 15 minutes to one hour even when the computing load changes at finer time steps [50].

On a Metacentrum-derived workload spanning 7406 hours and modeled on 305 machines, all operations were processed in 98.42% of the evaluated time intervals [50]. The deadlines were derived from recorded waiting and execution times, so the result describes completion of divisible work under those deadlines. This differs from assessing whether an individual rigid job starts within its queue's waiting limit [50].

Carbon Explorer studies datacenter design by jointly varying the renewable generation mix, battery capacity and additional server capacity for carbon-aware load shifting [51]. Its scheduling model moves a flexible share of hourly demand towards periods with more renewable supply, subject to available server capacity and completion within the same day [51]. The model accounts for both operational and embodied emissions, so the infrastructure required to reduce grid demand also contributes to the carbon objective [51].

With 40% of the workload assumed to tolerate delay, Carbon Explorer's modeled renewable-generation and scheduling configurations deploy 6% to 76% additional server capacity across the studied regions [51]. This result shows that concentrating computation into favorable energy periods can require additional infrastructure. The model represents flexibility as a share of hourly demand, rather than following the allocation of individual rigid jobs [51].

2.5 Synthesis and Research Gap

Carbon calculators, HPC workload simulators and energy testbeds expose different parts of a carbon-aware decision. Green Algorithms derives an estimate from aggregate computing inputs, Batsim represents jobs and scheduler decisions, and Vessim represents generation, storage, grid exchange, and carbon signals [6], [9], [45]. These boundaries determine what an evaluation can reveal, since a change in the computing power trace requires an energy-system model to expose its consequences for renewable use, storage and grid demand.

DATAZERO coordinates application schedules with energy supply through negotiated power profiles, while Carbon Explorer studies the infrastructure needed to shift flexible demand [49], [51]. Landré et al. also model heterogeneous machines and DVFS, representing work as divisible parts with arrivals and deadlines [50]. This dissertation focuses on rigid batch jobs with fixed simultaneous node allocations, evaluating their waiting and completion.

WAVE extends Vessim with an HPC workload model that follows each job from arrival to completion, retaining its queue constraints, node allocation and execution progress. Dispatch and frequency decisions generate power demand, while the evolving microgrid state informs subsequent compute budgets. This allows waiting and completion to be evaluated alongside renewable use, storage, grid exchange and operational emissions.

Chapter 3

WAVE: Workload-Aware Vessim Extension

WAVE extends Vessim with the jobs, computing resources and scheduling decisions that produce a facility's power demand. Rather than supplying the energy simulation with a fixed consumption trace, we derive demand from workload execution and node configuration, while energy availability constrains later dispatch and frequency decisions.

3.1 Research approach and scope

Evaluating scheduling and energy-management policies directly on a production supercomputer would require changing the allocation of real jobs and the way energy is supplied to the machine, then repeating the experiment despite workload arrivals and energy signals varying over time. We therefore use simulation to keep the workload, infrastructure and external energy signals fixed while changing the policy under evaluation, allowing differences between runs to be attributed to the decisions made by each policy.

We model batch workload execution and compute-resource allocation, using the state and hardware configuration of the nodes to determine facility power demand through a constant PUE. On the energy side, renewable generation, battery storage and grid exchange supply this demand. Operational grid emissions follow physical imports and time-varying grid carbon intensity, while the monitor also adds a simplified term intended to represent renewable infrastructure. Hardware execution is represented through compute cycles and execution overhead rather than cycle-accurate processor simulation, while detailed network contention, dynamic cooling behavior and a full life-cycle assessment remain outside the model's scope.

Comparisons between policies depend on the workload representation, hardware parameters and energy-system assumptions, so a policy that performs well in one configuration may behave differently when job characteristics, resource capacity or renewable availability change. Results must therefore be interpreted

within the tested conditions, with sensitivity experiments used to examine how far the observed behavior holds when those conditions vary.

3.2 Architecture and integration

3.2.1 Component overview

WAVE adds workload execution and resource management to Vessim's energy simulation, organizing computing resources into partitions connected to a shared energy system. Each partition has a job scheduler and a resource manager, while an energy scheduler coordinates compute budgets across partitions, as shown in Figure 3.

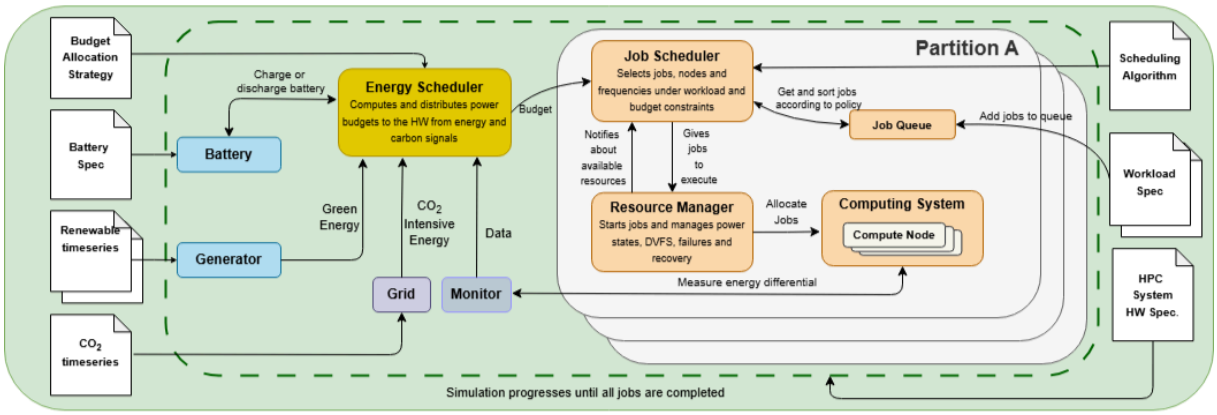


Figure 3: WAVE architecture, with arrows showing information and control exchanges, not electricity flows.

The job scheduler coordinates selection and execution as simulated time advances, using a scheduling policy to choose work from the jobs that have arrived. The resource manager maintains node states and running allocations, carrying out operations such as starting jobs, releasing nodes and changing their frequencies, providing a standardized interface that allows scheduling policies to be changed without modifying resource management.

The energy scheduler distributes compute budgets across partitions and forwards storage commands to the microgrid policy, as shown in Figure 4. Partition schedulers decide which jobs can run within their assigned budgets, while the microgrid policy handles storage and grid exchange.

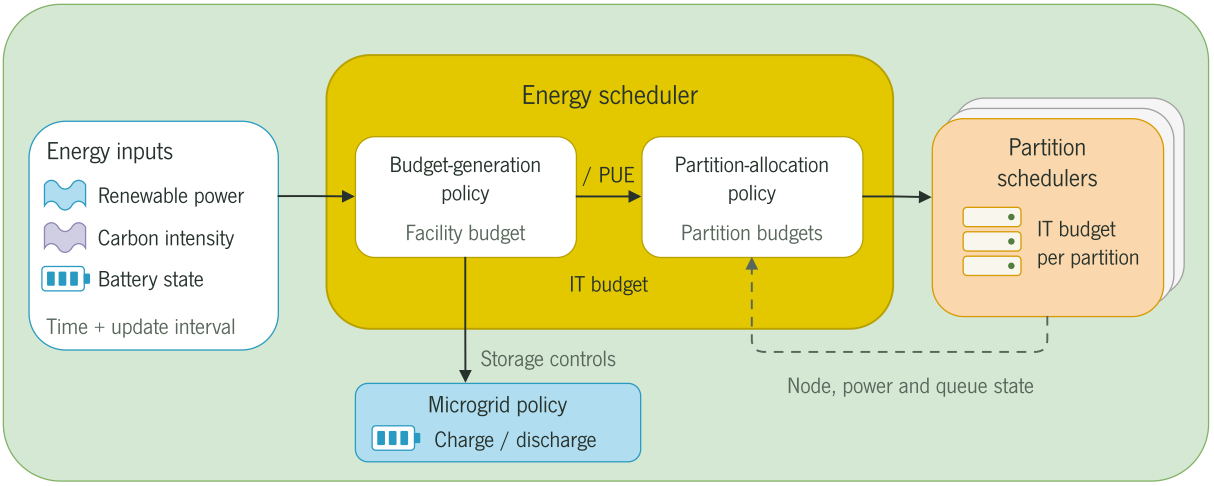


Figure 4: Compute budgets and storage controls coordinated by the energy scheduler.

3.2.2 Integration with Vessim

WAVE uses the actors and controllers introduced in Section 2.4.3. Compute nodes act as power meters for Vessim's computing-system actor, which sums their demand and applies facility overhead, while renewable-generator actors obtain production from input time series (Figure 5).

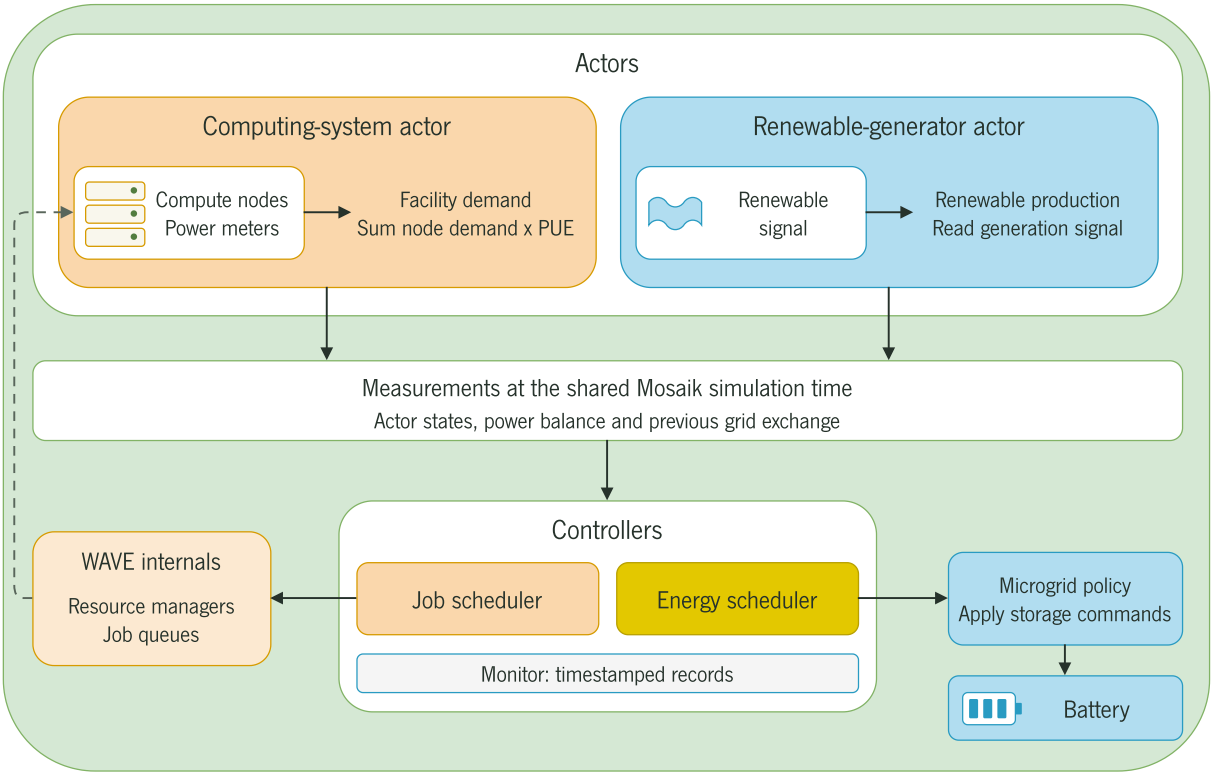


Figure 5: Measurements and control updates through Vessim interfaces under a shared Mosaik clock.

The job schedulers and energy scheduler are registered as controllers in the same microgrid, with Mosaik coordinating their updates. Vessim supplies the time and measurements shown in Figure 5, with grid

exchange referring to the preceding interval. WAVE connects these controllers to its queues, resource managers and energy inputs so they can inspect and update the system.

Controller decisions take effect through node-state changes and storage commands. Figure 5 places the components within Vessim's interfaces, distinguishing them from WAVE's internal queues and resource managers.

3.3 Workload execution and resource management

3.3.1 HPC system and workload model

WAVE represents the computing system as a set of heterogeneous partitions, each containing nodes with a shared hardware profile. Jobs are assigned to a compatible partition and request a fixed number of nodes, which they retain throughout execution without being paused or moved to another partition, so resources available elsewhere cannot automatically satisfy that request.

For example, the x86 workload record in Figure 6 describes a job arriving at simulation start and requesting eight nodes, with a requested wall-clock limit of 1252 s and execution attributes supplied alongside that request.

The requested wall-clock limit determines which logical queue can accept the job, while instruction count, cycles per instruction and memory demand describe its execution. Jobs with the same requested limit can therefore take different times to finish, with Figure 6 distinguishing these attributes from the queue's waiting bound. WAVE uses the requested limit for queue classification, not to terminate execution when that duration is exceeded.

When loading a workload, WAVE creates a job object from each record and converts its arrival offset into a submission time relative to the simulation start. Each input file is associated with a logical queue, and the loader checks that the requested wall-clock limit falls within that queue's runtime range before attaching its priority weight, node limit and maximum waiting time to the job. The resulting jobs are ordered by arrival and considered for dispatch at their submission times, following the conversion shown in Figure 6. Jobs requesting more nodes than their queue allows are rejected by the scheduler and marked as failed.

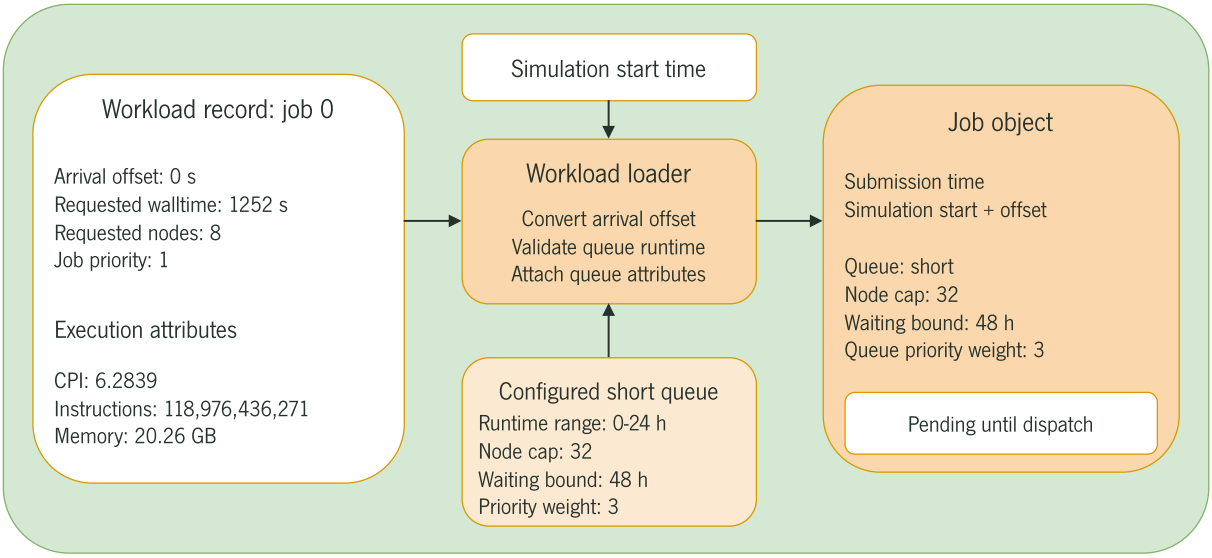


Figure 6: A workload record becomes a pending job with the attributes of its configured queue.

The job object retains its submission attributes alongside mutable state such as its status, allocation and execution times, keeping scheduler-facing runtime estimates separate from the remaining work used to advance execution.

3.3.2 Job execution model

WAVE separates compute work from memory and multi-node overhead. For a job with N_I instructions and c cycles per instruction, compute work is $C = cN_I$, giving the following nominal duration at a constant frequency f in GHz, without contention:

$$T(f) = \frac{C}{f \cdot 10^9} + \frac{M}{B_{\text{mem}}} + H(n)$$

Here, M is the job's memory demand in GB, B_{mem} is the partition's configured memory bandwidth in GB/s, and $H(n)$ represents the overhead associated with an allocation of n nodes. The current model sets this overhead to zero for a single-node job and applies a penalty of 0.1 s per allocated node otherwise, using an uncalibrated heuristic rather than simulating network communication. Memory cost is likewise represented by an aggregate bandwidth term rather than individual accesses, while node count determines resource occupancy and multi-node overhead without introducing an additional parallel speedup.

During execution, remaining compute cycles C_{rem} and remaining overhead O_{rem} are tracked separately, with each simulation interval first reducing the overhead and then using any time left to advance compute work at the current frequency. Partition utilization u is the fraction of all partition nodes that are busy, including off and down nodes in the total node count, so $0 \leq u \leq 1$. An aggregate contention factor $\alpha(u)$ then reduces the effective progress interval as utilization rises:

$$\alpha(u) = \begin{cases} 1 & \text{if } u \le 0.8 \\ 1 - 0.5(u - 0.8) & \text{otherwise,} \end{cases} \quad \Delta t_{\text{eff}} = \alpha(u)\Delta t$$

As Figure 7 shows, the effective progress interval falls to 90% of the simulated interval at full utilization.

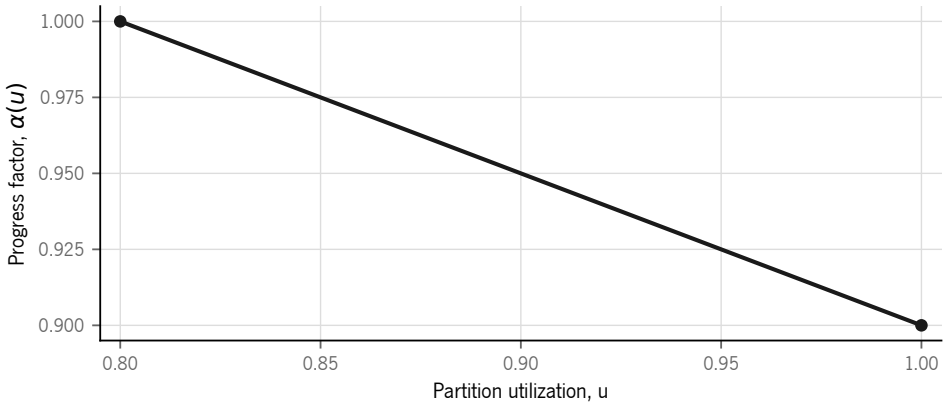


Figure 7: Reduction in effective progress as partition utilization increases.

For an interval of Δt seconds, the consumed overhead δO and compute cycles δC are bounded by the work remaining at its start:

$$\delta O = \min(O_{\text{rem}}, \Delta t_{\text{eff}}), \quad \delta C = \min\big(C_{\text{rem}}, f \cdot 10^9 (\Delta t_{\text{eff}} - \delta O)\big)$$

The next execution state follows by subtracting these quantities, so an interval may consume only overhead, cross from overhead into computation or exhaust the remaining work:

$$O_{\text{rem}'} = O_{\text{rem}} - \delta O, \quad C_{\text{rem}'} = C_{\text{rem}} - \delta C$$

A frequency change affects compute progress but not the remaining overhead. In Figure 8, overhead is consumed before computation begins, and a later frequency reduction slows progress without changing work already completed. Points mark step boundaries, with lines showing progress under constant frequency and contention within each interval.

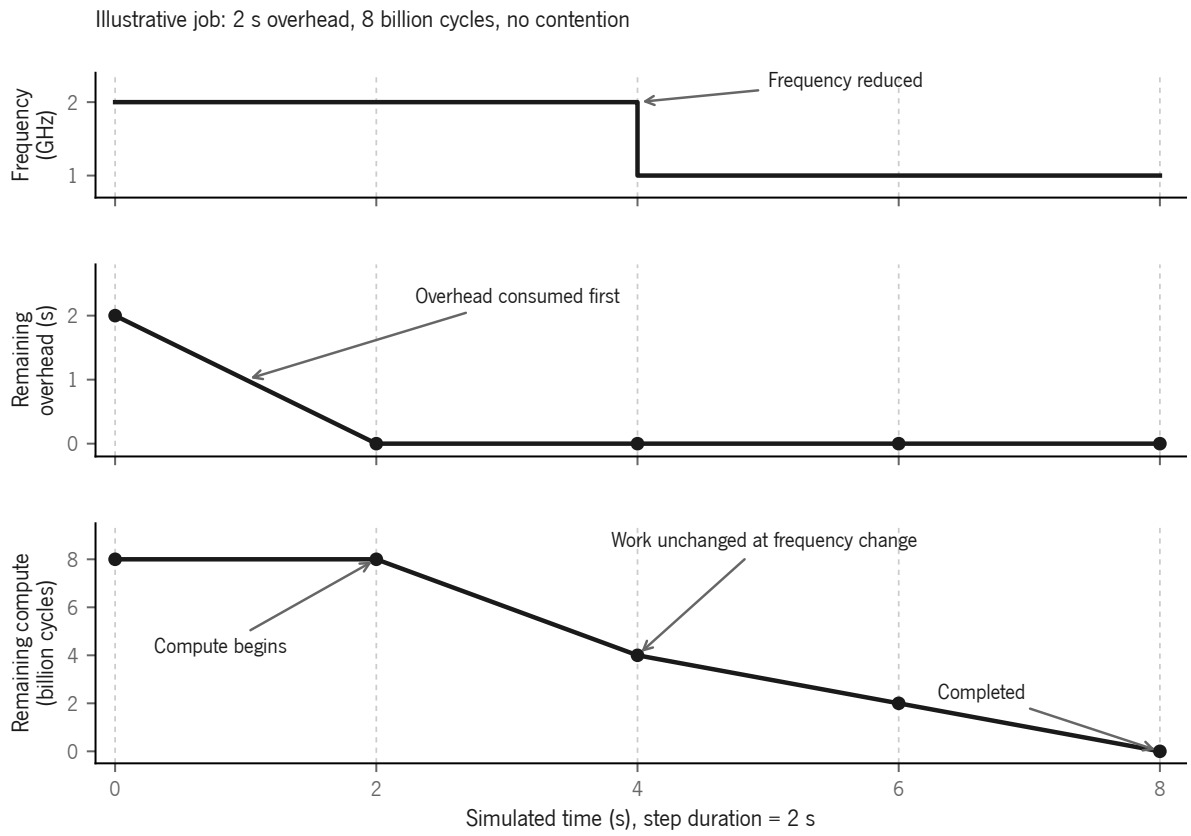


Figure 8: Illustrative execution progress through overhead, computation and a frequency change.

A job remains pending until it is dispatched, at which point its allocated nodes become unavailable to other jobs and its execution state becomes running. Completion occurs when both remaining overhead and compute work have been consumed, after which the allocation is released, while a modeled node failure can instead terminate the job as failed.

3.3.3 Resource management

The resource manager tracks running allocations, associating each job with its nodes and operating frequency (Figure 9). Dispatch requires idle nodes and marks them busy, while completion removes the allocation and returns its nodes to the idle pool.

Alongside idle and busy nodes, the resource manager tracks nodes that are off and nodes that are down, distinguishing resources switched off by power management from those unavailable after a failure. Only idle nodes can be switched off through this interface, so powering down unused resources does not interrupt a running allocation. An off node becomes idle when switched on, whereas a down node must first complete its modeled downtime before returning to the available pool.

The resource manager switches on nodes according to pending demand, within the budget available to cover their idle power. A stricter mode considers only jobs that the scheduling policy would admit within

the compute budget, keeping nodes off when no such job can use them. Idle nodes beyond the resulting target, or whose demand exceeds the budget, are switched off without interrupting running jobs.

When failure injection is enabled, a failure event selects a busy node and identifies the job occupying it. The current implementation then marks the job as failed and places all nodes in its allocation in the down state, assigning each a downtime sampled from the configured range and removing the allocation. This treats the event as an outage of the whole allocation rather than an isolated hardware fault, although its nodes can return to service at different times as their respective downtimes expire, following the node-state transitions shown in Figure 9.

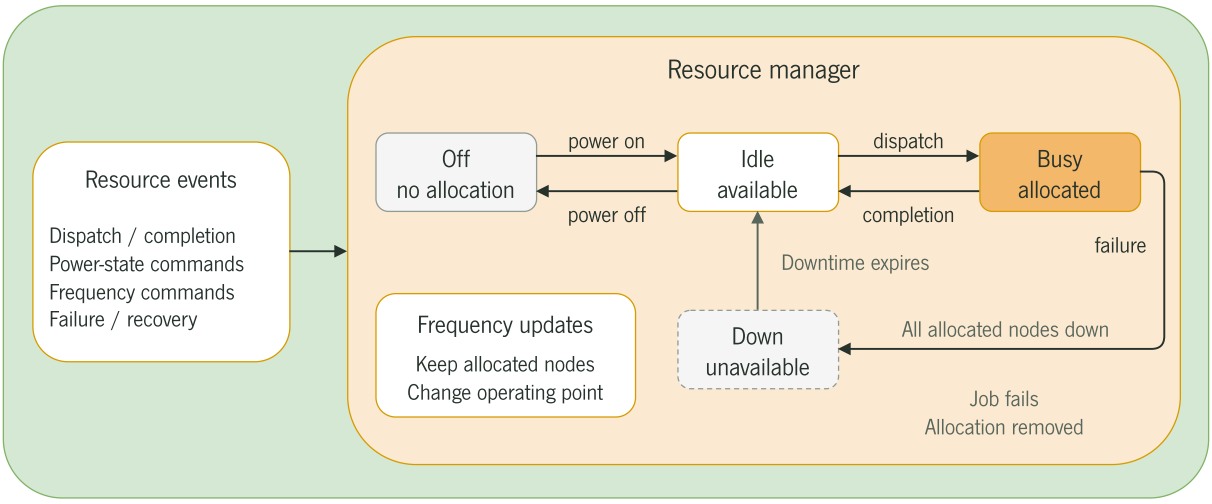


Figure 9: Resource-management operations and the resulting node-state transitions.

Frequency changes apply to all nodes in an allocation without releasing them. The resource manager updates their frequency and dynamic power, records the change in the job's history and recalculates its remaining duration.

3.4 Power demand and energy model

3.4.1 Node and facility power model

Node demand combines an idle contribution with a dynamic contribution at the selected frequency. An idle node retains its base demand, while off and down nodes draw no power in the model. For node i in partition k, power in watts is:

$$P_{k,i}(t) = \begin{cases} P_{\text{idle},k} + P_{\text{dyn},k}(f_i(t)) & \text{if busy} \\ P_{\text{idle},k} & \text{if idle} \\ 0 & \text{if off or down} \end{cases}$$

Dynamic power comes from the partition's configured frequency-power pairs, so jobs on the same hardware at the same frequency have the same modeled node demand. The transitions in Figure 9 determine which contributions apply, and the power-meter interface in Figure 10 reports their sum to Vessim.

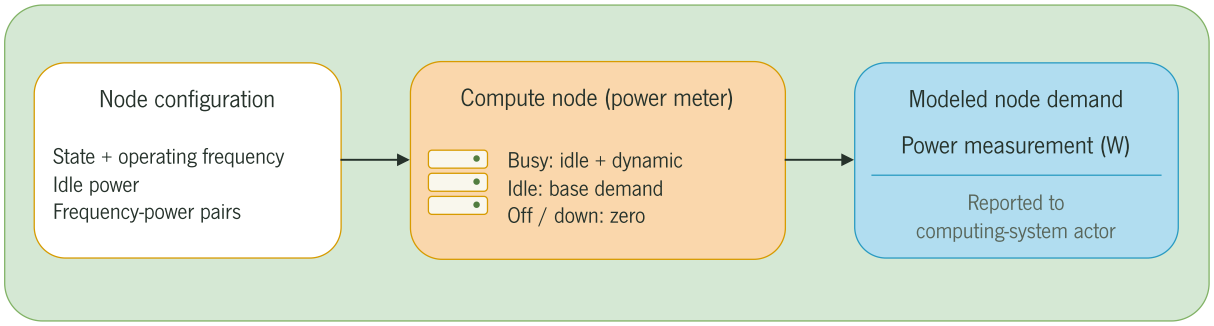


Figure 10: Node state and frequency determine the reported power demand.

Partition demand is obtained by summing the contributions of all its nodes, including those that remain idle, while total IT demand combines the partitions:

$$P_k(t) = \sum_{i \in \mathcal{N}_k} P_{k,i}(t), \quad P_{\mathrm{IT}}(t) = \sum_k P_k(t)$$

We convert this IT demand into facility demand through a constant PUE, keeping the power associated with compute resources separate from the total demand supplied by the energy system:

$$P_{\mathrm{facility}}(t) = \mathrm{PUE} \cdot P_{\mathrm{IT}}(t)$$

PUE represents cooling and power-delivery overhead without modeling that equipment separately. Node power remains constant between state or frequency changes, without resolving intra-job activity or localized contention. Powering on or recovering a node returns it directly to idle, with no separately timed boot phase or associated power profile.

With PUE held constant, the same multiplier applies to energy, as illustrated in Figure 11 for a fixed IT demand.

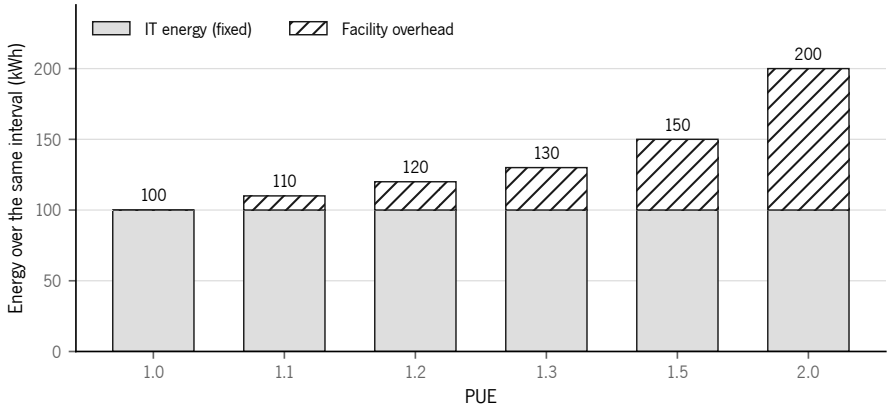


Figure 11: Illustrative facility energy at different PUE values for a fixed IT demand of 100 kWh.

3.4.2 Energy supply, storage and grid exchange

The facility demand obtained from the node power model is supplied by renewable generation, battery discharge and grid import, while battery charging and grid export absorb power that is not used by the facility. Renewable generation varies over time according to the input data, with the available energy and storage state used to establish the compute budget within which the job scheduler can dispatch work. These input profiles and their power scales can be adapted to different locations and installation sizes, with an example solar profile shown in Figure 12.

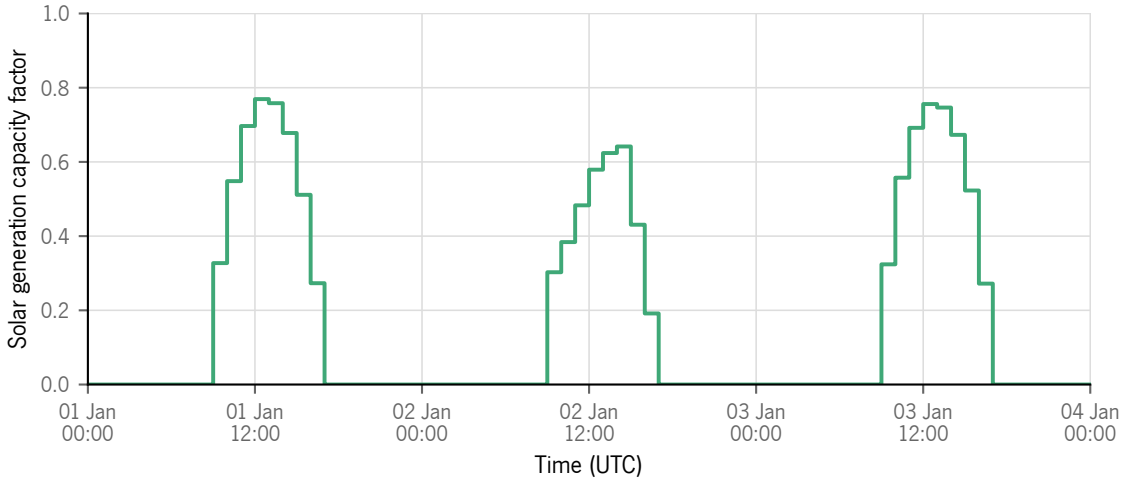


Figure 12: Example solar input: capacity factor before power scaling, 1 to 4 January 2023.

The energy system then balances the resulting facility demand, charging the battery with surplus power where permitted and exchanging the remaining imbalance with the grid. Using nonnegative quantities for each flow, the power balance is:

$$P_{\text{renewable}} + P_{\text{discharge}} + P_{\text{import}} = P_{\text{facility}} + P_{\text{charge}} + P_{\text{export}}$$

Battery charging and discharging are limited by their configured maximum power and by the energy available within the allowed state-of-charge range, with conversion losses determining how much of the exchanged energy is stored or delivered. For an interval of Δt seconds, the stored energy E_{battery}, expressed in Wh, evolves as:

$$E_{\text{battery}}(t+\Delta t) = E_{\text{battery}}(t) + \left(\eta_{\text{charge}} P_{\text{charge}} - \frac{P_{\text{discharge}}}{\eta_{\text{discharge}}}\right)\frac{\Delta t}{3600}$$

Here, η_{charge} and $\eta_{\text{discharge}}$ are the charging and discharging efficiencies, with the exchanged power reduced whenever the requested action would exceed a storage bound. In grid-connected operation, the grid supplies any remaining deficit or receives surplus energy that is not stored, whereas islanded operation cannot sustain demand beyond the available renewable generation and battery discharge. The grid allowance used to construct the compute budget limits the power offered to the scheduler, rather than imposing a physical limit on the remaining exchange with the grid. Figure 13 distinguishes the microgrid policy's storage request from the exchange that the battery can actually accept.

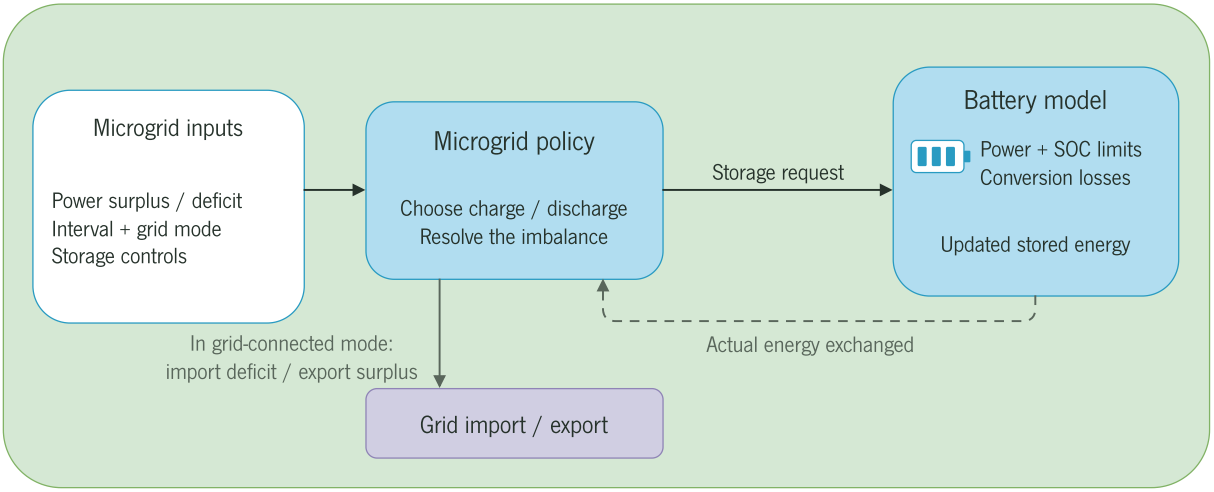


Figure 13: Battery exchange followed by grid import for a remaining deficit or grid export for a surplus in grid-connected operation.

3.4.3 Carbon emissions calculation

Carbon emissions are calculated from the energy physically imported from the grid after battery charging or discharging, using the carbon intensity at the time of import so that drawing the same amount of energy at different times can produce different emissions. With imported energy $E_{\text{import},j}$ expressed in kWh and carbon intensity I_j in gCO_2/kWh, cumulative grid-related emissions in grams are:

$$C_{\text{grid}} = \sum_j E_{\text{import},j} I_j$$

This calculation includes grid energy used to charge the battery, so using that stored energy later does not add another emissions contribution, while exporting surplus energy to the grid does not subtract emissions from the total. The monitor also adds a baseline term intended to represent renewable infrastructure, calculated from carbon intensity at each record rather than from renewable energy production. Because it is added once per record without scaling by interval duration, it depends on monitoring frequency and is excluded from the operational grid emissions used to compare policies. The monitor records the combined value, and the analysis separates its two contributions as described in Section 5.1.4. The monitoring interface is summarized in Figure 14.

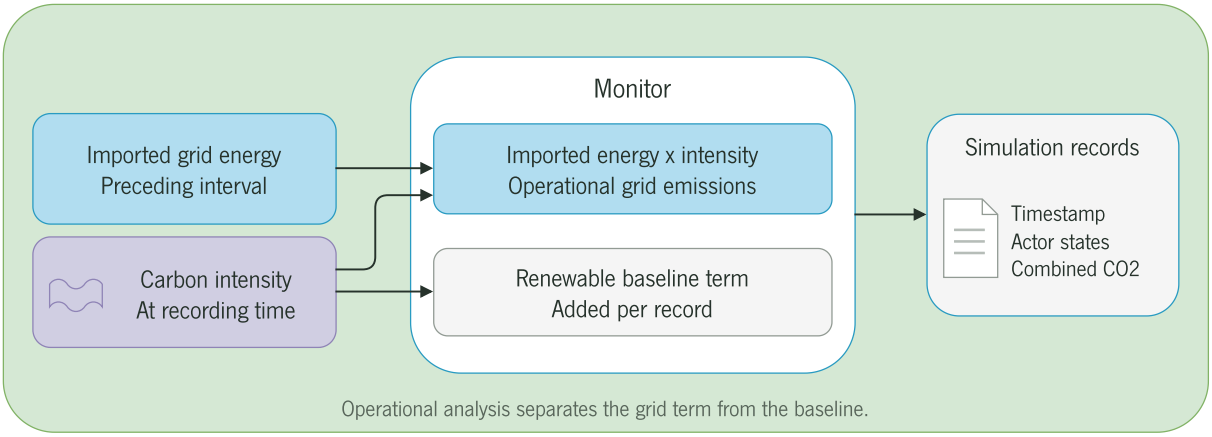


Figure 14: The monitor combines grid emissions and a baseline term in timestamped records.

For partition-level analysis, emissions are divided according to each partition's share of instantaneous IT demand, while energy sources are assigned using the proportions in the global compute budget. This breakdown does not trace electricity to individual nodes, so attributed grid energy is not a measurement of physical grid import.

The carbon-intensity time series can be replaced with data for the region being studied without changing the workload or resource model. Figure 15 shows an example input, with each value held until the next sample becomes available.

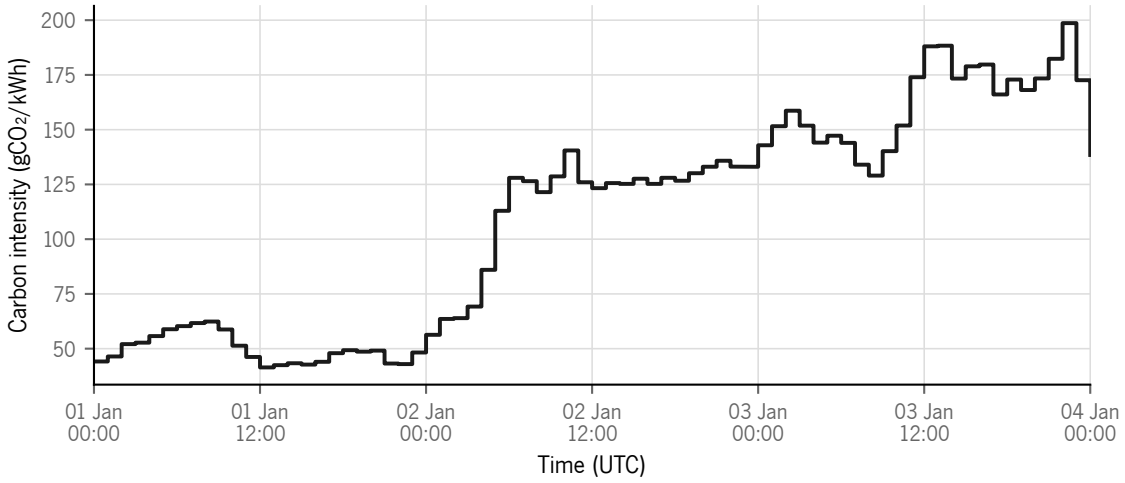


Figure 15: Example carbon-intensity input series, 1 to 4 January 2023.

3.5 Energy scheduler

The energy scheduler coordinates the power offered to computation and the storage commands passed to the microgrid, through the interface shown in Figure 4. It delegates budget generation and partition allocation to separate policies, so either decision can be changed independently.

3.5.1 Budget generation and storage commands

The energy scheduler asks a budget-generation policy how much power to offer to computation, using the inputs shown in Figure 4. The policy can read current signal values or request forecasts for a planning window, returning a compute budget together with storage commands that the energy scheduler forwards to the microgrid policy.

These commands specify whether battery discharge is allowed and can request a charging power, while the microgrid and battery model determine the actual exchange within the physical limits described in Section 3.4.2.

Since this budget includes facility overhead, we divide it by PUE before distributing it among partitions, keeping both budgets in watts:

$$B_{\mathrm{IT}}(t) = \frac{B_{\mathrm{facility}}(t)}{\mathrm{PUE}}$$

3.5.2 Partition budget allocation

The allocation policy divides the IT budget using each partition's current demand and available capacity, or planned shares when following an energy plan (Figure 4). The energy scheduler passes these allocations to the partition schedulers.

When a partition finishes, it notifies the energy scheduler, which asks the allocation policy to redistribute the existing IT budget. This does not wait for the next periodic budget update or generate a new total budget.

3.6 Job scheduler

Each partition's job scheduler uses its assigned compute budget to select work and apply resource changes. Scheduling policies can choose jobs and frequencies while using the same resource and execution models.

3.6.1 Scheduling inputs and feasibility

At dispatch, the scheduler uses the arrived jobs, available nodes and current compute budget, together with runtime estimates at supported frequencies. Planning policies may also use future arrivals and energy signals, but cannot start a job before it arrives.

In practical terms, scheduling a job means deciding where and when it will start, which nodes it will occupy and the frequency at which those nodes will operate. For a job j, we represent this decision as:

$$d = (j, k, \mathcal{N}, t_s, f)$$

Here, k identifies the execution partition, $\mathcal{N}$ contains the allocated nodes, t_s is the start time and f is the selected frequency. In the workloads used by this study, each job is associated with a compatible partition before scheduling, which means that the scheduler chooses nodes and a frequency within that partition but does not move the job to a different hardware architecture.

The partition's compute budget is expressed as IT power, excluding the facility overhead represented by PUE.

Before jobs are ordered according to a scheduling objective, each possible decision must satisfy the system constraints. If a_j is the arrival time of job j, n_j is its requested number of nodes, $\mathcal{K}_j$ contains its compatible partitions and $\mathcal{I}_k\,(t_s)$ contains the idle nodes available in partition k, a feasible decision must satisfy:

$$t_s \geq a_j, \quad k \in \mathcal{K}_j, \quad \mathcal{N} \subseteq \mathcal{I}_k(t_s), \quad |\mathcal{N}| = n_j$$

$$P_k(t_s) + |\mathcal{N}| \; P_{\mathrm{dyn},k}(f) \leq B_k(t_s)$$

The first conditions prevent a job from starting before its arrival, running on incompatible hardware or sharing nodes with another allocation, while the final condition ensures that adding its dynamic power does not exceed the partition's compute budget. Idle-node demand is already included in $P_k(t_s)$, so only the dynamic contribution associated with the new job and its frequency is added. If an explicitly configured grid-support rule is active for a job approaching its waiting bound, the effective compute budget is increased before applying this same feasibility test.

3.6.2 Scheduling policies and queue coordination

Within a partition, the scheduling policy proposes assignments using the jobs and resources available at dispatch, as summarized in Figure 16. The common selection interface receives energy availability through the budget, while policies that follow an energy plan obtain it through a reference to the planner supplied when they are configured.

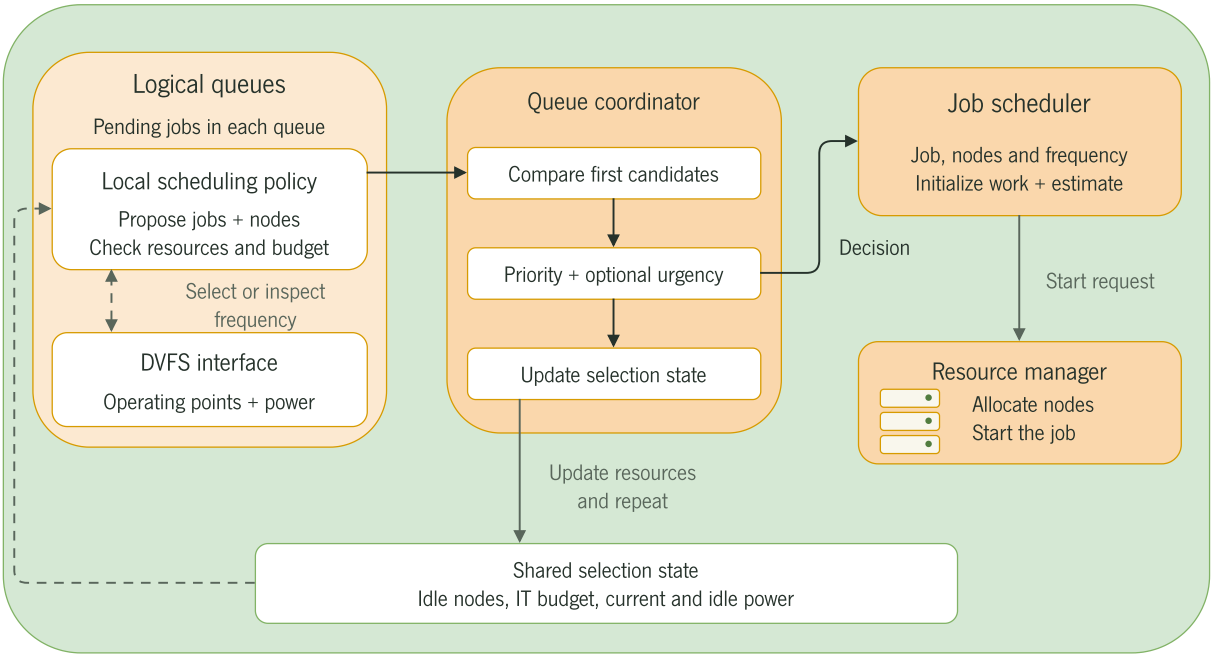


Figure 16: From local scheduling policies and queue coordination to dispatch through the resource manager.

The DVFS interface exposes the supported operating points and can select a frequency using the inputs in Figure 16. A scheduling policy may delegate this choice or compare the supported frequencies itself

when evaluating jobs, so frequency selection can be studied separately or combined with job ordering. The selected frequency still has to pass the dispatch feasibility test.

Logical queues share the nodes and compute budget of their partition, with a coordinator asking each queue's local policy for its first proposed decision. It chooses among these candidates using queue and job priorities, together with waiting-bound urgency when enforcement is enabled, then removes the selected nodes and updates the power available for further choices. Repeating this process lets queues use different local policies while competing for the same resources, without assigning each queue a separate power budget. Figure 16 summarizes this coordination over the local policies.

The job scheduler applies each decision by initializing the job's execution state and runtime estimate, then asking the resource manager to start it (Figure 16).

3.6.3 Runtime estimates

The runtime estimate available to a scheduling policy is kept separate from the work that determines when a job actually finishes. The general prediction model starts from the nominal duration $T_j(f)$ at frequency f and applies a configurable factor ε_j:

$$\hat{T}_j(f) = \varepsilon_j T_j(f)$$

When the prediction-error rate ρ is zero, $\varepsilon_j = 1$ and the estimate equals the nominal duration at that frequency, before contention and any later frequency changes affect execution. Otherwise, the factor is obtained from a uniformly sampled perturbation, bounded below by zero:

$$\varepsilon_j = \max\bigl(0, 1 + U_j\bigr), \quad U_j \sim \mathcal{U}(-\rho, \rho)$$

A factor below one underestimates the nominal duration, while a factor above one overestimates it. The factor is generated from the prediction seed and job identifier, so repeated estimates for the same job use the same error across frequencies and policies when the error rate and seed are held fixed.

Prediction error changes neither compute work nor execution overhead, so nodes are released when execution finishes, not when the predicted duration elapses.

In the controlled examples of Figure 17, execution duration is fixed to isolate prediction error. For long-horizon planning, an underestimate can delay later jobs and extend demand into another energy window, while an overestimate can reserve resources longer than necessary unless the plan is updated after

completion. In a simulation, contention and frequency changes also affect when those resources become available.

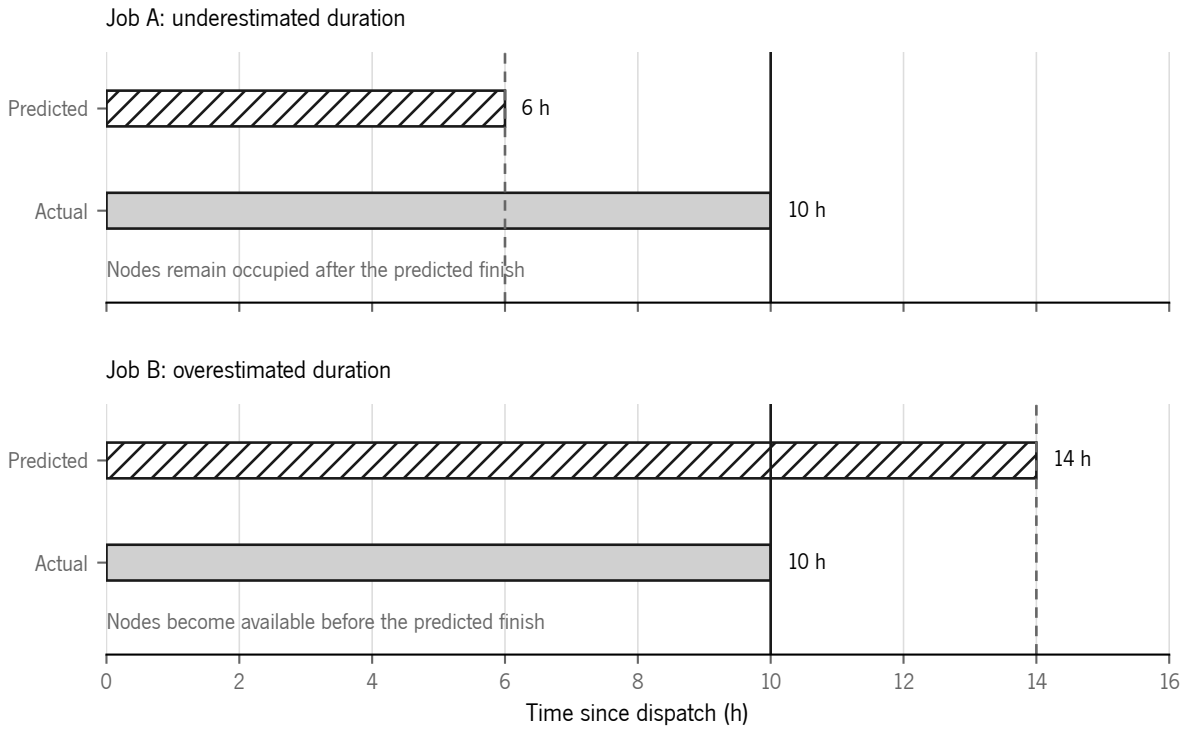


Figure 17: Illustrative prediction errors for a 10 h job, with constant frequency, no contention and immediate completion recognition.

3.6.4 Service constraints and budget enforcement

Waiting for more favorable energy conditions leaves jobs in the queue for longer, so WAVE allows each queue to set a maximum waiting time measured from submission. When this limit is used to guide scheduling, jobs that have already reached it are considered first, followed by those with less time left to wait, before the policy's usual ordering takes effect. The same urgency is considered when comparing candidates from different logical queues. The waiting limit can also be used only to record deadline misses, without changing the order in which jobs are selected.

Giving a job priority does not ensure that it can start if the compute budget is too small. An optional grid-support rule therefore increases the effective budget as jobs approach their waiting limit, adding the dynamic power they would need at the highest supported frequency. This extra allowance is used both when powering resources on and when checking dispatch, although the job still needs enough available nodes and may consequently miss its deadline despite the additional power.

Once a job starts, it keeps its allocation even if the compute budget later falls, since the execution model does not pause running jobs to bring demand back within the new allowance. Where runtime frequency

changes are enabled, the resource manager lowers operating frequencies to reduce demand, while unused nodes are switched off as described in the resource-management model. If these adjustments are insufficient, execution continues above the assigned budget and the grid supplies any remaining deficit in grid-connected operation. The budget therefore limits which jobs can start, but jobs already running may continue to draw more power than it allows.

When runtime frequency increases are enabled, the resource manager can also raise the frequency of running jobs if no jobs are pending and the additional power fits within the current budget. Spare capacity can therefore shorten ongoing execution without taking power away from waiting work.

Together, Figure 4 and Figure 16 connect budget generation to dispatch.

3.7 Simulation timing and adaptive stepping

3.7.1 Coupled update cycle

WAVE and Vessim share a simulation clock, but job scheduling, energy-budget generation and the microgrid have separate update intervals. At each scheduling update, running jobs advance through the elapsed interval and completed jobs release their nodes before new work is selected. The scheduler then considers jobs whose submission time has been reached, using the available resources and the current compute budget.

Changes to allocations and frequencies affect subsequent execution and the power demand presented to Vessim. The microgrid uses this demand to update storage and grid exchange on its own schedule, while the energy scheduler periodically revises the compute budgets using the available energy and battery state.

3.7.2 Adaptive stepping

A fixed scheduling interval can delay resource release when a job finishes between updates. In Figure 18, Job A requires six seconds of execution, but four-second updates keep its nodes allocated until the eighth second. An update at completion allows the waiting Job B to start earlier, without shortening every scheduling interval.

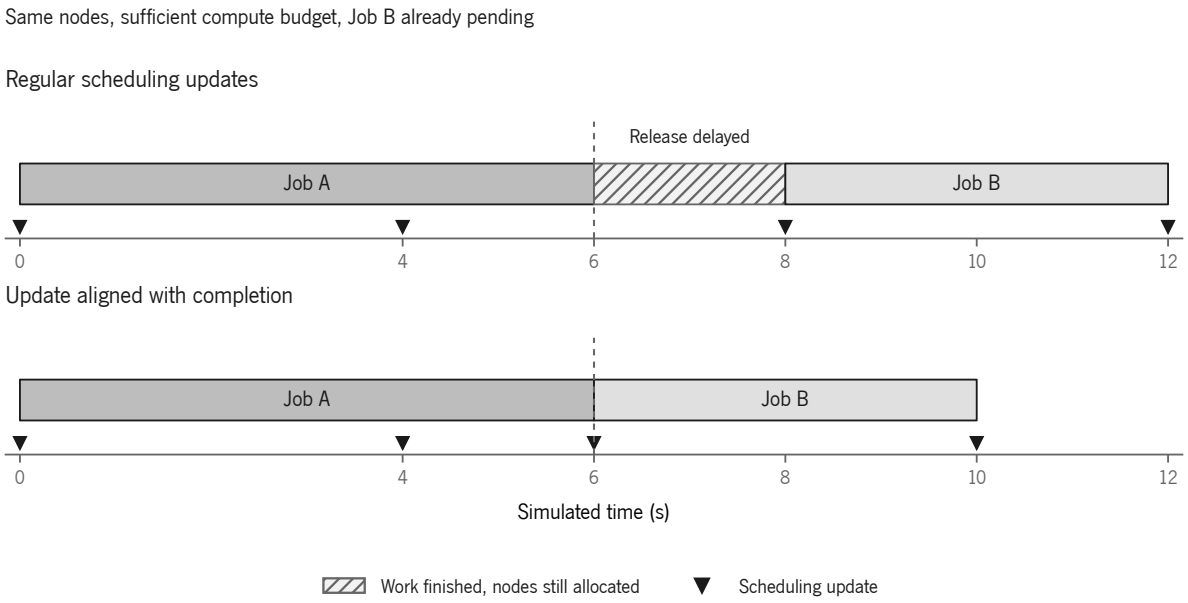


Figure 18: Earlier resource release with a scheduling update aligned with job completion.

With adaptive stepping enabled, WAVE selects the shortest positive remaining time among running jobs, rounds it up to whole seconds and limits the next interval to the configured scheduling interval, with a minimum of one second. If no such remaining time exists, it uses the configured interval. This timing estimate comes from remaining work at the assigned frequency, not the runtime prediction supplied to the scheduling policy, and does not account for future contention, so the job may still have work left at the next update.

Vessim adds the selected interval to the current simulation time and passes the next update time to Mosaik. Future arrivals do not shorten this interval, so a newly submitted job waits until the next scheduling update to be considered. Energy-budget and microgrid updates retain their own timing, allowing the available budget to change between scheduling decisions without triggering an immediate dispatch.

3.8 Monitoring and simulation output

The Vessim monitor records power and storage state at each monitoring update, alongside grid energy exchanged over the preceding interval and the carbon contribution described in Section 3.4.3. WAVE adds partition-level records of node utilization, job counts and compute budgets to this time series, using the monitor interface shown in Figure 14. Scheduler energy accumulators are also recorded, but integrate IT demand at scheduling updates rather than measuring facility energy or physical grid import.

Job-level output complements these measurements with waiting and execution times, runtime estimates and frequency changes. The metrics table includes only completed jobs, while the saved scheduler state retains the full workload, including failed and unfinished jobs. When event logging is enabled, a separate trace records scheduling actions and their context, allowing changes in demand to be examined alongside the decisions that produced them.

With automatic stopping enabled, a partition finishes once it has no pending, future or running jobs and no nodes still recovering from failure. The energy scheduler ends the run when all partitions have finished, although the simulation can also be bounded by a configured end time. A user interruption or an energy deficit that cannot be supplied in islanded operation causes the runner to save partial results, which describe only the recorded portion of the run and must not be treated as a completed workload.

Chapter 4

The Puzzle Scheduler

4.1 Scheduling problem and overview

The Puzzle scheduler uses a forecast of available power to decide when each job starts and at which frequency it runs. It may delay a job until more renewable energy is available or increase its frequency so that it finishes before that supply falls, while reserving enough nodes and respecting the allowed waiting time where resources permit. If renewable generation and the battery cannot cover the demand, the plan may also assign grid power.

Figure 19 shows an illustrative plan, with the lower panel marking each job's start, reserved duration and frequency and the upper panel showing the combined power demand. Labels beginning with A and R distinguish the ARM and EPYC Rome partitions, while green marks demand assigned to renewables and the battery and orange marks demand assigned to the grid.

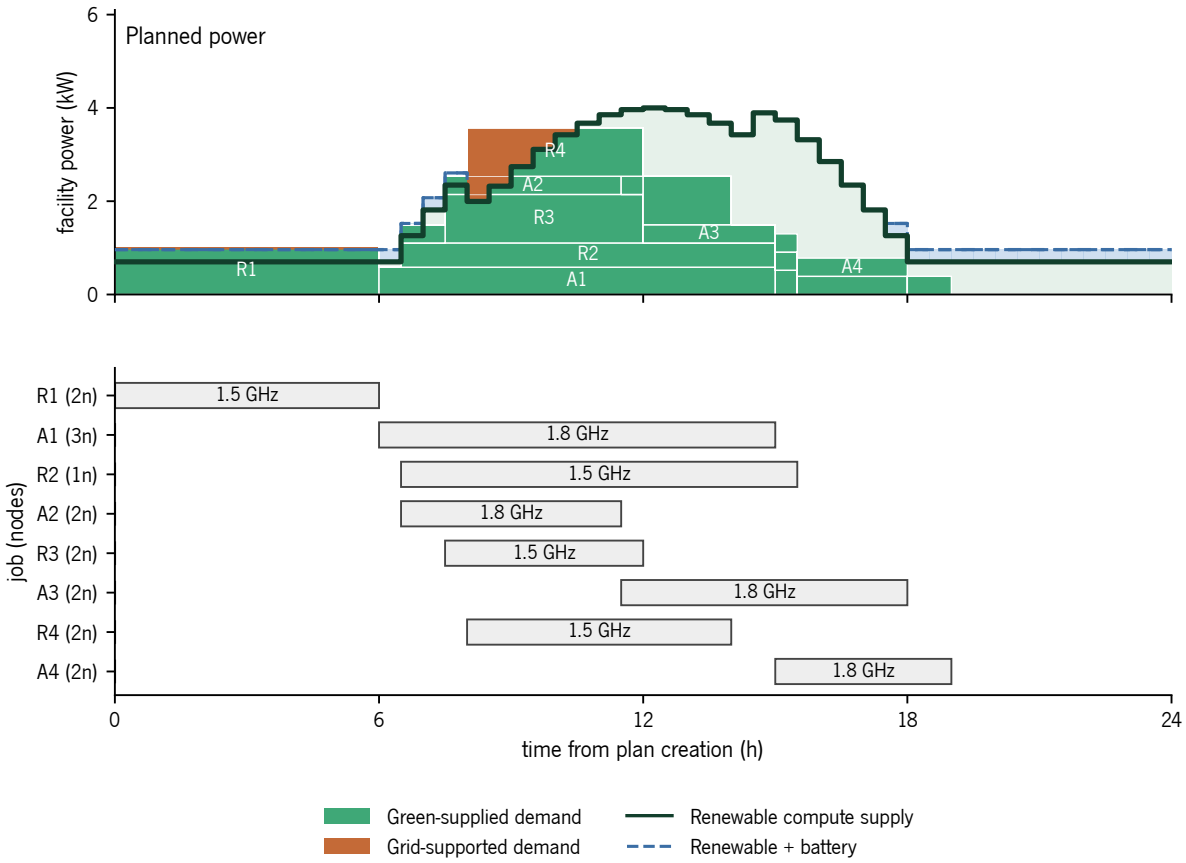


Figure 19: Illustrative 24-hour plan for jobs submitted at plan creation. Bars show reserved execution intervals.

As execution changes job progress and battery state, the scheduler rebuilds the plan from the jobs submitted so far, keeping running jobs in place while reconsidering those still waiting. It reads 24-hour renewable-generation and grid-carbon-intensity forecasts from the same traces used during execution, so these signals are known exactly within that window in the experiments. Future job arrivals remain unknown to the planner, while jobs that run longer depend on the extrapolation described in Section 4.4.

4.2 Representing jobs and available resources

4.2.1 Jobs as pieces

Puzzle represents a job as a rectangular piece whose width is its estimated runtime and whose height is the facility power that the planner must reserve for it at a given frequency. Both values come from the execution and power models in Chapter 3.

When all nodes in a partition remain powered on, Puzzle reserves their idle demand before placing jobs. Each piece then contains only the allocated nodes' dynamic power, scaled by PUE. If idle demand is not reserved separately, the piece includes the allocated nodes' idle draw as well.

The area of the piece gives the predicted energy associated with this reservation at frequency f:

$$E_j(f) = P_j(f)T_j(f).$$

Here, $P_j(f)$ is facility power in kW and $T_j(f)$ is predicted duration in hours, so $E_j(f)$ is expressed in kWh. With idle demand reserved separately, this is the job's incremental energy above that demand.

Each supported frequency gives the same job a different piece. In Figure 20, the lower frequency uses less energy but makes the job too long for the renewable peak, whereas the higher frequency produces a shorter, taller piece that fits inside it. Puzzle can therefore spend more energy in total to avoid drawing power from the grid.

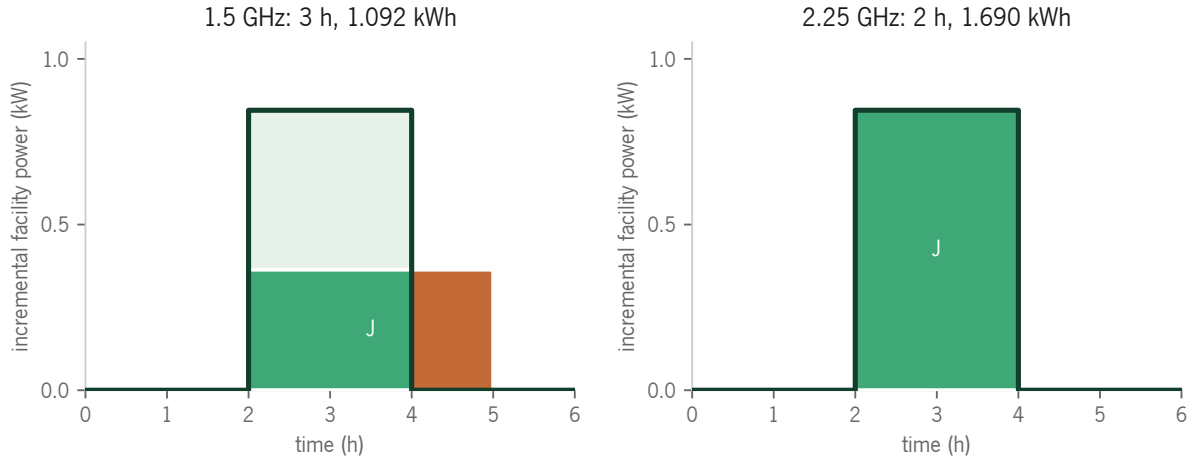


Figure 20: Frequency alternatives for a compute-only job on one EPYC Rome node, with PUE 1.3 and idle demand reserved separately. Orange marks the job's grid demand.

The configurable slot duration sets the plan's temporal resolution, since shorter slots allow starts closer together and reduce the rounding of predicted runtimes. With five-minute slots, a job expected to run for twelve minutes occupies three slots and reserves its nodes and power for fifteen minutes, although execution may release them sooner if the job finishes early.

Puzzle takes planned durations directly from WAVE's execution model without applying the prediction-error factor in Section 3.6.3. Varying that factor alone therefore does not test uncertainty in Puzzle's

reservations, although shared-resource contention or frequency changes can still make a job finish earlier or later than planned.

4.2.2 Power available to jobs

Within the forecast window, the board makes green power available for placing jobs in each slot s according to

$$G_s = R_s - C_s + B_s.$$

Here, R_s is forecast renewable power, C_s is the power reserved for charging, and B_s is planned battery discharge, all measured at facility level. Under the study's charging assumptions, stored energy counts as green, while demand beyond G_s is considered separately for grid supply.

Puzzle reserves renewable power for charging where estimated job demand leaves a surplus, making it unavailable for placement. It counts that energy as stored only when a later plan observes it in the battery, whose limits follow the model in Chapter 3.

Puzzle shares this board across partitions because earlier versions divided green power into fixed shares before placing jobs. When supply was scarce, each share could be too small to start a job on green power even if the total could support one, whereas a shared board leaves that power available to any partition with free nodes. Node capacity itself remains separate, so an ARM job cannot use free Rome nodes, and Puzzle derives each partition's power plan from the jobs placed there instead of setting quotas in advance.

4.2.3 When a job can start

A submitted job can be placed from the start of the current plan, while a queue with a maximum waiting time also gives it a latest start after Puzzle subtracts a safety margin for execution delays. If that latest start has already passed, the search begins at the earliest currently possible slot.

Before placing waiting jobs, Puzzle reserves the always-on idle draw, followed by the nodes and power needed for the remaining execution of running jobs, using grid power if green supply cannot keep them running. Since running jobs cannot be paused, each new placement must also have enough free nodes in its own partition throughout its planned duration.

4.3 Constructing the schedule

4.3.1 Placing jobs

Puzzle places waiting jobs in order of the time left until their latest start, leaving those without a waiting bound until last and giving the job with higher predicted energy priority when two have the same remaining time. For each job, the planner tries frequency alternatives in order of increasing energy and then duration, searching from the earliest allowed start for consecutive slots with enough remaining green and carbon-permitted grid power, together with free nodes. Once an alternative fits, its first feasible placement reserves those resources and the planner moves on, even if a higher frequency could have started the job earlier.

In Figure 20, grid power is excluded and the alternatives that use less energy last beyond the renewable peak, so the planner selects the higher frequency that fits. The same search must also account for earlier reservations in the full plan of Figure 19, as summarized in Algorithm 1.

Algorithm 1: Construction of a Puzzle plan.

```
procedure BuildPlan(t, submitted jobs, running jobs, forecast, battery, partition nodes)
    waiting ← pending submitted jobs with submission time at or before t
    board ← shared power slots, extended if a submitted job outlasts the forecast
    reserve idle draw, then running jobs' power and nodes
    order waiting jobs by time left to start, then decreasing energy
    for each job j in waiting do
        for each frequency, from lowest to highest energy do
            placement ← earliest start fitting green, permitted grid and free partition nodes
            if placement exists then break
        end
        if no normal placement exists then
            placement ← exceptional grid search under carbon and waiting rules
        end
        if placement exists then
            reserve its power and partition nodes
        else
            keep the job waiting for the next plan
        end
    end
    if repair is enabled then try moving placed jobs to reduce grid use
    derive per-partition power plans from the placements
    return placements and per-partition power plans
```

4.3.2 Using grid power

A placement can draw from green power and the grid within the same slot. Puzzle estimates grid emissions for a placement of job j as

$$C_j = \sum_{s \in S_j} \max\left(0, P_{j(f)} - A_s\right) \Delta t_s I_s.$$

Here, S_j contains the occupied slots, A_s is green power left after earlier reservations, Δt_s is slot duration in hours, and I_s is forecast grid carbon intensity in gCO_2/kWh. With power in kW, the score C_j is expressed in grams of CO_2.

The normal search permits a configured amount of grid power in slots whose carbon intensity does not exceed a threshold, deducting grid already reserved for idle draw, running jobs and earlier placements. Within a selected slot, green power is consumed first and only the shortfall counts against the grid limit. An earlier admissible mixed placement can therefore be chosen before a later fully green placement, without exceeding the remaining allowance in any occupied slot.

Figure 21 applies this limit to the same job under different renewable supplies: the first placement fits the grid allowance and the second exceeds it, while high carbon intensity removes the allowance in the third even though the grid could physically supply the shortfall.

Figure 21: Illustrative placements under the normal grid allowance. The dashed line includes permitted grid power.

4.3.3 Deciding how long to wait

When no normal placement fits, Puzzle's carbon-regret rule compares the earliest start with enough nodes against the start with the lowest carbon score, considering both without the normal grid restriction. It chooses the earlier start, possibly using more grid power, when the predicted emissions saved per hour

of additional waiting do not exceed a configured threshold. This threshold rises as the job ages, so a small predicted saving becomes less reason to delay it again.

Let C_e and C_c be predicted grid emissions in grams at the earliest and cleanest starts, separated by Δt hours, and let w be the time the job has already waited. With H denoting the forecast length in hours and θ the configured threshold in gCO_2/h, Puzzle chooses the earlier start when

$$\frac{C_e - C_c}{\Delta t} \leq \theta\left(1 + \frac{w}{H}\right), \quad \Delta t > 0.$$

Figure 22 keeps the two candidate starts fixed to show where the rising threshold overtakes a given saving rate. When both choices start at the same time, Puzzle uses a zero saving rate.

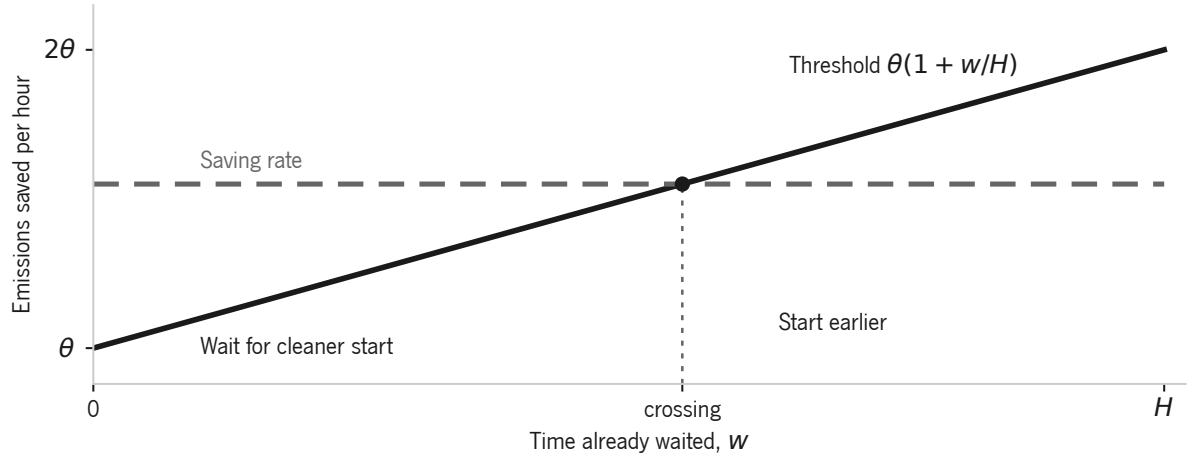


Figure 22: Illustrative crossing of the saving rate and the carbon-regret threshold.

For the remaining cases, Puzzle selects the lowest-carbon placement that meets the normal grid conditions. If none exists and the latest allowed start lies inside the forecast window, it can relax those conditions, first seeking a start within the timing bound and then looking later if necessary. A job with no feasible placement remains waiting for another plan, and relaxing the grid limit cannot help when nodes are unavailable.

An optional, bounded repair pass tries to move jobs assigned grid power onto green power, either directly or by relocating a job that already has a green placement. It retains timing and node constraints and is disabled in the default configuration.

In Figure 23, A can move to the next slot and free green power for B, which must start in the first. Both jobs are still waiting for execution, so this repair changes their reservations without interrupting work.

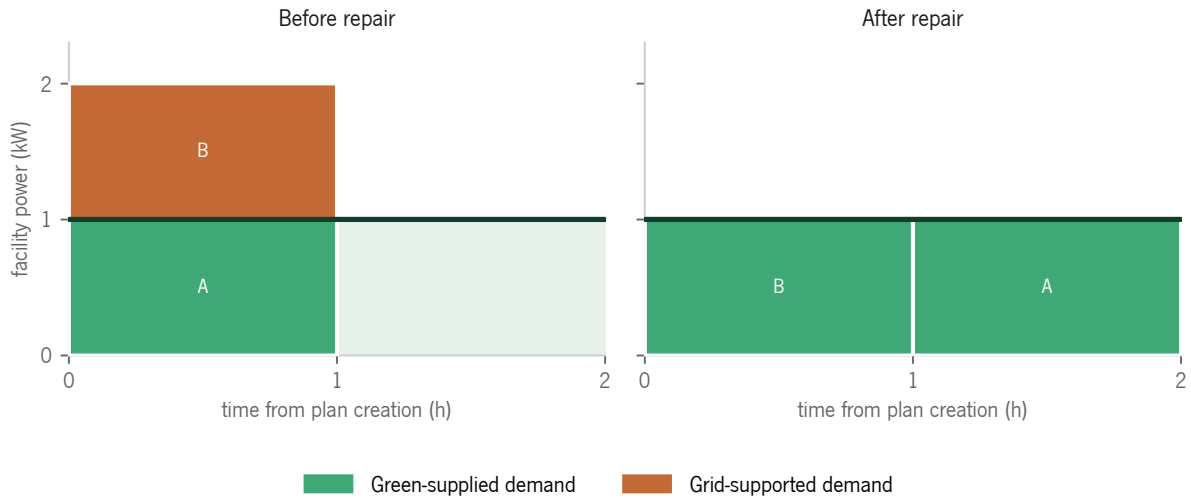


Figure 23: Illustrative placement before and after repair moves A to supply both jobs from green power.

4.4 Jobs that run beyond the forecast

4.4.1 Extending the plan

The long job L in Figure 24 cannot finish within the 24-hour forecast even at its highest frequency, so a board confined to that window would leave it waiting indefinitely. Puzzle extends the board to let L start inside the known window while its execution continues beyond it. The other jobs come from Figure 19, but L now competes for their nodes and power, which can change their starts and frequencies. In this example, Puzzle estimates later renewable supply by repeating the known profile without repeating the energy stored in the battery.

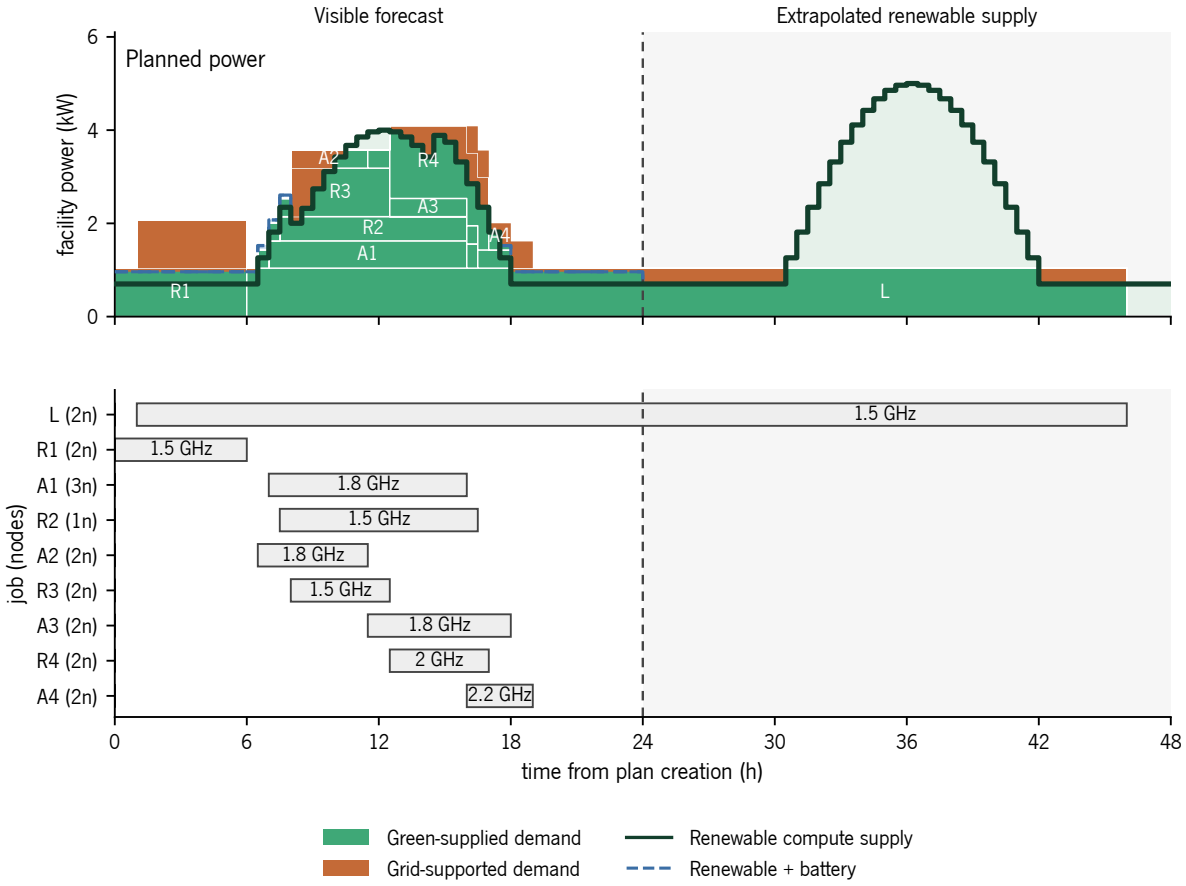


Figure 24: Illustrative plan extended to 48 hours after adding job L. The dashed boundary at 24 hours separates the visible forecast from extrapolation.

Puzzle extends the board only when no frequency alternative would let a job finish inside the forecast from its earliest start. Since a new plan contains only submitted jobs, this identifies work whose shortest available duration exceeds the forecast length. Every job must still start inside the known window, and those that can also finish there must do so, preventing extrapolation from becoming a way to defer other waiting work.

Beyond the forecast window, Puzzle repeats the visible renewable profile and estimates how much later generation could fall below or rise above it, as illustrated for one slot in Figure 25. The robustness setting controls how far the renewable power available for placing jobs moves towards the pessimistic estimate, while the optimistic estimate is recorded for inspection only. Puzzle derives these margins by comparing earlier and later periods of observed generation, using a configured fraction when there is too little history, and does not let them shrink for more distant periods.

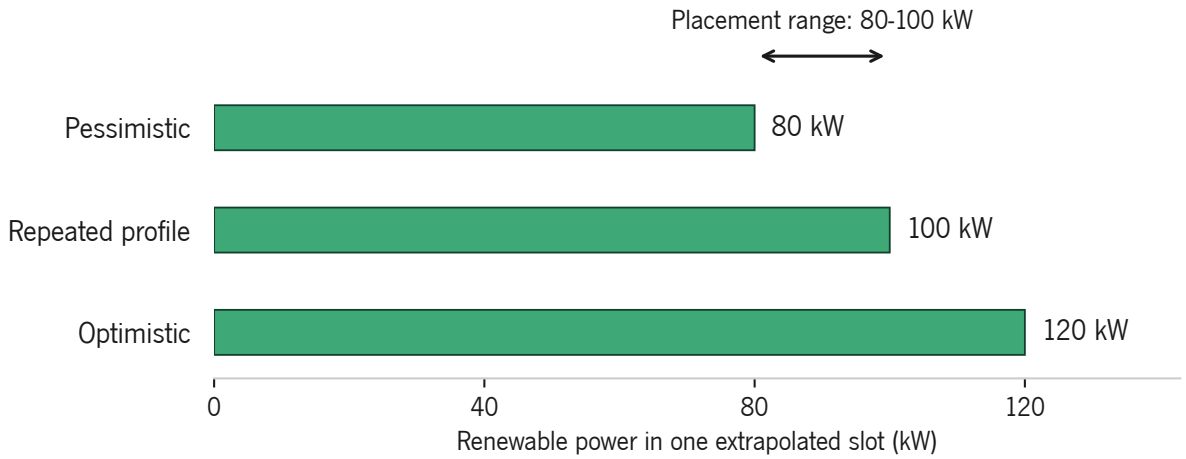


Figure 25: Illustrative renewable estimates with a 20% margin in either direction for one extrapolated slot.

In Figure 24, Puzzle uses the repeated profile without a robustness reduction, while battery energy and charging reservations are not repeated because that would count the same stored energy twice. Puzzle also repeats grid carbon intensity beyond the window but adds a penalty to the carbon score of those slots, making them less attractive without changing emissions measured during execution. The plan still expires at the end of the real forecast, when its assumptions must be reconsidered.

4.4.2 Saving battery energy for running jobs

Once L has started, a later plan cannot move it to a cleaner period when the renewable forecast changes, so Puzzle marks jobs extending beyond the known window as protected across updates. Running protected jobs then receive priority for battery support wherever their demand exceeds forecast renewable supply.

For protected running jobs whose execution still extends beyond the current window, Puzzle estimates later demand from their current power and remaining duration, limited by what the battery can deliver. It reserves a configured fraction of the jobs' full demand in that unknown period rather than only the expected shortfall in renewable supply. Waiting jobs cannot claim this reserve, and a running job stops contributing once its remaining work fits inside the current window.

Within the visible window, protected running jobs receive battery energy first where their demand exceeds renewable supply, with discharge directed to the periods of highest grid carbon intensity in that group. Energy left afterward supports other jobs across slots not reserved for charging, optionally favoring higher-carbon periods, while a protected job can draw battery power even in a slot previously reserved for charging. The reserve fraction controls how much energy is saved for later, whereas the robustness setting controls the renewable supply assumed beyond the forecast.

4.5 From the plan to execution

4.5.1 Distributing power and starting jobs

Figure 26 follows the plan through WAVE's energy and job schedulers, which may delay a reserved start when nodes or power are unavailable and feed that delay back into planning.

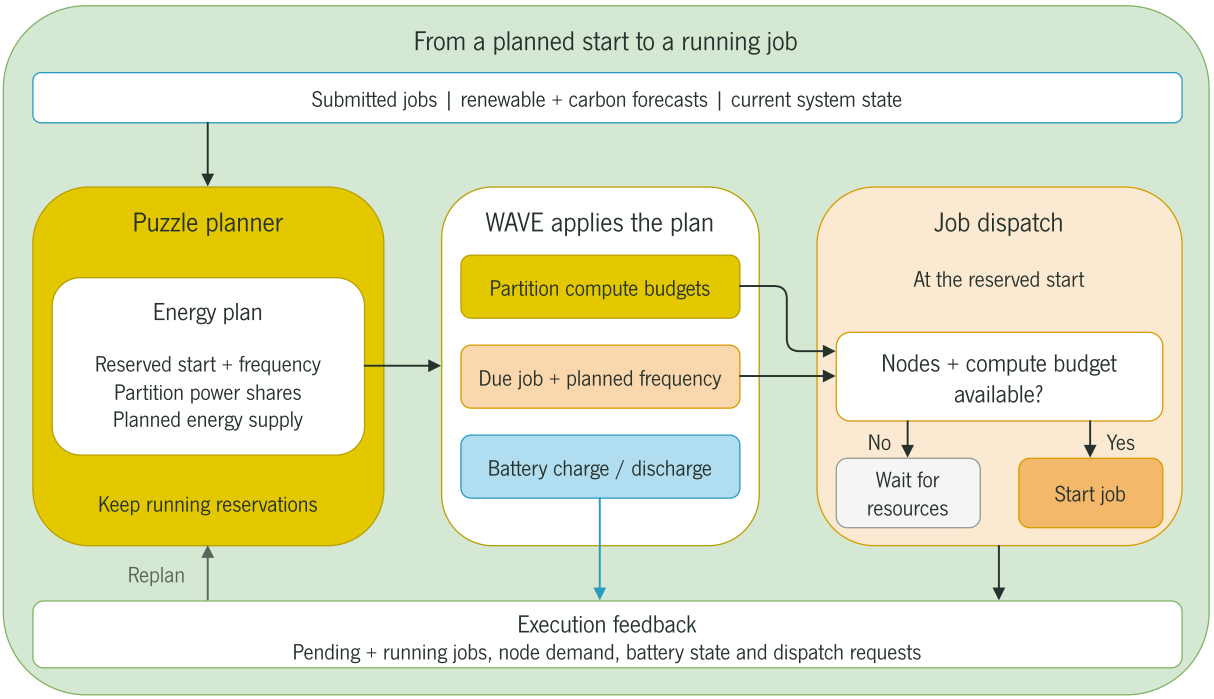


Figure 26: Applying the energy plan through compute budgets, dispatch checks and battery controls.

Puzzle first covers the measured demand of running jobs in each active partition, then distributes remaining power towards the shares implied by the planned placements, converting facility power to IT compute budgets as described in Section 3.5. With the resulting budget, the job scheduler starts a due job at its planned frequency only if enough nodes are ready and the partition can support it. Otherwise, the job remains waiting and can request a new plan. In the default carbon-aware configuration, Puzzle

does not use spare capacity to start unplanned jobs or advance planned starts, although both behaviors can be enabled for comparison.

4.5.2 Updating the plan

At each energy scheduler update, Puzzle checks for new submissions and, when jobs have arrived since the last plan, rebuilds it with all currently waiting jobs. Arrivals observed together bypass the regular refresh interval and the minimum interval for dispatch requests, while running jobs retain reservations updated to their progress and waiting jobs may receive new starts and frequencies. Otherwise, the scheduler rebuilds at regular intervals, when the forecast expires or after a dispatch request once its minimum interval has passed, with an optional trigger for persistently low planned use of available power.

Between updates, the compute budget retains the planned renewable supply but limits battery discharge to the energy and power actually available in storage. If that supply no longer covers running demand, Puzzle increases grid support to keep work going, so the grid limit used when selecting new starts does not cap grid use throughout execution.

WAVE advances the jobs and Vessim determines their resulting energy exchange, which may include grid import even for a job planned to run entirely on green power. The monitor records this exchange and its emissions for the evaluation.

4.6 Planning cost and limitations

Planning cost grows with the number of submitted jobs, slots and frequency alternatives: for J jobs, S slots and V alternatives per job, the main search scales as $O(JSV)$, with $O(J \log J)$ to order the jobs. Puzzle reuses accumulated energy and carbon sums and precomputed slot checks rather than scanning a job's full duration at every possible start, although extending the board adds slots and optional repair adds attempts to move placed jobs. Observed construction times for the evaluated configurations appear in Table 6.

Frequent submissions increase total planning cost because each arrival update rebuilds the plan for waiting jobs, even though the grid search skips starts a job cannot reach and reuses node checks across its frequency alternatives. To limit memory use without changing placement decisions, Puzzle retains detailed past plans in a bounded sample while keeping compact summaries and the sequence of active plan intervals for the full execution.

Larger slots reduce planning cost but space possible starts farther apart and round durations more coarsely, which may rule out placements that fit at finer resolution. Puzzle also places jobs one by one without examining every combination, so a choice that suits the current job may take renewable power from a later one and cannot guarantee the lowest total emissions.

Chapter 5

Evaluation

Scheduling decisions affect both when jobs finish and how the facility uses energy. We evaluate these effects together, comparing waiting times and workload completion with energy use, grid import and operational emissions. The comparison follows the same workload through grid supply, Photovoltaic (PV), and PV with wind, placing the Puzzle scheduler alongside FIFO, Priority and Earliest Finish Time (EFT) in each case. We then examine smaller workloads and changes to the energy policies to understand how the observed outcomes depend on the experimental conditions.

5.1 Experimental setup

5.1.1 Workload and computing system

We used a synthetic workload of 2,523 jobs for the modeled ARM, x86 and GPU partitions, with arrivals spread over 96 h. Each job specifies its arrival, partition, node request, priority and execution attributes. These records remain identical across the main comparisons, so changes in the outcomes follow the simulated configurations rather than different input jobs.

Most jobs have short computation times, with a small group of longer jobs extending the range in Figure 27. The mean is 1.818 h, with 40 jobs above 6 h and eight at or above 24 h. These times are calculated from CPI and instruction count at base frequency. Memory and network overheads, together with frequency changes, can make execution longer.

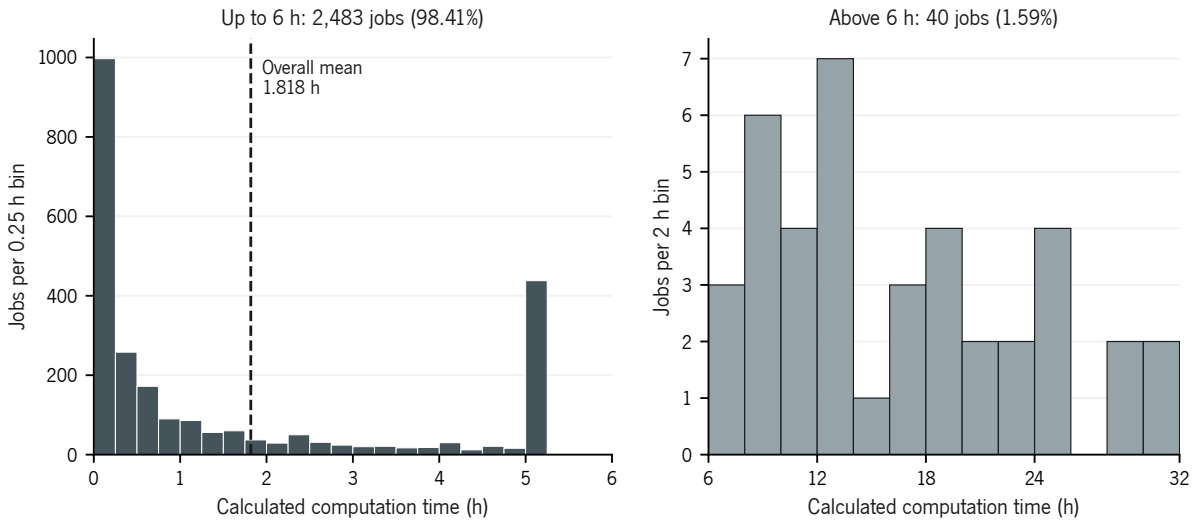


Figure 27: Calculated computation times at base frequency, with the sparse tail shown separately.

The workload combines short jobs arriving throughout the submission window with a randomly sampled group of longer computations. Jobs exceeding 6 h are restricted to arrivals within the first 48 h. The sampling procedure and the distribution across partitions and queues are described in Section A.2.

All 1,632 ARM nodes, 500 x86 nodes and 33 GPU nodes remain powered on throughout each run. Their idle powers are 40 W, 120 W and 450 W per node, giving a constant IT demand of 140.13 kW. With the configured PUE of 1.3, this corresponds to 182.17 kW at facility level before adding the dynamic power of running jobs. Delaying completion therefore has an energy cost even on partitions that have already finished their work.

5.1.2 Energy supply

The grid profile has no renewable generation and offers a 900 kW grid budget. The PV profile adds generation with a 400 kW peak, while the combined profile adds wind with a 400 kW peak. Both renewable profiles use a smaller grid budget of 350 kW and a carbon threshold of 250 gCO_2/kWh. These limits govern the power offered for scheduling. The physical grid connection can still supply demand that exceeds that budget.

The battery has a capacity of 2,000 kWh, starts with 1,000 kWh stored and operates between 20% and 80% state of charge. Grid charging is disabled in the policy comparisons. Figure 28 shows the renewable and carbon signals over 160 h from the common simulation origin, covering every main run. Energy and emissions are accumulated until each run finishes, so longer executions cover a longer part of these signals.

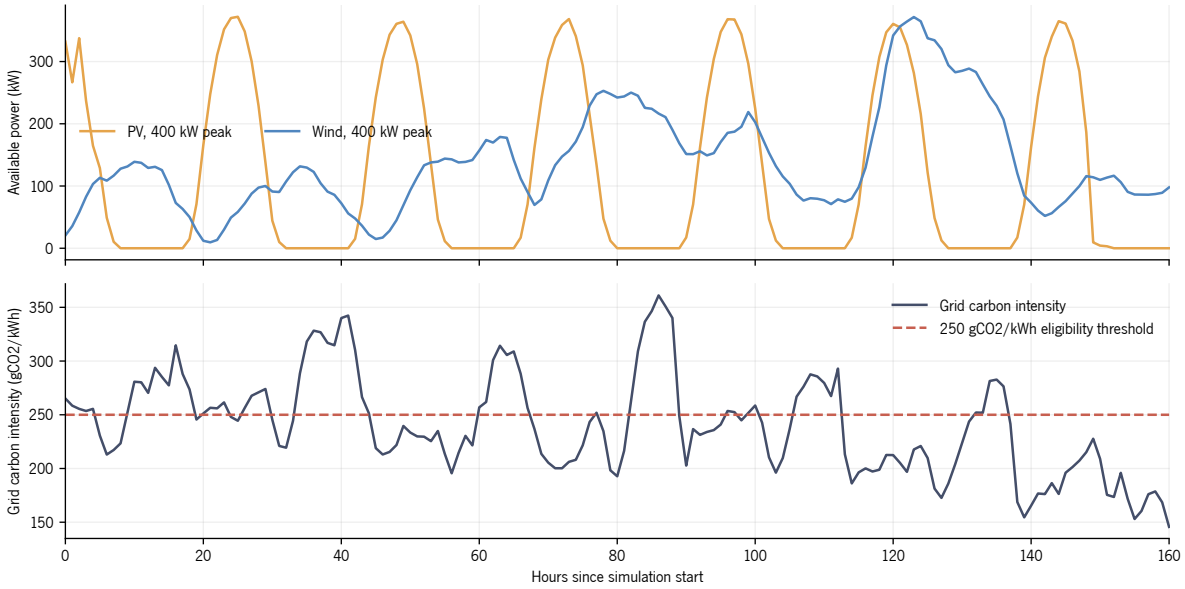


Figure 28: Available renewable power and grid carbon intensity over 160 h.

5.1.3 Scheduling configurations

FIFO follows arrival order without backfilling. Priority favors higher-priority jobs and can select another job when the first does not fit, while EFT combines a time and energy score for frequency selection with a preference for earlier predicted completion. In the experiments, all three first consider the urgency of the configured waiting bounds. Their individual selection rules apply after this shared ordering, which can reduce differences between them. A simplified illustration of the selection rules appears in Section A.3.

The energy policy determines how much power these schedulers can use. Grid offers a fixed grid allowance alongside any renewable generation, without testing carbon intensity. CarbonMinimizing reserves renewable power for charging and offers new grid budget only when carbon intensity is at or below the threshold. Its storage commands preserve the battery during low-carbon periods and daytime, allowing controlled discharge on higher-carbon nights. The compute-prioritized allocation rule first covers the demand of active partitions, then distributes any remaining budget to partitions with pending jobs.

Puzzle obtains job placements, source contributions and partition shares from the energy plan described in Chapter 4. Comparing it with the reference policies therefore changes job selection, budget generation and allocation together. Table 2 identifies the combinations used in the main comparisons. The full energy rules and allocation procedures are given in Section A.4.1 and Section A.4.2.

Supply	Job schedulers	Budget / allocation	Carbon threshold	Grid exception
Grid	FIFO, Priority, EFT	Grid / compute-prioritized	None	Wait override
Grid	Puzzle	Puzzle / Puzzle-plan	Disabled	Carbon test disabled
PV	FIFO, Priority, EFT	CarbonMinimizing / compute-prioritized	250 gCO_2/kWh	Wait override
PV	Puzzle	Puzzle / Puzzle-plan	250 gCO_2/kWh	Planner rules
PV + wind	FIFO, Priority, EFT	CarbonMinimizing / compute-prioritized	250 gCO_2/kWh	Wait override
PV + wind	Puzzle	Puzzle / Puzzle-plan	250 gCO_2/kWh	Planner rules

Table 2: Job and energy scheduling configurations in the main comparisons.

In the reference configurations, an external override makes additional grid power available when a job approaches its waiting bound. Puzzle disables this override and uses its own carbon-regret and emergency rules when carbon-aware placement is active (Section 4.3.2).

The Puzzle runs use the placement procedure described in Chapter 4, reserving idle demand before placing jobs and considering carbon-permitted grid power during the normal search when enabled. The Grid Puzzle run disables carbon-aware placement, while the PV and PV-and-wind runs enable it. These comparisons therefore connect supply profiles with their configured placement rules, rather than isolating the effect of energy supply alone.

5.1.4 Measures and execution checks

Waiting time is the interval from a job's arrival to its actual start, and we report its mean and 95th percentile (P95). A deadline miss is recorded when a scheduling update finds a job still waiting once it has reached its queue's waiting limit. Makespan measures the period from simulation start to the last job completion.

The queue settings in Table 3 apply to all three partitions. Each job enters a queue according to its requested wall-clock limit rather than its actual execution duration. We use these waiting limits to assess service in the experiments, without treating them as production commitments.

Queue	Requested limit (h)	Max. nodes per job	Wait limit (h)	Priority weight
short	0 to less than 24	32	48	3
medium	24 to less than 168	128	168	2
long	168 and above	128	336	1

Table 3: Modeled queue limits and priorities used in the experiments.

Puzzle subtracts a 6 h safety margin when calculating a job's latest planned start, leaving room for execution delays as described in Section 4.2.3. The reported deadline misses use the original queue bounds in Table 3, so this margin changes the planner's search rather than the criterion used to assess service.

IT energy is the electricity used by all compute nodes during the monitored run, calculated from the recorded partition power. Physical grid import is measured at the facility connection and includes PUE and storage exchanges. Operational emissions depend on both this imported energy and the carbon intensity at import time. Their ratio, the effective import intensity, indicates whether lower emissions came with less imported energy, cleaner imports, or both.

Run time measures the elapsed time needed to execute a simulation, excluding plot generation, while makespan measures time within the simulated system. For Puzzle, we also record the time spent constructing an energy plan, excluding history recording and the rest of the simulation. Both timing measures are reported in Section 5.3.4.

All 12 main configurations completed the same 2,523 jobs without deadline misses. Each was executed once with a fixed hash seed, adaptive DVFS and runtime estimates without added prediction error. The inputs, settings, source snapshots and execution commands are documented in Section A.1. Separate fixed-frequency and grid-charging runs illustrate those controls in the following section.

We checked the simulator using cases whose results can be calculated directly, covering job execution, facility power, grid import and operational emissions. The storage cases also tested charging and discharging losses, power limits and charge bounds. Table 4 compares the expected results with the values returned by the model.

Case	Expected	Observed	Unit
Job duration at 2 GHz	5.2	5.2	s
Adaptive completion at 2 GHz	10	10	s
Facility demand at PUE 1.2	1,200	1,200	W
Grid import over 600 s	200	200	Wh
Stored energy after charging	800	800	Wh
Stored energy after discharging	200	200	Wh
Operational emissions at 250 gCO_2/kWh	50	50	gCO_2

Table 4: Verification of execution, energy and emissions.

All displayed values agree with the expected results within an absolute tolerance of 10^{-9} in their stated units. The accompanying verification files give the inputs and numerical errors for each test. These checks establish internal consistency for the tested cases, while comparison with hardware measurements remains outside this evaluation.

5.2 Battery operation and frequency control

5.2.1 Battery operation

The battery supplies deficits and stores renewable surplus within its operating limits and the energy policy's storage commands. Enabling grid charging also allows imports to meet a charging command when renewable surplus is insufficient. Figure 29 compares FIFO runs with grid supply, PV, and PV with grid charging enabled.

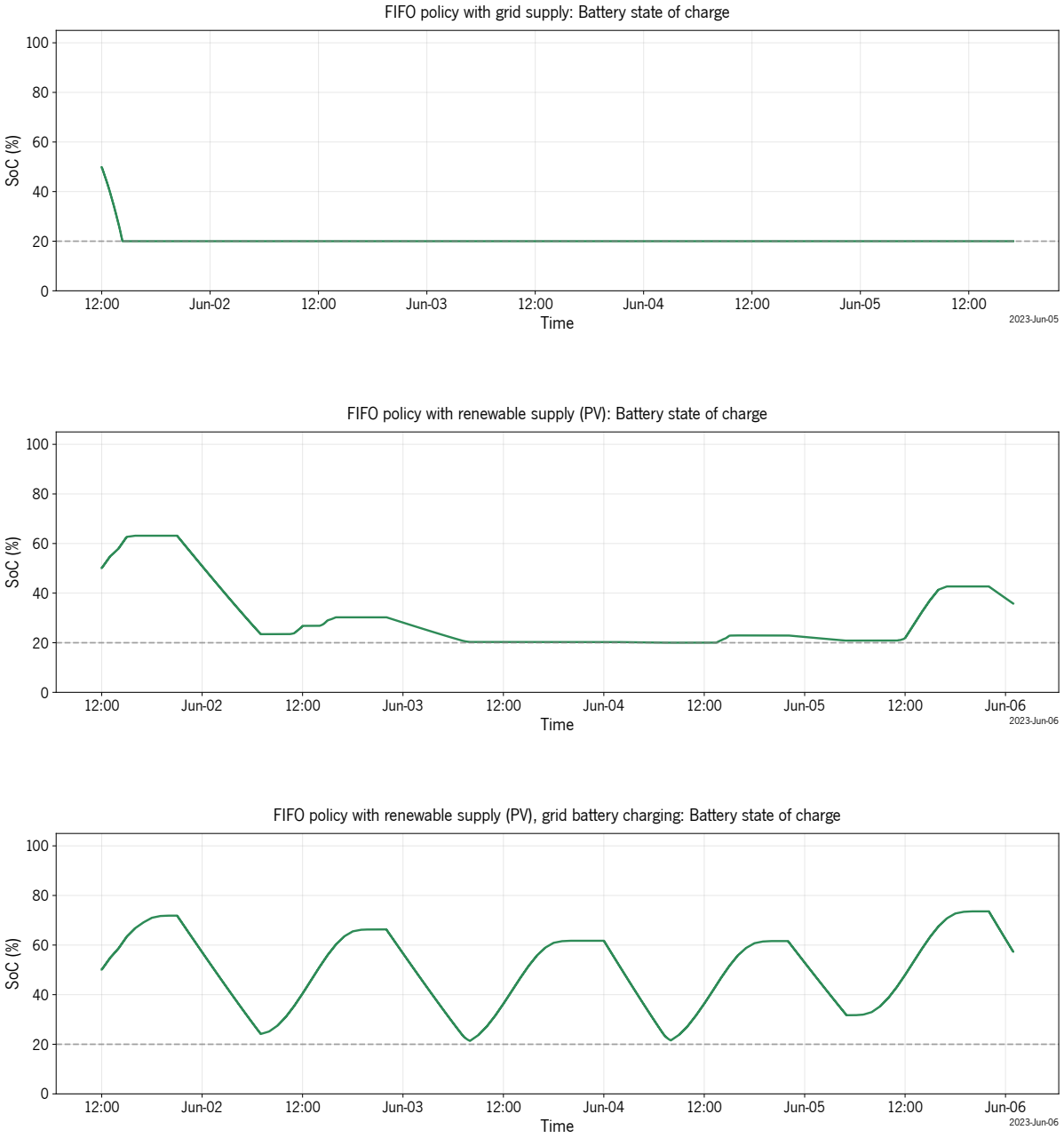


Figure 29: Battery state of charge with grid supply, PV, and PV with grid charging enabled.

With grid supply, the battery quickly reaches its minimum charge and remains there because there is no renewable generation to replenish it. PV allows some charging, but the battery still spends a long period near its lower limit. Grid charging produces more regular daily charge and discharge cycles and leaves 1,147.2 kWh stored at the end, against 716.4 kWh with PV alone. However, the two PV runs have the same waiting times and makespan, so the higher charge does not improve job service in this comparison.

5.2.2 Frequency control

Adaptive DVFS changes running-job frequencies as the partition budget changes, affecting both power and the remaining execution time. The matched FIFO runs with PV in Figure 30 and Figure 31 show this response beside fixed-frequency execution. The workload, supply and job-selection policy are the same in both runs.

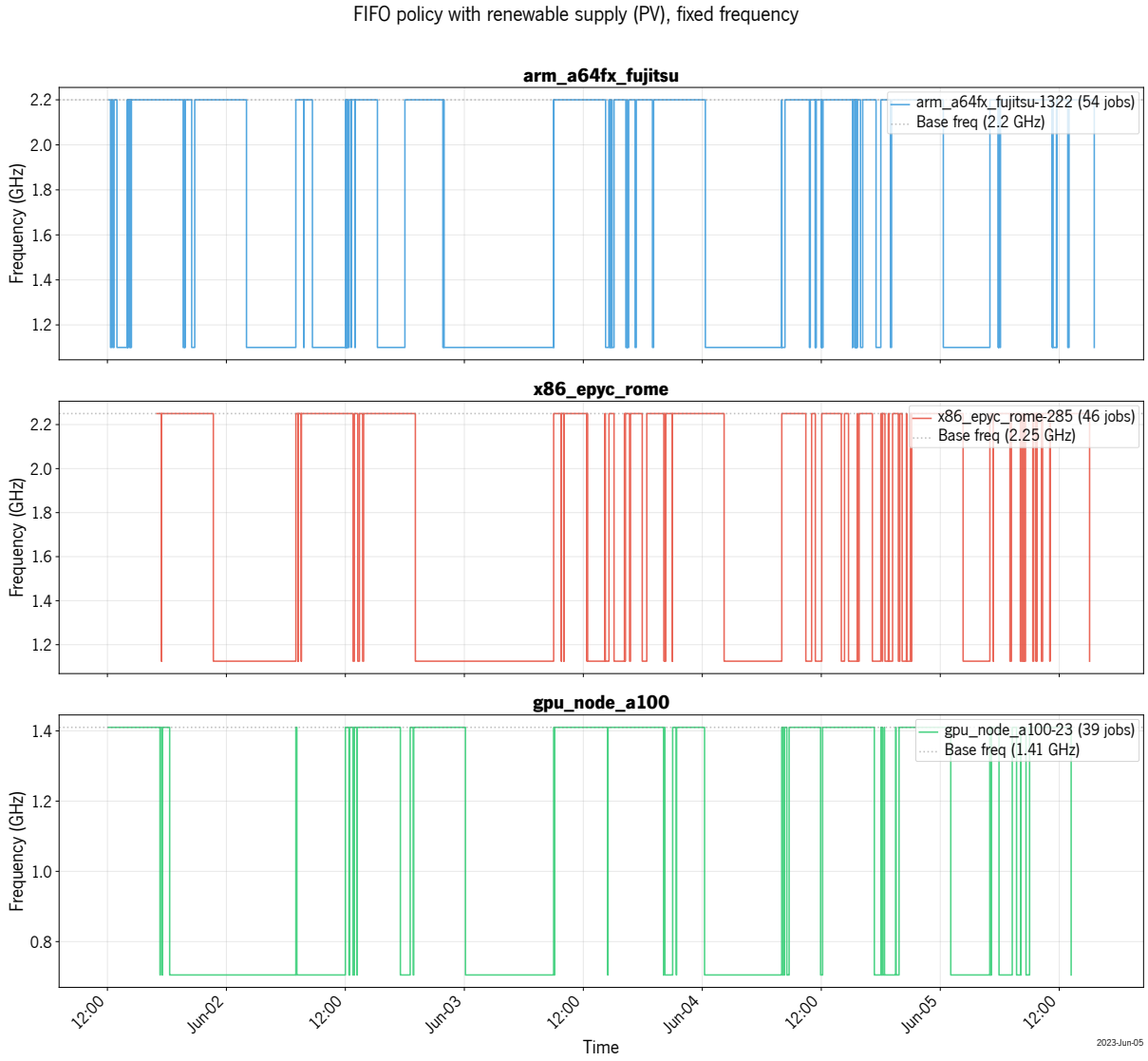


Figure 30: Busiest-node frequency timelines with fixed control.

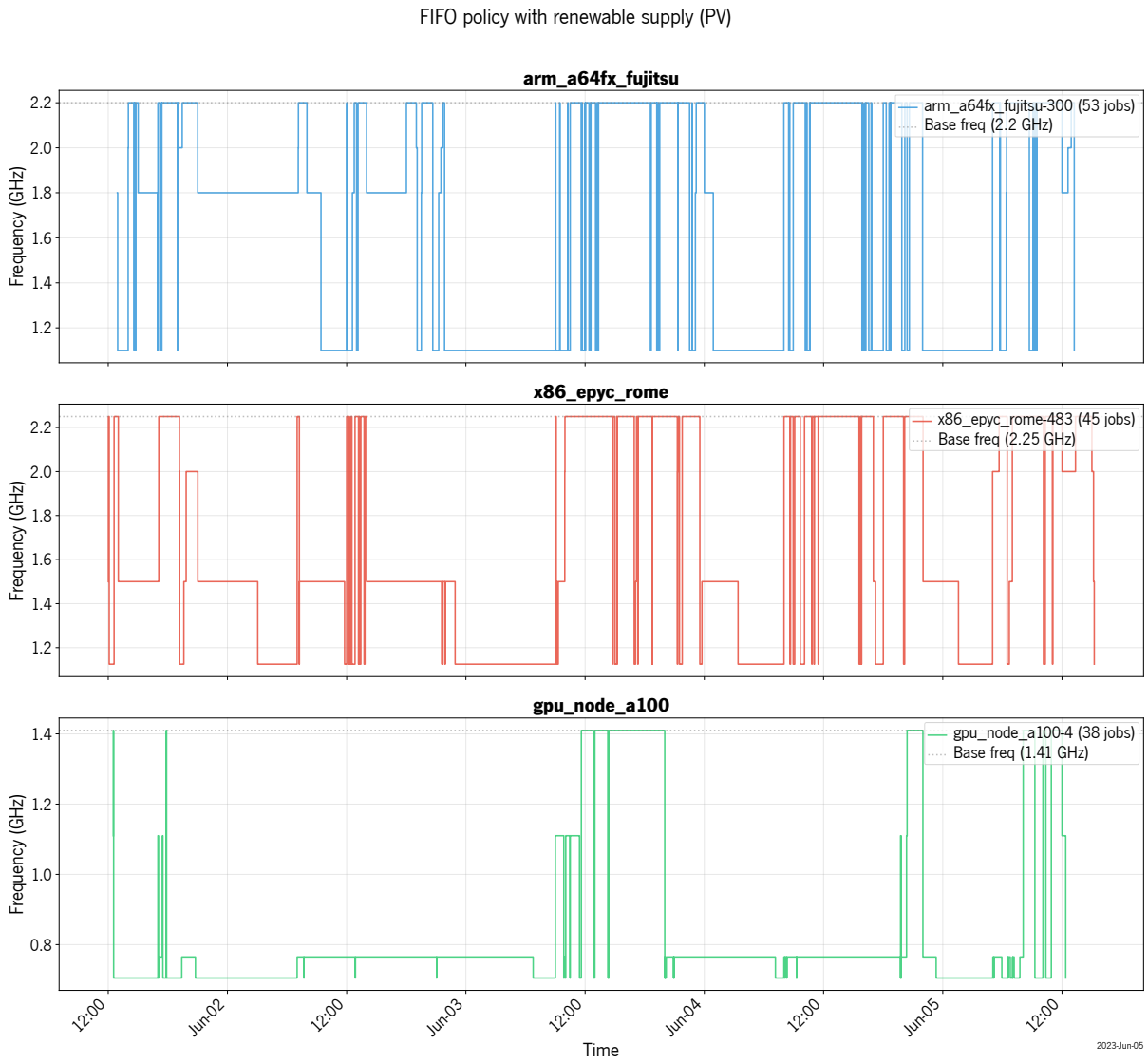


Figure 31: Busiest-node frequency timelines with adaptive Dynamic Voltage and Frequency Scaling (DVFS).

With fixed control, the traces alternate between idle intervals and execution at the maximum modeled frequency, which equals the base frequency in these partitions. With adaptive control, jobs also use intermediate frequencies as the available budget changes, so a reduction in power can extend execution. Idle nodes continue to draw their configured idle power in either case. The following policy comparisons all use adaptive control, keeping this response common to the configurations.

The main runs also use the adaptive scheduling updates described in Section 3.7, allowing completed jobs to release their nodes before the next regular update. Energy budgets are revised every 5 min and the microgrid advances every minute. Puzzle's separate fallback dispatch is disabled, so its job starts follow the energy plan. Front-running and repair are also disabled, and the planner only sees jobs already submitted.

5.3 Scheduling outcomes across energy profiles

Table 5 brings the main comparisons together. With grid supply, the reference policies produce nearly identical outcomes. With PV, Priority and EFT combine shorter waits with lower emissions than Puzzle. With PV and wind, Puzzle achieves the lowest emissions, but jobs wait longer and the workload finishes later. The following subsections explain these differences using the service measures and the physical energy flows together.

Profile	Policy	Mean wait (h)	P95 wait (h)	Makespan (h)	IT energy (kWh)	Grid import (kWh)	CO_2 (kg)	Misses	Run time no plots (s)
Grid	FIFO	0.03	0.08	100.88	24,010.9	30,643.1	7,895	0	38.06
Grid	Priority	0.03	0.08	100.88	24,010.9	30,643.1	7,895	0	38.05
Grid	EFT	0.03	0.08	100.88	24,010.9	30,643.1	7,895	0	37.03
Grid	Puzzle	0.05	0.09	100.96	23,212.5	29,603.9	7,584	0	60.96
PV	FIFO	2.52	9.50	108.89	24,291.6	18,211.5	4,636	0	54.33
PV	Priority	2.25	8.73	101.35	23,208.5	17,065.2	4,371	0	52.52
PV	EFT	2.20	8.73	101.30	23,194.7	17,068.4	4,362	0	53.08
PV	Puzzle	4.61	17.32	126.16	25,741.1	17,763.5	4,458	0	173.11
PV + wind	FIFO	1.90	8.48	100.87	23,369.0	6,956.1	1,683	0	53.99
PV + wind	Priority	1.81	8.38	100.86	23,360.9	6,862.6	1,664	0	52.98
PV + wind	EFT	1.80	8.37	100.86	23,383.4	7,035.1	1,695	0	53.78
PV + wind	Puzzle	3.83	14.84	110.41	23,948.0	5,436.9	1,317	0	150.27

Table 5: Service, energy and execution time for the main runs. Pale green and red mark the lowest and highest values within each profile.

Figure 32 shows how the recorded outcomes change across the profiles. Moving from grid supply to PV also reduces the grid budget and introduces carbon-based eligibility. Adding wind increases generation as well as changing its distribution over time. The lines therefore connect complete configurations, rather than isolating the effect of renewable capacity or temporal complementarity.

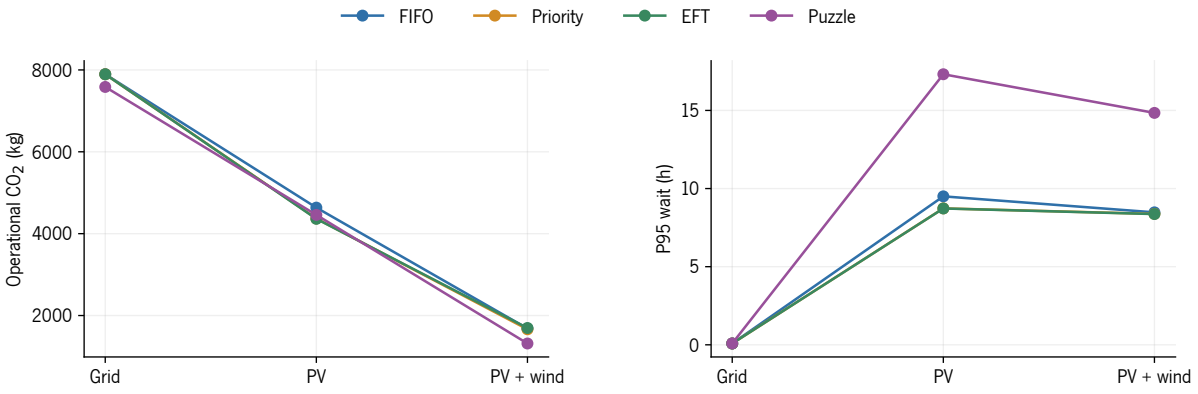


Figure 32: Operational emissions and P95 wait across the three supply profiles.

5.3.1 Grid supply

The 900 kW grid allowance left little waiting for FIFO, Priority or EFT. Their P95 wait was 0.08 h and their makespan was 100.88 h, with matching energy and emissions at the reported precision. Most jobs could start shortly after arrival, leaving little scope for the selection rules to improve service by changing the order.

Puzzle's service measures stayed close to the grid references, with a P95 wait of 0.09 h. It recorded 3.9% less CO_2 than FIFO on the same 2,523-job workload with grid supply. Carbon-aware placement was disabled, so this difference follows its placement and budget allocation rather than deliberate deferral for cleaner electricity. The detailed traces are retained in Section A.6.

5.3.2 PV supply

Under PV supply, Priority and EFT finished earlier than FIFO and recorded both shorter waits and lower emissions. Priority reduced makespan from FIFO's 108.89 h to 101.35 h, with EFT close at 101.30 h. Their effective import intensities were similar to FIFO's, so the emissions improvement primarily accompanied lower grid import. Shorter execution also limited the time for which the powered-on fleet consumed idle energy.

Puzzle finished later than all three reference policies. Against Priority with PV on the same 2,523-job workload, makespan rose from 101.35 h to 126.16 h and P95 wait from 8.73 h to 17.32 h. Grid import increased by 4.1% and emissions by 2.0%. Effective import intensity fell from 256 to 251 gCO_2/kWh, so the cleaner average imports did not offset the larger volume. Both runs remained within the waiting bounds, but Priority provided shorter waits and lower emissions.

The comparison with FIFO explains the energy cost of Puzzle's longer execution. Its monitored run lasted 17.33 h longer, adding about 2,429 kWh of idle energy. Dynamic energy fell by about 979 kWh, but this reduction was insufficient to offset the extra idle consumption, so total IT energy increased by about 1,449 kWh. Lower dynamic consumption alone did not produce a lower energy total. The corresponding power and execution histories appear in Section A.7.

5.3.3 PV and wind supply

With wind added to PV, FIFO, Priority and EFT again produced similar outcomes. Their makespans were close to 100.86 h and their P95 waits ranged from 8.37 h to 8.48 h. Priority recorded the lowest emissions among these references, although the differences in grid import and emissions were small compared with those between supply profiles.

Puzzle reduced grid import and emissions while increasing both waiting time and makespan. Against EFT under PV and wind on the 2,523-job workload, import fell by 22.7% and emissions by 22.3%, while IT energy increased by 2.4%. P95 wait rose from 8.37 h to 14.84 h and makespan from 100.86 h to 110.41 h. Neither configuration recorded a deadline miss.

Puzzle imported 5,436.9 kWh and emitted 1,317 kg of CO_2, the lowest totals among the policies in this profile. Effective import intensity was about 242 gCO_2/kWh, close to the references, so the lower emissions accompanied less imported energy despite the higher IT energy total. Puzzle ended with 949.3 kWh stored, against 1,600 kWh for the references, and its workload finished later. These totals therefore include different final storage states and run durations. Total consumption and operational emissions favored different configurations in this case.

The x86 histories in Figure 33 help explain the longer completion period. Puzzle spread the same 1,023 jobs across a longer interval, with a measured peak of 232.3 kW against EFT's 248.5 kW. The curves include each configuration's budget policy, so this is a comparison of the complete scheduling and energy configurations.

Rectangle widths show observed execution durations. Their heights use estimated dynamic power at the initial frequency, and their vertical positions separate concurrent jobs after execution. The measured-power curve also includes subsequent frequency changes, so it can differ from the rectangle tops. Both panels use identical axes beginning at the partition's idle baseline.

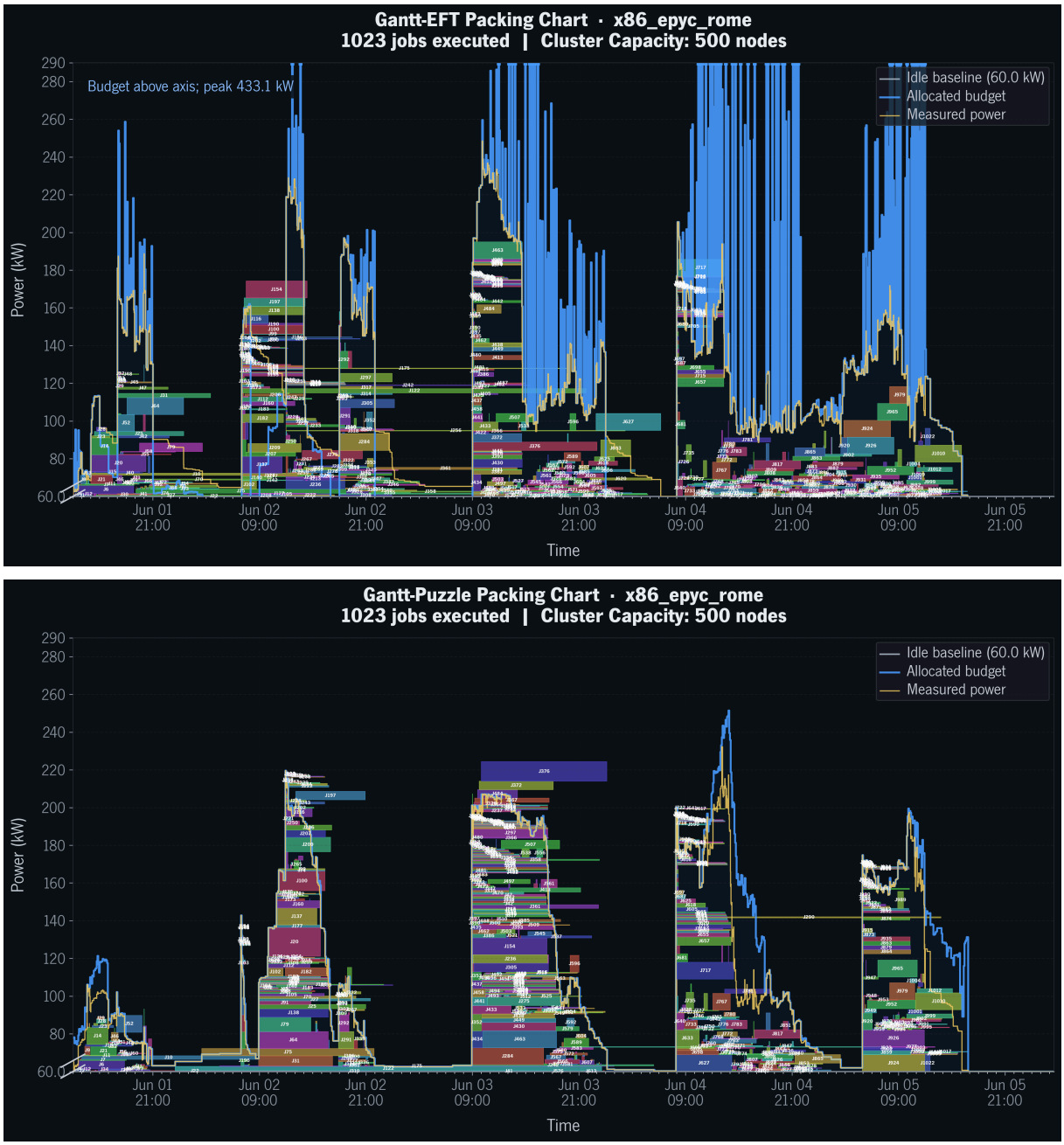


Figure 33: EFT (top) and Puzzle (bottom) execution in the x86 partition with PV and wind, on identical time and power axes.

Puzzle used less grid electricity in this configuration, but jobs waited longer and the workload finished later. Because job placement, compute budgets and partition allocation all change together, this comparison cannot determine how much each contributed to the reduction. The other partition histories and complete energy traces are available in Section A.8.

5.3.4 Simulation and planning time

The reference simulations in Table 5 finished in less than a minute on the 2,523-job workload, while Puzzle took about a minute with grid supply and between two and three minutes under the renewable profiles.

Each Puzzle run built a similar number of plans, but their accumulated construction time was highest with PV and lowest with grid supply (Table 6).

Supply	Plans	Total (s)	P95 (ms)	Max. (ms)
Grid	1,153	19.03	18.9	337.3
PV	1,160	108.06	188.8	465.6
PV + wind	1,156	88.45	170.9	414.7

Table 6: Observed plan-construction cost on the 2,523-job workload.

Each plan took less than half a second to build, but repeated construction accounted for a substantial part of Puzzle's run time. Processor specifications and concurrent machine load were not recorded, so these measurements describe the cost of the evaluated runs without establishing performance in a production scheduler. The effect of repeated replanning is discussed in Section 4.6.

5.4 Effect of workload size

We also compared the policies with 500-job and 1,250-job workloads drawn from the same 2,523-job input. The medium workload retains the smaller one and adds jobs selected by partition, runtime class, queue, priority and node request. Arrivals still span 96 h, and the same fleet remains powered on, allowing us to examine how the configurations behave when serving less work over a similar submission period.

Both workloads use PV and wind scaled to 400 kW each, adaptive DVFS and a threshold of 250 gCO_2/kWh. FIFO, Priority and EFT use CarbonMinimizing with compute-prioritized allocation, while Puzzle uses its energy plan and partition allocation. The Puzzle runs share the same planner and settings as the main PV-and-wind comparison, reserving the fleet's idle demand before placing jobs and including carbon-permitted grid power in the normal search.

In both workloads, Puzzle produced the lowest emissions but the longest waits (Table 7). Its P95 wait was 9.76 h with 500 jobs and 12.32 h with 1,250 jobs, while the reference policies ranged from 1.89 h to 2.05 h and stayed near 7.52 h, respectively. All runs completed without deadline misses, and makespan remained close to 101 h because submissions continued throughout the same arrival window.

Jobs / policy	P95 wait (h)	Makespan (h)	Grid import (kWh)	CO_2 (kg)	Misses
500 / FIFO	2.05	100.80	1,250.8	284	0
500 / Priority	1.89	100.80	1,245.1	283	0
500 / EFT	1.90	100.81	1,245.9	283	0
500 / Puzzle	9.76	100.80	777.1	200	0
1,250 / FIFO	7.52	100.82	3,198.4	761	0
1,250 / Priority	7.52	100.76	3,192.4	760	0
1,250 / EFT	7.52	100.75	3,185.3	758	0
1,250 / Puzzle	12.32	100.82	2,510.5	613	0

Table 7: Small and medium workload outcomes under PV and wind.

Puzzle's lower emissions accompanied a reduction in imported energy. With 500 jobs, it imported 777.1 kWh at an effective intensity of 258 gCO_2/kWh, against Priority's 1,245.1 kWh at 227 gCO_2/kWh. With 1,250 jobs, these values were 2,510.5 kWh at 244 gCO_2/kWh for Puzzle and 3,192.4 kWh at 238 gCO_2/kWh for Priority. Under PV and wind, Puzzle emitted 29.2% less CO_2 than Priority with 500 jobs and 19.3% less with 1,250 jobs, despite the higher effective import intensities. All policies ended with 1,600 kWh stored in both workloads.

The threshold constrains normal grid-backed job placement, while physical imports also cover idle demand and ongoing execution when renewable generation and battery supply are insufficient. In the small Puzzle run, 55.0% of imported energy arrived above the threshold, although the saved plans excluded those periods from the normal grid allowance. Exceptional grid placements can also relax this allowance, as described in Section 4.3.2.

The constant idle demand also affects comparisons between workload sizes. For the reference policies, IT energy per job fell from about 31.7 kWh with 500 jobs to 15.15 kWh with 1,250 jobs, as more work shared the cost of keeping the same fleet powered on. This does not establish an improvement in individual job efficiency.

Section A.9 gives the execution histories and full metrics, including the sampled excesses above each policy's own compute budget (Table 16).

5.5 Carbon threshold and energy-policy sensitivity

These studies keep the workload and energy inputs fixed while varying the carbon threshold or energy-budget policy. Both use the 2,523-job workload, PV and wind scaled to 400 kW each, and a 350 kW grid budget, with one execution per configuration.

Urgency ordering remains active, but the reference policies disable the external grid override used in the main comparisons. Puzzle also disables that override, while retaining the planner's own grid exceptions described in Section 4.3.2.

5.5.1 Carbon threshold

Lowering the threshold removes higher-carbon periods from the normal grid allowance, giving the schedulers fewer opportunities to start grid-backed work. Figure 34 shows the resulting changes in emissions and waiting time. The threshold varies between runs, and the Puzzle case at 250 gCO_2/kWh is the main PV-and-wind result. Its audited plans correctly apply each configured threshold.

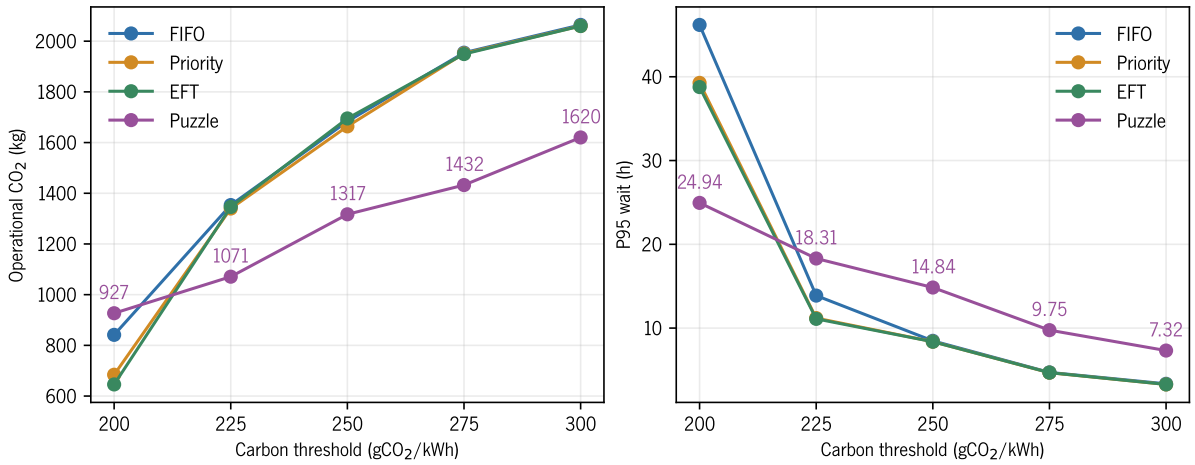


Figure 34: Operational emissions and P95 wait across carbon thresholds under PV and wind.

Lowering the threshold from 250 to 225 gCO_2/kWh reduced emissions for FIFO, Priority and EFT, while increasing their P95 waits (Table 8). For example, Priority's emissions fell from 1,664 to 1,340 kg as P95 wait rose from 8.38 h to 11.19 h. At 200 gCO_2/kWh, the reference policies recorded their lowest emissions but exceeded waiting bounds: FIFO had 51 deadline misses, Priority 23 and EFT 22.

Threshold (gCO_2/kWh)	Scheduler	P95 wait (h)	Makespan (h)	Grid import (MWh)	CO_2 (kg)	Misses
200	FIFO	46.19	158.60	3.838	841	51
200	Priority	39.27	145.26	3.013	684	23
200	EFT	38.77	135.16	2.788	646	22
200	Puzzle	24.94	120.61	3.766	927	0
225	FIFO	13.88	121.47	5.757	1,353	0
225	Priority	11.19	100.90	5.592	1,340	0
225	EFT	11.10	100.87	5.641	1,346	0
225	Puzzle	18.31	123.09	4.549	1,071	0
250	FIFO	8.48	100.87	6.956	1,683	0
250	Priority	8.38	100.86	6.863	1,664	0
250	EFT	8.37	100.86	7.035	1,695	0
250	Puzzle	14.84	110.41	5.437	1,317	0
275	FIFO	4.71	100.87	7.898	1,954	0
275	Priority	4.67	100.86	7.884	1,952	0
275	EFT	4.68	100.88	7.876	1,949	0
275	Puzzle	9.75	107.85	5.827	1,432	0
300	FIFO	3.35	100.87	8.242	2,064	0
300	Priority	3.27	100.87	8.228	2,060	0
300	EFT	3.27	100.87	8.225	2,060	0
300	Puzzle	7.32	100.94	6.508	1,620	0

Table 8: Service and energy outcomes at each carbon threshold under PV and wind.

Puzzle completed every job without deadline misses across all five thresholds. From 200 to 300 gCO_2/kWh, its emissions increased from 927 to 1,620 kg and grid import from 3.77 to 6.51 MWh, while P95 wait fell from 24.94 h to 7.32 h. At 200 gCO_2/kWh, Puzzle recorded 927 kg of CO_2, compared with 841 kg for FIFO, 684 kg for Priority and 646 kg for EFT, while keeping every job within its waiting bound. The reference policies' lower emissions at this threshold therefore came with deadline misses.

Puzzle included both types of exceptional grid placement in its plans at this threshold, while the references had their grid override disabled. Its ability to meet the waiting limits was therefore demonstrated under different exception rules, so the comparison cannot isolate a benefit from packing.

These runs also recorded no monitor samples with IT power above Puzzle's own compute budget. This budget covers idle demand and jobs already running, with grid support added when needed. The zero count therefore partly reflects how Puzzle sets its budget, and does not establish better compliance with a common power limit. The excess counts in Table 16 likewise compare each policy with its own budget.

A control at 2,000 gCO_2/kWh permitted grid power throughout the trace, producing 2,035 kg of emissions with a P95 wait of 0.09 h. The 300 gCO_2/kWh case still excludes some slots, and a non-preemptive job must fit every slot it spans. Even a short high-carbon period can therefore delay placement. The planner's

carbon-regret and waiting-bound emergency paths can also admit grid above the threshold, as described in Section 4.3.2.

5.5.2 Energy-budget policy

Changing the energy rule had a larger effect on waiting and emissions than changing among FIFO, Priority and EFT in these runs. Table 9 compares Grid, DaytimeRenewable and CarbonMinimizing with the same PV, wind, battery and grid limit. Grid still allows renewable generation to supply demand. Its name identifies the budget rule, rather than a scenario without renewables. Between 08:00 and 20:00, DaytimeRenewable reserves 30% of renewable generation for charging, subject to battery limits, blocks battery discharge and offers half the configured grid allowance. At night it offers all renewable generation, controlled battery discharge and the full grid allowance. It follows time of day without testing carbon intensity.

Scheduler	Energy policy	P95 wait (h)	Makespan (h)	Grid import (MWh)	CO_2 (kg)	Misses
FIFO	Grid	0.08	100.88	7.650	2,144.3	0
Priority	Grid	0.08	100.88	7.650	2,144.3	0
EFT	Grid	0.08	100.88	7.650	2,144.3	0
FIFO	DaytimeRenewable	0.18	100.87	7.645	2,076.3	0
Priority	DaytimeRenewable	0.09	100.88	7.638	2,074.0	0
EFT	DaytimeRenewable	0.09	100.88	7.638	2,073.9	0
FIFO	CarbonMinimizing (250)	8.48	100.87	6.956	1,683.4	0
Priority	CarbonMinimizing (250)	8.38	100.86	6.863	1,663.9	0
EFT	CarbonMinimizing (250)	8.37	100.86	7.035	1,695.2	0

Table 9: Service and energy outcomes across the three energy-budget policies.

Under Grid, the scheduling policies matched at the displayed precision. With the same 2,523-job workload and PV-and-wind supply, DaytimeRenewable reduced emissions by about 3% relative to Grid while P95 waits remained below 0.18 h. CarbonMinimizing at 250 gCO_2/kWh reduced FIFO emissions by 21% relative to Grid in this setting, while its P95 wait rose from 0.08 h to 8.48 h. None of these configurations recorded a deadline miss. The choice of when to offer power for new work shaped the result more strongly than the selection order within the resulting budgets.

5.6 Discussion and limitations

The evaluation demonstrates WAVE's use for developing and testing policies that connect workload execution with energy management. FIFO, Priority and EFT provide reference behaviors, while the Puzzle scheduler demonstrates how a new policy can use WAVE's execution and power models to plan job start times, frequencies and energy allocation. The experiments follow these decisions through simulated

execution, recording their consequences for jobs, storage, grid import and emissions under different conditions.

The Puzzle scheduler completed every submitted job without deadline misses in the main comparisons, the experiments with smaller workloads and the carbon threshold study. Under PV and wind, it recorded the lowest operational emissions at each tested workload size, while keeping its longer waits within the configured queue bounds. On the full 2,523-job workload with PV and wind, emissions were 22.3% lower than EFT's, with P95 wait increasing from 8.37 h to 14.84 h. Lower emissions also accompanied waits close to the references under grid supply. In the threshold study on the full workload, Puzzle remained below all three reference policies in emissions at every tested threshold from 225 to 300 gCO_2/kWh.

At the most restrictive threshold, 200 gCO_2/kWh, Puzzle was the only policy to avoid deadline misses and had a shorter P95 wait than every reference. Compared with FIFO on the same 2,523-job workload under PV and wind, it emitted 10.1% more CO_2 but reduced P95 wait by 46.0%, with no misses against FIFO's 51. Priority and EFT also emitted less than Puzzle, but recorded 23 and 22 misses, respectively. Puzzle's grid exceptions remained available when the references' waiting-time override was disabled, so its service advantage reflects the full policy configuration.

With PV alone on the 2,523-job workload, Puzzle emitted about 2% more CO_2 than Priority and EFT and had longer waits, although its emissions remained below FIFO's. The recorded histories explain this outcome: lower dynamic energy was outweighed by the idle cost of finishing later. Under PV and wind, lower grid import instead reduced emissions even though total IT energy rose. These mechanisms show how WAVE can help assess a policy and identify where its planning choices require improvement.

Changing when power was offered for new work affected waiting and emissions more strongly than changing among FIFO, Priority and EFT. WAVE therefore supports experiments on both job selection and energy management, with recorded histories guiding subsequent policy changes.

The comparisons describe complete scheduling and energy configurations, so they assess Puzzle's combined behavior without separating each mechanism's contribution. Energy and emissions include each execution's duration and changes in stored energy, with potentially different final battery charges. Keeping the same fleet powered on also means that the workload-size results include changes in the share of idle energy per job.

The experiments use fixed energy traces and a synthetic workload with long jobs arriving early, while runtime estimates include no added prediction error. The queue waiting limits determine the room for

deferral, and the power profiles and constant PUE have not been validated against measured Deucalion consumption.

Chapter 6

Conclusions and Future Work

This dissertation presented WAVE, an extension of Vessim that couples HPC workload execution with energy-system simulation. We used it to compare reference policies and develop the Puzzle scheduler, following their effects on job service and emissions under different energy conditions. Its configurable models also support the choice of renewable generation and storage for computing centers with different hardware, workloads and local energy conditions.

6.1 Answers to the research questions

WAVE connects computing and energy resources through power demand and compute budgets. Running jobs and node states determine demand, while renewable availability, storage and grid conditions influence dispatch and frequency decisions. This coupling allows the effects of a policy to be followed across workload execution and the energy system.

The experiments showed that the energy-budget rule could affect waiting and emissions more strongly than the choice among reference job selection policies. They also showed that lower emissions can accompany higher IT energy use when execution relies less on the grid, with WAVE's recorded histories explaining how these outcomes arise.

The Puzzle scheduler demonstrated how this environment can support a policy that plans job timing and frequency using energy forecasts. It completed every job without deadline misses in the main comparisons, the smaller workload experiments and the threshold study. Under PV and wind, it recorded the lowest emissions at each tested workload size, including a 22.3% reduction relative to EFT on the full 2,523-job workload, with longer waits remaining within the configured bounds. At the most restrictive threshold, it combined higher emissions than the references with shorter P95 waits and no misses. With PV alone, the

idle cost of prolonged execution outweighed the reduction in dynamic energy, suggesting a refinement to the planner.

6.2 Contributions

WAVE provides configurable workload generation, execution and power models, with monitoring that follows jobs alongside energy flows. Its separate interfaces for job selection, budget generation and partition allocation allow policies to be developed and compared using the same resource models, while retained inputs and histories support the interpretation of their results.

The same models support the selection and sizing of renewable infrastructure for different computing centers. After adapting the hardware and workload to a target center, solar, wind and battery configurations can be compared through grid import, operational emissions and job service. This connects infrastructure choices to the center's demand, the work that can be deferred and the storage needed to bridge generation and consumption.

The Puzzle scheduler contributes a planning method that coordinates job placement, frequency and energy allocation using forecasts and the current system state, providing a concrete example of policy development within WAVE.

6.3 Limitations

The evaluation uses a Deucalion-based configuration with synthetic workloads and long jobs arriving early. Arrival patterns, runtimes, resource requests and partition balance affect how much demand fits within renewable periods, while waiting bounds determine the room for deferral. These conditions should reflect each center's service requirements. Runtime estimates include no added prediction error, and renewable and carbon forecasts come from execution traces, leaving the planner's response to inaccurate information untested.

Node demand is modeled through state and frequency, without resolving differences in application activity at the same frequency, while facility overhead uses a fixed multiplier. Hardware composition, idle demand and actual overhead can therefore change the power that a renewable installation must cover.

Infrastructure comparisons also depend on local generation and grid-carbon profiles. The renewable data cover a full year, so seasonal conditions can be tested by changing the simulation start date, while comparisons across locations and years require representative data for those conditions.

Battery capacity, power limits, efficiencies and usable charge range determine how much renewable surplus can be used later. Within each comparison, the runs start with the same stored energy but may finish at different charge levels, so grid import and emissions should be interpreted alongside the change in stored energy.

The grid allowance controls scheduling rather than imposing a physical connection limit, which would need to be represented explicitly for a center with a restricted connection. The assessment covers operational grid emissions, leaving installation costs, available space, equipment aging and life-cycle emissions for a broader investment analysis.

6.4 Future work

Further evaluation should use real workload traces from other computing centers, compare seasons within the existing annual profiles and add data from other locations and years. Measured consumption could refine the execution and power models, while experiments with uncertain runtime estimates and renewable and carbon forecasts should apply those errors to the planner's reservations to assess its response to departures from predicted conditions.

An automated search over solar, wind and battery configurations could extend WAVE's current comparisons into infrastructure planning. Candidate designs could be ranked by grid import, operational emissions and service performance, with physical grid limits, costs, equipment lifetime and site constraints included in the selection.

For the Puzzle scheduler, a refinement would be to include the facility idle energy associated with a longer makespan in its placement score, complementing the idle demand already reserved in the plan. WAVE's existing controls and policy interfaces can also support experiments on how planning resolution, update frequency and alternative budget and allocation rules affect planning cost, waiting and emissions.

Bibliography

[1] "Deucalion (EuroHPC JU)." [Online]. Available: https://www.eurohpc-ju.europa.eu/supercomputers/our-supercomputers_en#deucalion

[2] "TOP500 List - June 1993." [Online]. Available: https://www.top500.org/lists/top500/list/1993/06/

[3] "TOP500 List - November 2024." [Online]. Available: https://www.top500.org/lists/top500/2024/11/

[4] K. L. Veigas, A. Chinnici, D. D. Chiara, and M. Chinnici, "Towards Energy Efficiency of HPC Data Centers: A Data-Driven Analytical Visualization Dashboard Prototype Approach," *Electronics*, 2025, doi: 10.3390/electronics14163170.

[5] C. A. Silva, R. Vilaça, A. Pereira, and R. J. Bessa, "A review on the decarbonization of high-performance computing centers," *Renewable and Sustainable Energy Reviews*, vol. 189, p. 114019, 2024, doi: 10.1016/j.rser.2023.114019.

[6] L. Lannelongue, J. Grealey, and M. Inouye, "Green Algorithms: Quantifying the Carbon Footprint of Computation," *Advanced Science*, vol. 8, no. 12, p. 2100707, 2021, doi: 10.1002/advs.202100707.

[7] L. Tan, S. Kothapalli, L. Chen, O. Hussaini, R. Bissiri, and Z. Chen, "A survey of power and energy efficient techniques for high performance numerical linear algebra operations," *Parallel Computing*, vol. 40, pp. 559–573, 2014, doi: 10.1016/j.parco.2014.09.001.

[8] "Green500 - The List." [Online]. Available: https://www.top500.org/lists/green500/

[9] P. Wiesner, I. Behnke, P. Kilian, M. Steinke, and O. Kao, "Vessim: A Testbed for Carbon-Aware Applications and Systems," in *3rd Workshop on Sustainable Computer Systems (HotCarbon'24)*, 2024. [Online]. Available: https://github.com/dos-group/vessim

[10] H. Hussain *et al.*, "A survey on resource allocation in high performance distributed computing systems," *Parallel Computing*, vol. 39, no. 11, pp. 709–736, 2013, doi: 10.1016/j.parco.2013.09.009.

[11] S. Mittal and J. S. Vetter, "A Survey of CPU-GPU Heterogeneous Computing Techniques," *ACM Computing Surveys*, vol. 47, no. 4, p. 69:1–69:35, 2015, doi: 10.1145/2788396.

[12] "Overview - Deucalion User Guide." [Online]. Available: https://docs.macc.fccn.pt/jobs/

[13] Lawrence Livermore National Laboratory, "Batch System Primer." [Online]. Available: https://hpc.llnl.gov/banks-jobs/running-jobs/batch-system-primer

[14] D. G. Feitelson, *Workload Modeling for Computer Systems Performance Evaluation*. Cambridge University Press, 2015. doi: 10.1017/CBO9781139939690.

[15] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009, doi: 10.1145/1498765.1498785.

[16] U. Lublin and D. G. Feitelson, "The workload on parallel supercomputers: modeling the characteristics of rigid jobs," *Journal of Parallel and Distributed Computing*, vol. 63, no. 11, pp. 1105–1122, 2003, doi: 10.1016/S0743-7315(03)00108-4.

[17] A. Tarraf *et al.*, "Malleability in Modern HPC Systems: Current Experiences, Challenges, and Future Opportunities," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 9, pp. 1551–1564, 2024, doi: 10.1109/TPDS.2024.3406764.

[18] Y. Fan, "Job Scheduling in High Performance Computing." [Online]. Available: https://arxiv.org/abs/2109.09269

[19] A. W. Mu'alem and D. G. Feitelson, "Utilization, Predictability, Workloads, and User Runtime Estimates in Scheduling the IBM SP2 with Backfilling," *IEEE Transactions on Parallel and Distributed Systems*, vol. 12, no. 6, pp. 529–543, 2001, doi: 10.1109/71.932708.

[20] Y. Fan, P. Rich, W. E. Allcock, M. E. Papka, and Z. Lan, "Trade-Off Between Prediction Accuracy and Underestimation Rate in Job Runtime Estimates," in *2017 IEEE International Conference on Cluster Computing (CLUSTER)*, 2017, pp. 530–540. doi: 10.1109/CLUSTER.2017.11.

[21] M. A. Jette and T. Wickberg, "Architecture of the Slurm Workload Manager," in *Job Scheduling Strategies for Parallel Processing*, in Lecture Notes in Computer Science, vol. 14283. Springer Nature Switzerland, 2023, pp. 3–23. doi: 10.1007/978-3-031-43943-8_1.

[22] D. Tsafrir, Y. Etsion, and D. G. Feitelson, "Modeling User Runtime Estimates," in *Job Scheduling Strategies for Parallel Processing*, in Lecture Notes in Computer Science, vol. 3834. Springer Berlin Heidelberg, 2005, pp. 1–35. doi: 10.1007/11605300_1.

[23] Y. Fan *et al.*, "Scheduling Beyond CPUs for HPC," in *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, 2019, pp. 97–108. doi: 10.1145/3307681.3325401.

[24] S. Wallace *et al.*, "A Data Driven Scheduling Approach for Power Management on HPC Systems," in *SC '16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2016, pp. 656–666. doi: 10.1109/SC.2016.55.

[25] A. Radovanović *et al.*, "Carbon-Aware Computing for Datacenters," *IEEE Transactions on Power Systems*, vol. 38, no. 2, pp. 1270–1280, 2023, doi: 10.1109/TPWRS.2022.3173250.

[26] International Organization for Standardization and International Electrotechnical Commission, "Information technology - Data centres - Key performance indicators - Part 2: Power usage effectiveness (PUE)," technical report ISO/IEC 30134-2:2016, 2016. [Online]. Available: https://www.iso.org/standard/63451.html

[27] T. van de Voort, V. Zavrel, J. I. T. Galdiz, and J. L. M. Hensen, "Analysis of performance metrics for data center efficiency: should the Power Utilization Effectiveness PUE still be used as the main indicator? (Part 1)," *REHVA Journal*, pp. 5–11, 2017, [Online]. Available: https://www.rehva.eu/rehva-journal/chapter/analysis-of-performance-metrics-for-data-center-efficiency-should-the-power-utilization-effectiveness-pue-still-be-used-as-the-main-indicator-part-1

[28] C. Conficoni, A. Bartolini, A. Tilli, C. Cavazzoni, and L. Benini, "HPC Cooling: A Flexible Modeling Tool for Effective Design and Management," *IEEE Transactions on Sustainable Computing*, vol. 6, no. 3, pp. 441–455, 2021, doi: 10.1109/TSUSC.2018.2809574.

[29] J. Oltmanns, D. Sauerwein, F. Dammel, P. Stephan, and C. Kuhn, "Potential for waste heat utilization of hot-water-cooled data centers: A case study," *Energy Science & Engineering*, vol. 8, no. 5, pp. 1793–1810, 2020, doi: 10.1002/ese3.633.

[30] CSC - IT Center for Science, "Vastuullisuusraportti 2025," technical report, 2026. [Online]. Available: https://csc.fi/app/uploads/2026/03/Vastuullisuusraportti_2025.pdf

[31] R. Ge, X. Feng, H. Pyla, K. W. Cameron, and W.-chun Feng, "Power Measurement Tutorial for the Green500 List," technical report, 2007. [Online]. Available: https://www.top500.org/files/green500/tutorial.pdf

[32] L. Szustak, R. Wyrzykowski, L. Kuczynski, and T. Olas, "Architectural Adaptation and Performance-Energy Optimization for CFD Application on AMD EPYC Rome," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 12, pp. 2852–2866, 2021, doi: 10.1109/TPDS.2021.3078153.

[33] J. Corbalan, L. Alonso, J. Aneas, and L. Brochard, "Energy Optimization and Analysis with EAR," in *2020 IEEE International Conference on Cluster Computing (CLUSTER)*, 2020, pp. 464–472. doi: 10.1109/CLUSTER49012.2020.00067.

[34] P. Shenoy, "Energy-Efficiency versus Carbon-Efficiency: What's the Difference?," *ACM SIGEnergy Energy Informatics Review*, vol. 2, no. 4, pp. 1–2, 2022, doi: 10.1145/3584024.3584025.

[35] The Green Grid, "Carbon Usage Effectiveness (CUE): A Green Grid Data Center Sustainability Metric," technical report White Paper 32, 2010. [Online]. Available: https://archive.thegreengrid.org/en/resources/library-and-tools/241-WP

[36] International Organization for Standardization and International Electrotechnical Commission, "Information technology - Data centres key performance indicators - Part 8: Carbon usage effectiveness (CUE)," technical report ISO/IEC 30134-8:2022, 2022. [Online]. Available: https://www.iso.org/standard/77691.html

[37] B. Whitehead, D. Andrews, and A. Shah, "The life cycle assessment of a UK data centre," *The International Journal of Life Cycle Assessment*, vol. 20, no. 3, pp. 332–349, 2015, doi: 10.1007/s11367-014-0838-7.

[38] B. Li, R. B. Roy, D. Wang, S. Samsi, V. Gadepally, and D. Tiwari, "Toward Sustainable HPC: Carbon Footprint Estimation and Environmental Implications of HPC Systems," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, p. 19:1–19:15. doi: 10.1145/3581784.3607035.

[39] I. Goiri, W. Katsak, K. Le, T. D. Nguyen, and R. Bianchini, "Parasol and GreenSwitch: Managing Datacenters Powered by Renewable Energy," in *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2013, pp. 51–64. doi: 10.1145/2451116.2451123.

[40] P. Wiesner, I. Behnke, D. Scheinert, K. Gontarska, and L. Thamsen, "Let's Wait Awhile: How Temporal Workload Shifting Can Reduce Carbon Emissions in the Cloud," in *Proceedings of the 22nd International Middleware Conference*, 2021, pp. 260–272. doi: 10.1145/3464298.3493399.

[41] D. Chen, Y. Ma, L. Wang, and M. Yao, "Spatio-temporal management of renewable energy consumption, carbon emissions, and cost in data centers," *Sustainable Computing: Informatics and Systems*, vol. 41, p. 100950, 2024, doi: 10.1016/j.suscom.2023.100950.

[42] H. Casanova, A. Giersch, A. Legrand, M. Quinson, and F. Suter, "Versatile, Scalable, and Accurate Simulation of Distributed Applications and Platforms," *Journal of Parallel and Distributed Computing*, vol. 74, no. 10, pp. 2899–2917, 2014, doi: 10.1016/j.jpdc.2014.06.008.

[43] H. Casanova, S. Pandey, J. Oeth, R. Tanaka, F. Suter, and R. F. da Silva, "WRENCH: A Framework for Simulating Workflow Management Systems," in *2018 IEEE/ACM Workflows in Support of Large-Scale Science (WORKS)*, 2018, pp. 74–85. doi: 10.1109/WORKS.2018.00013.

[44] S. S. Vatsavai *et al.*, "CGSim: A Simulation Framework for Large Scale Distributed Computing Environment," in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 1478–1483. doi: 10.1145/3731599.3769277.

[45] P.-F. Dutot, M. Mercier, M. Poquet, and O. Richard, "Batsim: A Realistic Language-Independent Resources and Jobs Management Systems Simulator," in *Job Scheduling Strategies for Parallel Processing*, 2017, pp. 178–197. doi: 10.1007/978-3-319-61756-5_10.

[46] C. Galleguillos, Z. Kiziltan, A. Netti, and R. Soto, "AccaSim: A customizable workload management simulator for job dispatching research in HPC systems," *Cluster Computing*, vol. 23, no. 1, pp. 107–122, 2019, doi: 10.1007/s10586-019-02905-5.

[47] M. A. Amrizal, R. S. Prasasta, S. Y. Pradata, K. G. Santiyuda, R. Pulungan, and H. Takizawa, "SPARS: A reinforcement learning-enabled simulator for power management in HPC job scheduling," *SoftwareX*, vol. 34, p. 102693, 2026, doi: 10.1016/j.softx.2026.102693.

[48] C. Steinbrink *et al.*, "CPES Testing with mosaik: Co-Simulation Planning, Execution and Analysis," *Applied Sciences*, 2019, doi: 10.3390/app9050923.

[49] J.-M. Pierson *et al.*, "DATAZERO: Datacenter With Zero Emission and Robust Management Using Renewable Energy," *IEEE Access*, 2019, doi: 10.1109/ACCESS.2019.2930368.

[50] D. Landré, L. Philippe, and J.-M. Pierson, "Sufficiency power consideration to run a workload on renewable energy operated datacenter," *Future Generation Computer Systems*, vol. 167, p. 107710, 2025, doi: 10.1016/j.future.2025.107710.

[51] B. Acun *et al.*, "Carbon Explorer: A Holistic Framework for Designing Carbon Aware Datacenters," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ACM, 2023, pp. 118–132. doi: 10.1145/3575693.3575754.

Appendix A

Additional evaluation material

The following material supports Chapter 5 with reproduction settings, workload details, complete energy-policy rules and execution histories. The main comparisons are reported in Table 5.

A.1 Reproducing the experiments

Reproduction repository: **URL to be added before submission.**

The reproduction material preserves the inputs, settings and source snapshots used in the experiments. Table 10 identifies the source directories, while `thesis/reproducibility/manifest.json` records the execution commands and file checksums for each reported configuration.

Reported configurations	**Source directory under `results/`**
Main reference policies and controls	`results_thesis_final`
Grid and PV Puzzle	`results_thesis_final/rerun_current_20260929`
PV plus wind Puzzle and its threshold runs	`thesis_puzzle_admissible_grid_2026-09-28`
Reference threshold and energy-policy runs	`thesis_sensitivity_2026-09-28`
Reference policies with smaller workloads	`workload_low_500_2026-09-29`
Puzzle with smaller workloads	`workload_puzzle_corrected_2026-09-29`

Table 10: Source selection for reproduction. Each directory contains `source_snapshot/`.

The runs used Python 3.12.3, Mosaik 3.5.0 and `mosaik-api-v3` 3.0.13, with a fixed hash seed (`PYTHONHASHSEED=0`). Since a complete dependency lock from the original environment was not retained, the reproduction README also documents the environment used to check the execution commands.

A.1.1 Model settings

Table 11 gives the shared physical and timing parameters. The supply profiles and policy combinations are defined in Section 5.1.2 and Table 2, while Table 3 gives the queue restrictions. Grid charging remains disabled in the policy comparisons and enabled only in its separate control.

Parameter	Value
Simulation origin	`2023-06-01 12:00:00`
PUE	1.3
Battery capacity and initial energy	2,000 kWh and 1,000 kWh
Battery state-of-charge interval	20% to 80%
Maximum charge and discharge power	500 kW in each direction
Charge and discharge efficiencies	0.95 in each direction
Energy-scheduler and microgrid updates	300 s and 60 s
Job scheduler updates	Adaptive, with a maximum interval of 60 s
Node state	On at start and kept on while idle
Failure and prediction error rates	Both zero, prediction seed 42
EFT DVFS weights	$\alpha = 0.1$, $\beta = 0.9$

Table 11: Shared simulation parameters.

The modeled node profiles appear in Table 12. Each frequency selects a per-node dynamic power value, which is added to idle power before applying the facility PUE. These are the configured simulation profiles, including the GPU-node profile, rather than measurements of complete physical systems.

Profile	Nodes	Idle (W)	Frequency (GHz): dynamic power (W)
ARM A64FX	1,632	40	1.8: 110, 2.0: 145, 2.2: 180
x86 EPYC Rome	500	120	1.5: 280, 2.0: 480, 2.25: 650
GPU A100	33	450	0.765: 1,100, 1.11: 1,600, 1.41: 2,100

Table 12: Per-node IT power points used by DVFS.

Puzzle uses a 24 h forecast horizon with 5 min slots and a periodic replan interval of 6 h. New pending jobs, plan expiry and explicit replan requests can also trigger rebuilding at a budget update. Only arrived jobs are visible, the global green board is enabled, and front-running, packing repair and fallback dispatch are disabled. Carbon-aware placement is disabled for Grid and enabled in the renewable cases, with the 6 h planned-start margin defined in Section 5.1.4.

Beyond this forecast horizon, Puzzle uses a carbon multiplier of 1.25 and a 10% battery reserve. The renewable uncertainty fraction is 20%, but the robustness weight is zero, selecting the central renewable scenario. The carbon-regret threshold is 1 g/h. Carbon-aware battery dispatch is enabled with a zero carbon-priority reserve fraction.

A.1.2 Inputs and execution

The simulations use the saved workload CSVs, whose generation is described in Section A.2. The smaller experiments use stratified selections of 500 and 1,250 jobs, with the smaller selection included in the larger one. The sampling seeds and selected job identifiers are recorded with the inputs.

The carbon signal and hourly PV and wind profiles share a UTC time axis, with the latest available sample held between timestamps. The normalized generation profiles are aligned to the carbon timestamps, scaled by their configured source power and summed. Each generation profile is set to zero before its first available sample.

The retained metadata identifies wind as Portugal onshore generation from Open Power System Data, normalized to its peak. It does not identify the original provider exports for the carbon and PV signals, so the preserved CSVs and their checksums define the inputs used in these experiments.

Run the following commands from the package root to reproduce the FIFO and Puzzle configurations with their saved inputs and source versions. Results are written to new directories. Omitting `--execute` checks the files and configuration without starting a simulation.

```
PYTHONHASHSEED=0 venv/bin/python -B thesis/reproducibility/replay.py \
  --case main/grid_fifo --output reproduced/grid_fifo --execute

PYTHONHASHSEED=0 venv/bin/python -B thesis/reproducibility/replay.py \
  --case main/solar_wind_puzzle \
  --output reproduced/solar_wind_puzzle --execute
```

A.2 Workload generation

The workload combines many short jobs with a small group of longer computations. Jobs enter this group with a probability of 2%, and their durations are sampled from a normal curve centered at 1.818 h with a standard deviation of 16 h, retaining values above 6 h and capping them at 32 h. Longer durations are therefore less likely to be sampled, apart from values accumulated at the cap. The resulting workload has a mean computation time of 1.818 h, with 40 jobs above 6 h and eight at or above 24 h, as shown in Figure 27.

The distribution across partitions and requested-walltime queues appears in Figure 35.

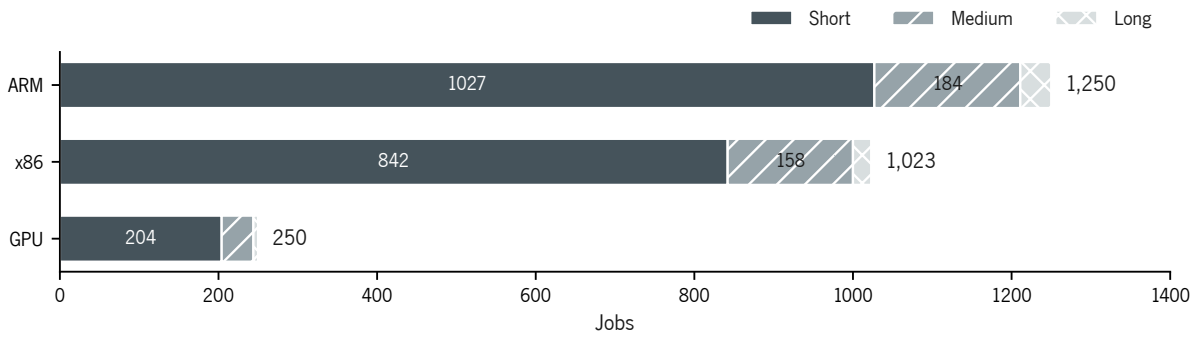


Figure 35: Jobs by partition and requested-walltime queue.

A.3 Scheduling-policy illustration

Figure 36 illustrates the FIFO, Priority and EFT selection rules using the same six pending jobs in a single queue. Each job requests one node, which becomes available for each successive selection. Frequency is fixed and waiting-bound ordering is disabled, so FIFO preserves arrival order, Priority selects the highest priority first, and EFT selects the shortest predicted execution first. The job durations and priorities are illustrative inputs.

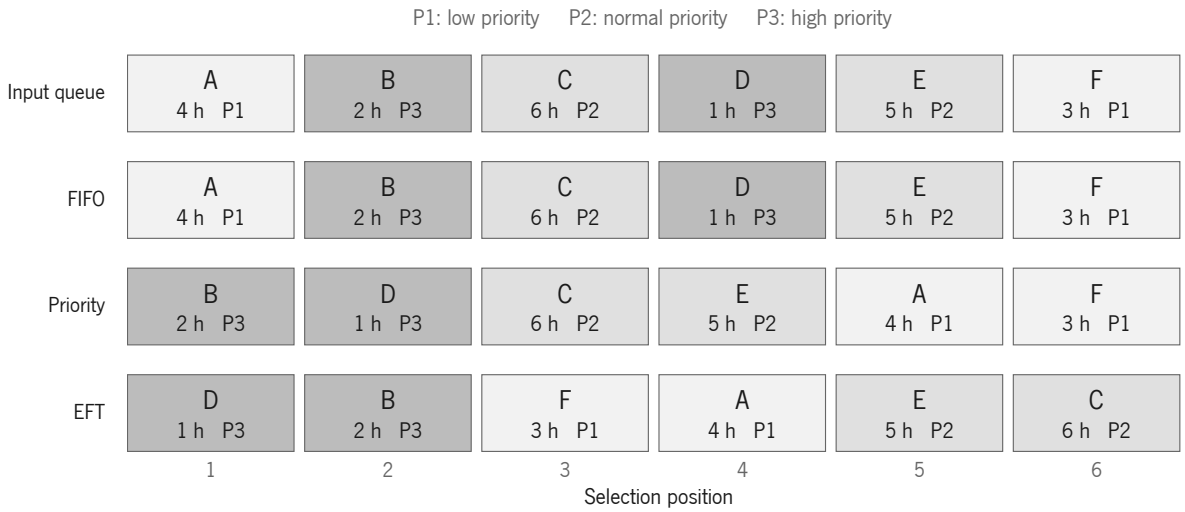


Figure 36: Successive selections from a single queue at fixed frequency, with waiting-bound ordering disabled.

A.4 Energy scheduling details

A.4.1 Energy-budget policies

The energy scheduler updates its total budget every 5 min, returning storage commands alongside the power offered to computation through the interface in Figure 4. The budget combines renewable power available to compute, planned battery discharge and the permitted grid contribution, while the microgrid applies the storage commands within its physical limits.

Grid adds a fixed grid allowance to the current renewable generation, independently of carbon intensity or battery state. It reserves no renewable power for charging and adds no planned battery discharge to the compute budget, although the microgrid can still use storage to settle the physical imbalance. When this rule is used in the energy-policy sensitivity comparison, the same PV and wind generation remains available to supply demand.

CarbonMinimizing first subtracts a charging reserve from renewable generation. The configured reserve is 30% of instantaneous generation, capped by the battery's charging limit and the capacity available over the update interval. The remaining renewable power enters the compute budget, with the grid and battery contributions selected according to carbon intensity and the configured daytime window, from 08:00 to 20:00.

When carbon intensity is at or below the threshold, CarbonMinimizing offers the full configured grid allowance and blocks battery discharge, preserving stored energy while requesting charging from renewable surplus. Above the threshold, it offers no new grid contribution. During daytime it continues to block discharge and request charging, while at night it requests gradual discharge based on usable stored energy and the remaining night, subject to battery and update-interval limits. The charging reserve is still deducted from the renewable compute budget in this night branch, although the storage command requests discharge.

DaytimeRenewable, included in the energy-policy sensitivity comparison, uses the same daytime window and charging fraction without testing carbon intensity. During daytime it reserves renewable power for charging, blocks discharge and offers half the configured grid allowance. At night it offers all current renewable generation, controlled battery discharge and the full grid allowance. This provides a comparison between a rule based on time of day and one that also responds to the grid-carbon signal.

Puzzle obtains the source contributions and storage commands from the current slot of the energy plan described in Chapter 4. These budgets therefore follow planned job execution and energy availability. In the

grid-only reference, grid power is permitted throughout the trace and carbon-driven placement is disabled, allowing the planner to use the first feasible start.

A.4.2 Partition budget allocation

The facility budget is divided by PUE before being distributed as IT power through the interface in Figure 4. The compute-prioritized rule first covers the current measured draw of partitions with unfinished work, including their powered-on idle nodes. If the budget is insufficient, it divides the available power in proportion to these demands. Otherwise, the remainder goes to partitions with pending jobs, in proportion to the configured power demand of their available nodes, giving them room to start additional work.

Puzzle-plan reserves the measured draw of completed partitions whose nodes remain powered on, then covers the current draw of active partitions where the budget permits. Remaining power first moves those partitions toward their planned shares, with any further surplus distributed in the same plan proportions. This connects allocation to the jobs placed in the energy plan, so the comparison with Puzzle changes budget generation and partition allocation together with job selection.

A.5 Reading the execution histories

The run figures show partition budgets, planned source contributions, simulated power, cumulative IT energy and operational emissions. The budget panels convert source contributions to IT power using PUE, allowing direct comparison with partition allocations and demand. In the power panels, partition and total IT demand exclude PUE, while the renewable-generation curve shows facility supply. Recorded IT demand includes idle nodes and running jobs, which can continue above a reduced scheduling budget.

In the cumulative energy panels, yellow denotes the renewable share and red the grid share, attributed using planned budget proportions. Physical grid import and emissions come from the microgrid monitor, as described in Section 3.4.3, and cannot be inferred from those colored shares.

The execution diagrams overlay partition budget, measured power and packed job rectangles on one power axis. Rectangle widths show observed durations, while vertical positions are assigned after execution to separate concurrent jobs. In ARM and x86, rectangles show each job's estimated dynamic power at its initial frequency above the idle component of all powered-on nodes. The axes begin at 65.28 and 60 kW, respectively, with upper limits of 130 and 220 kW for the individual views. The x86 EFT-Puzzle comparison uses a shared 280 kW upper limit to include EFT's measured power peaks. Budget peaks outside the plotted range are marked, while their full traces appear in the corresponding operation figures.

GPU rectangles use each job's estimated total node power. The measured curve also reflects frequency changes during execution, so rectangle tops can differ from it.

A.6 Grid supply

The reference-policy totals are compared in Table 13. Pale green and pale red mark the lowest and highest values in each numerical column. Ties share the highlight, while columns with identical values remain unshaded.

Policy	Mean wait (h)	P95 wait (h)	Makespan (h)	IT energy (kWh)	Grid import (kWh)	CO_2 (kg)	Effective import intensity (gCO_2/kWh)	Misses
FIFO	0.03	0.08	100.88	24,010.9	30,643.1	7,895	258	0
Priority	0.03	0.08	100.88	24,010.9	30,643.1	7,895	258	0
EFT	0.03	0.08	100.88	24,010.9	30,643.1	7,895	258	0

Table 13: FIFO, Priority and EFT with grid supply, each with 2,523 jobs.

A.6.1 FIFO

The recorded budgets, power and energy outcomes appear in Figure 37, Figure 38.

Figure 37: FIFO policy with grid supply: assigned budgets, planned sources and actor power.

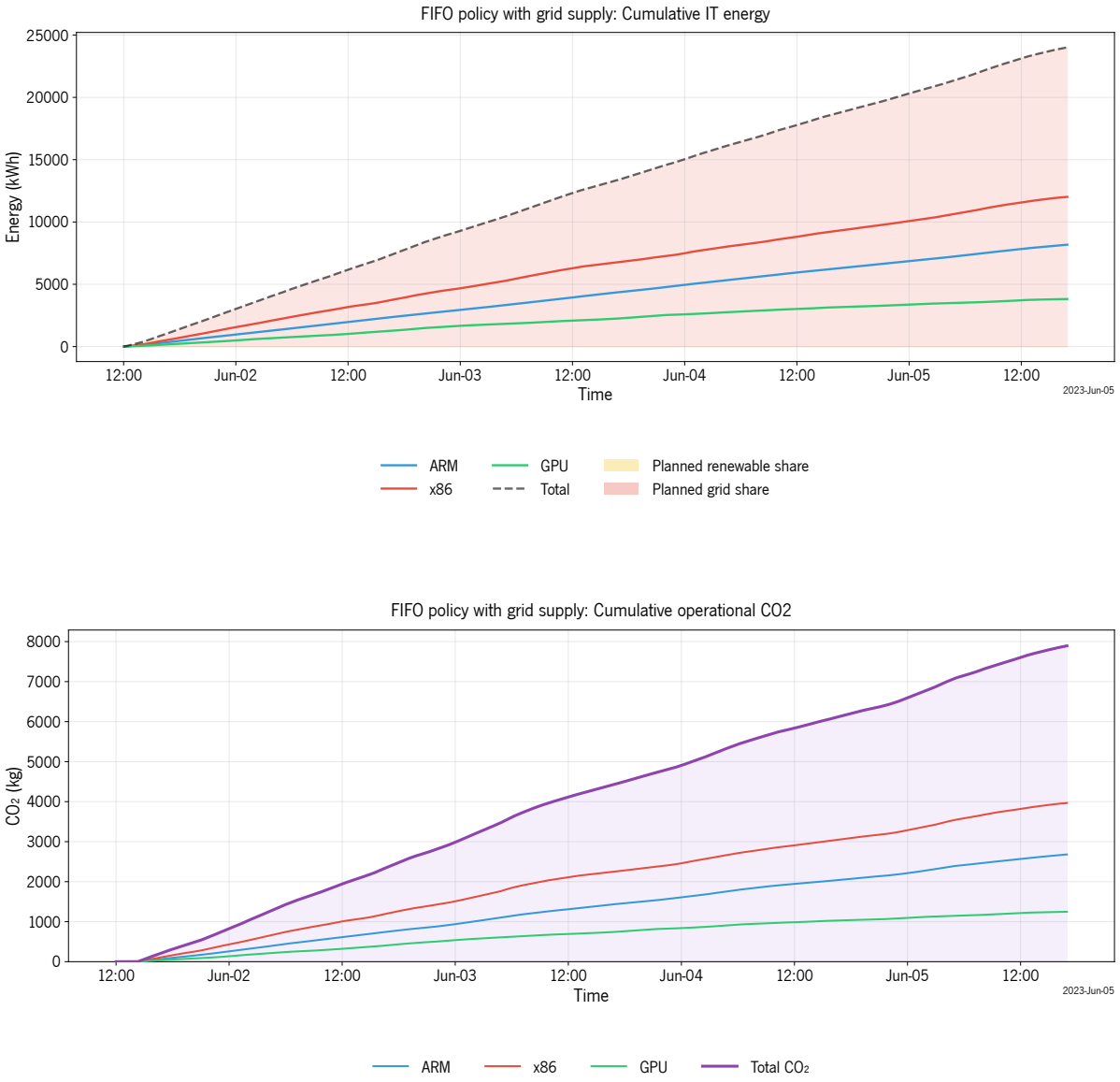


Figure 38: FIFO policy with grid supply: cumulative IT energy and operational emissions.

A.6.2 Priority

The recorded budgets, power and energy outcomes appear in Figure 39, Figure 40.

Figure 39: Priority policy with grid supply: assigned budgets, planned sources and actor power.

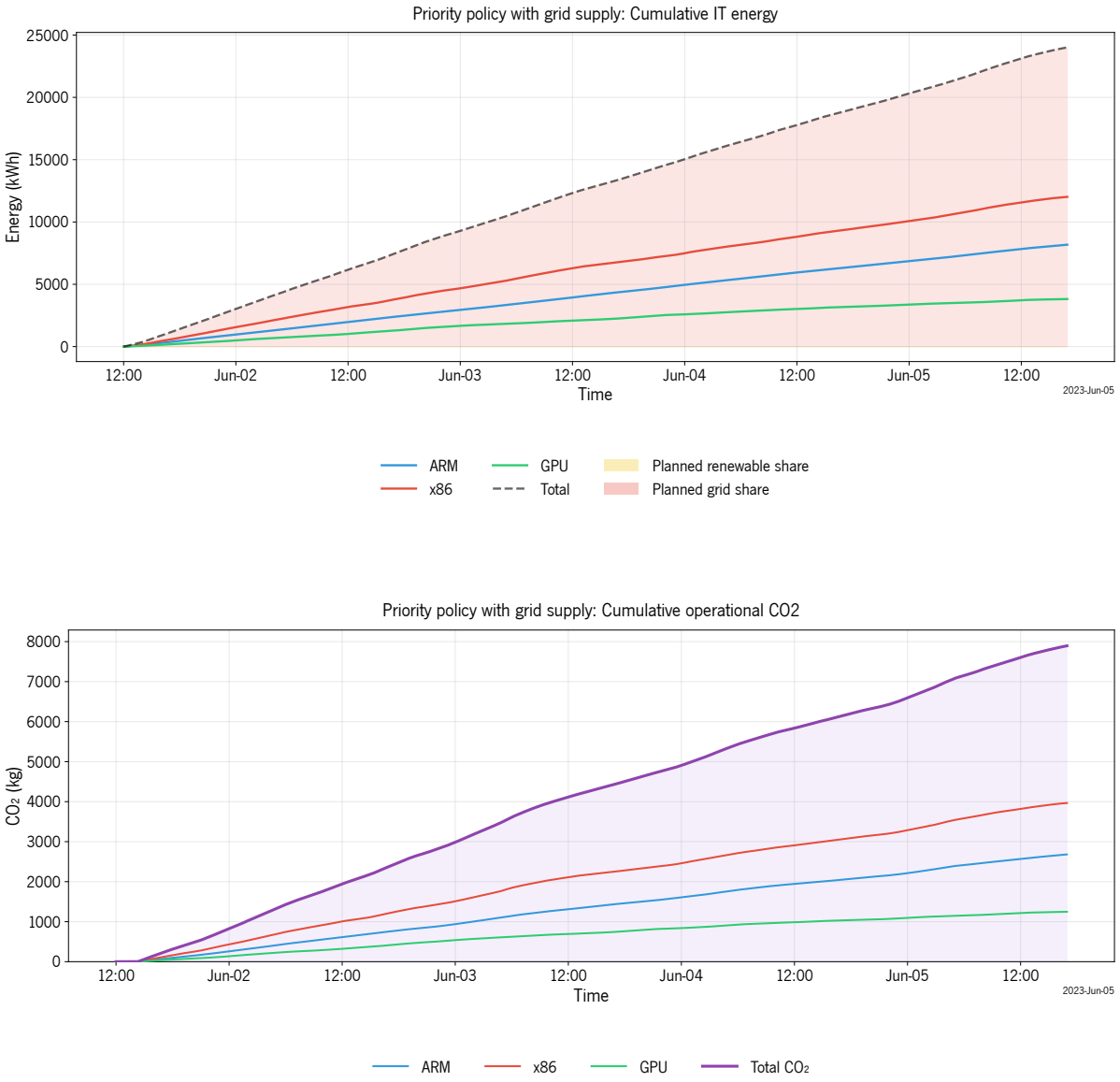


Figure 40: Priority policy with grid supply: cumulative IT energy and operational emissions.

A.6.3 EFT

The recorded budgets, power and energy outcomes appear in Figure 41, Figure 42.

Figure 41: EFT policy with grid supply: assigned budgets, planned sources and actor power.

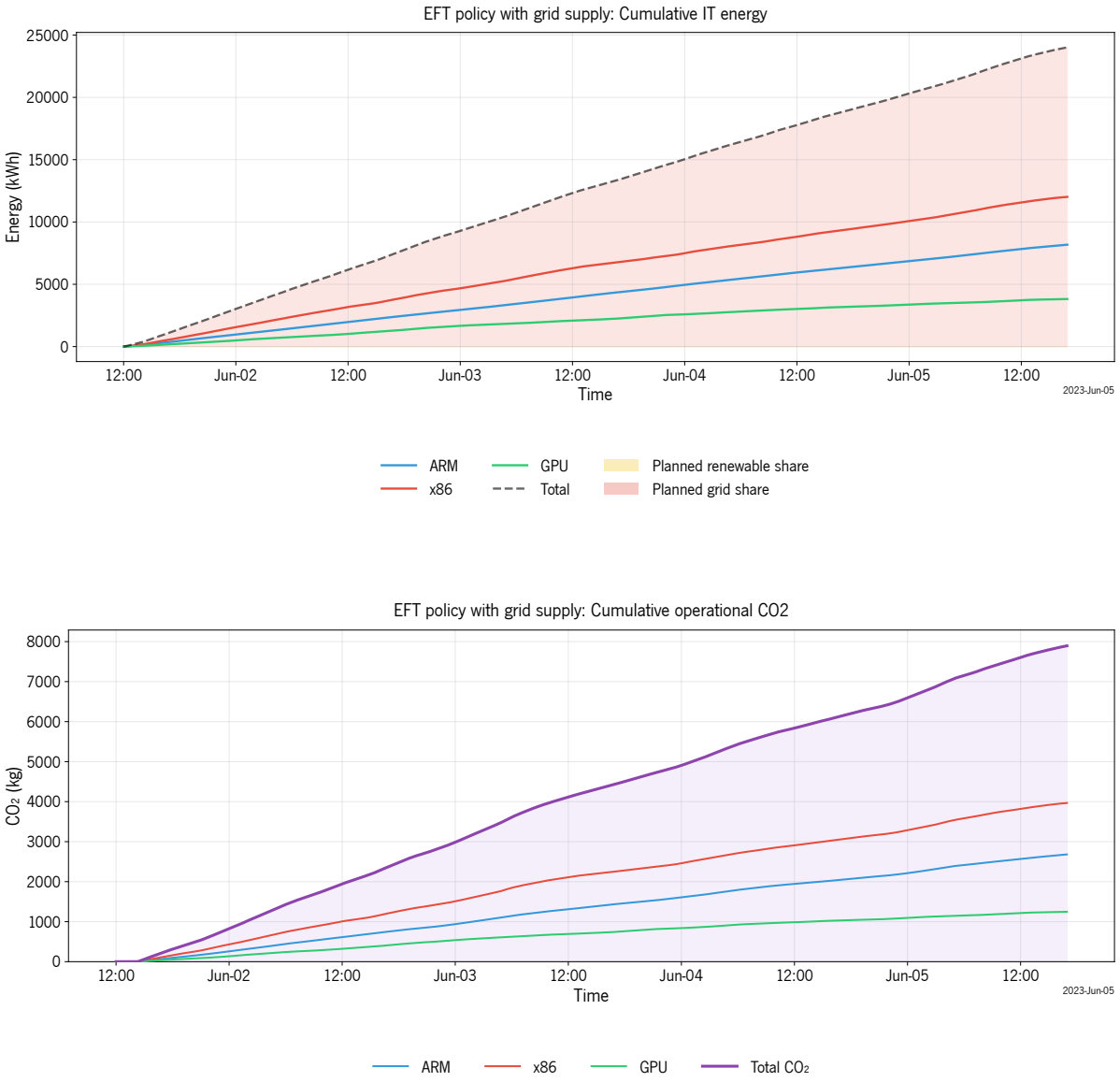


Figure 42: EFT policy with grid supply: cumulative IT energy and operational emissions.

A.6.4 Puzzle

The recorded budgets, power and energy outcomes appear in Figure 43, Figure 44, Figure 45, Figure 46, Figure 47.

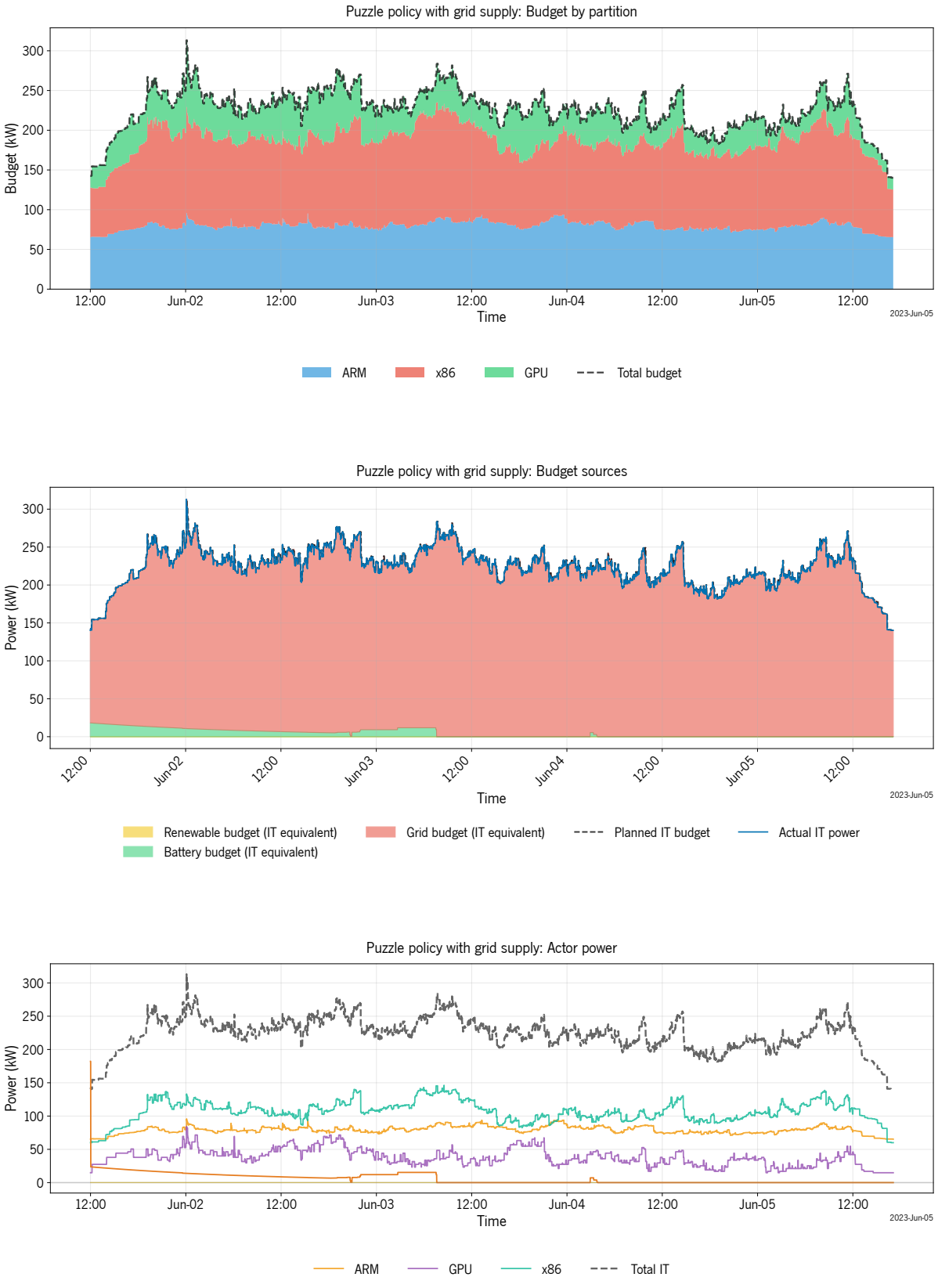


Figure 43: Puzzle with grid supply: assigned budgets, planned sources and actor power.

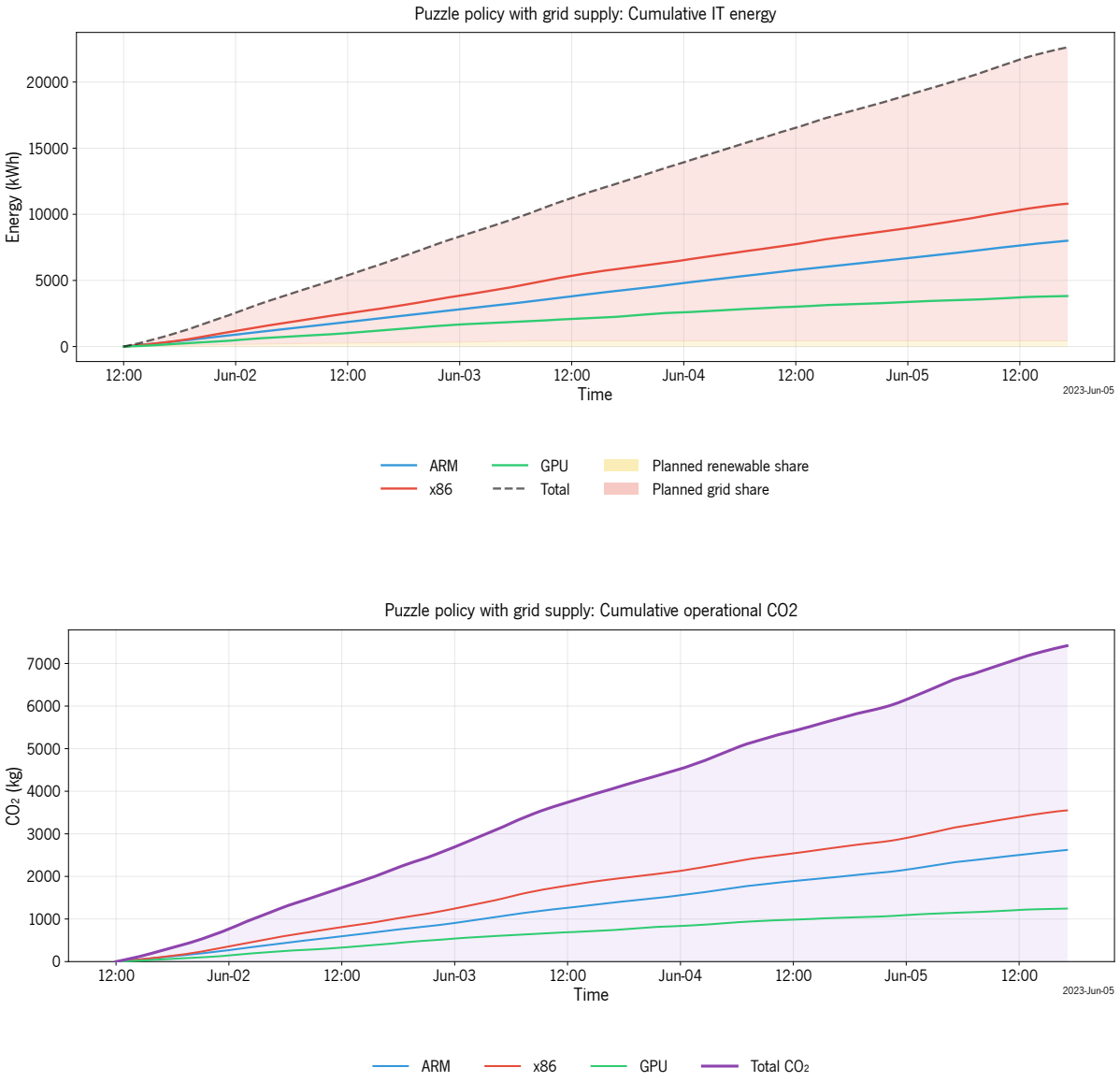


Figure 44: Puzzle with grid supply: cumulative IT energy and operational emissions.

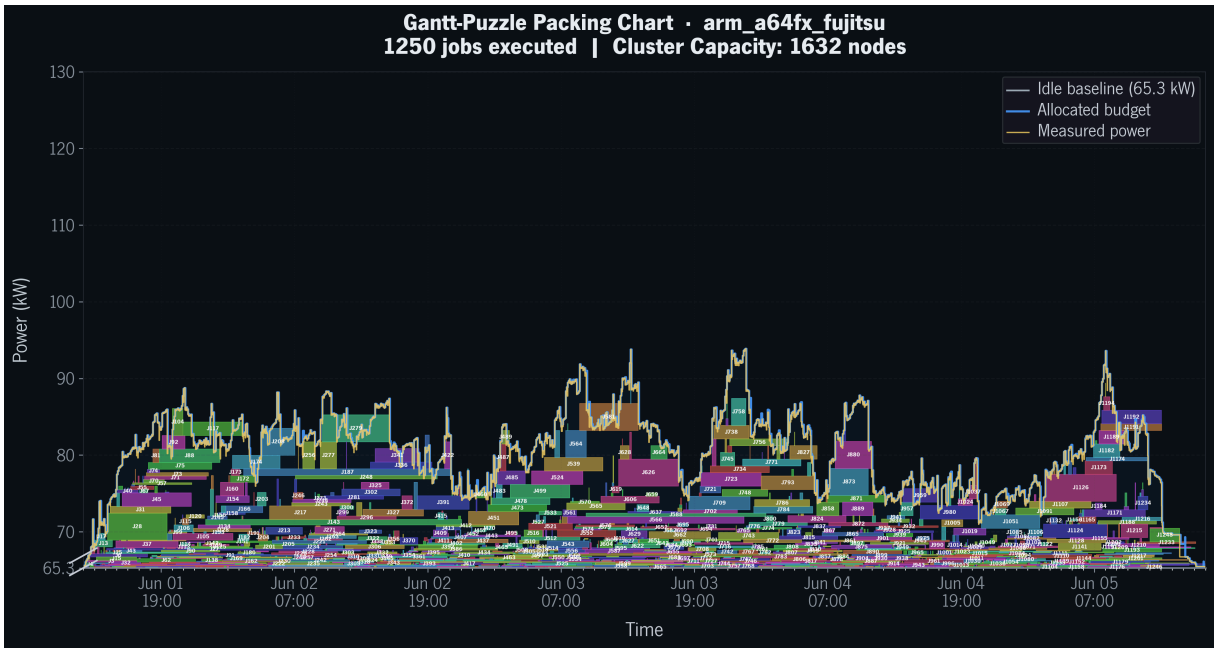


Figure 45: Puzzle execution in the ARM partition with grid supply. The power axis starts at the idle baseline.

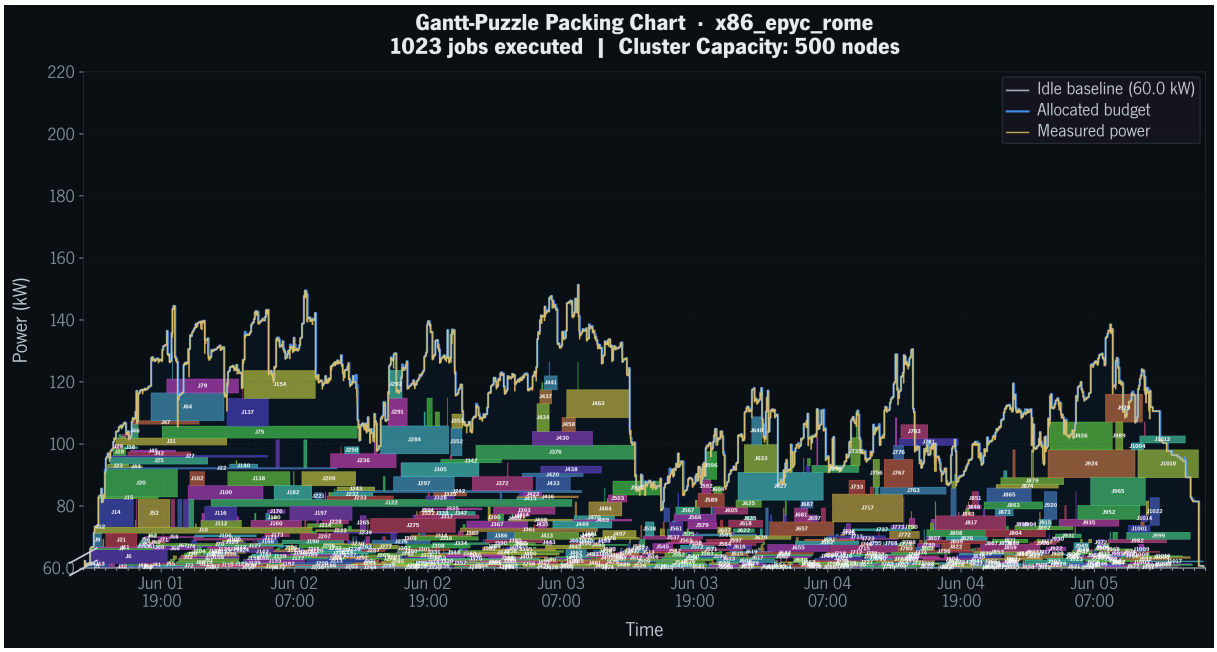


Figure 46: Puzzle execution in the x86 partition with grid supply. The power axis starts at the idle baseline.

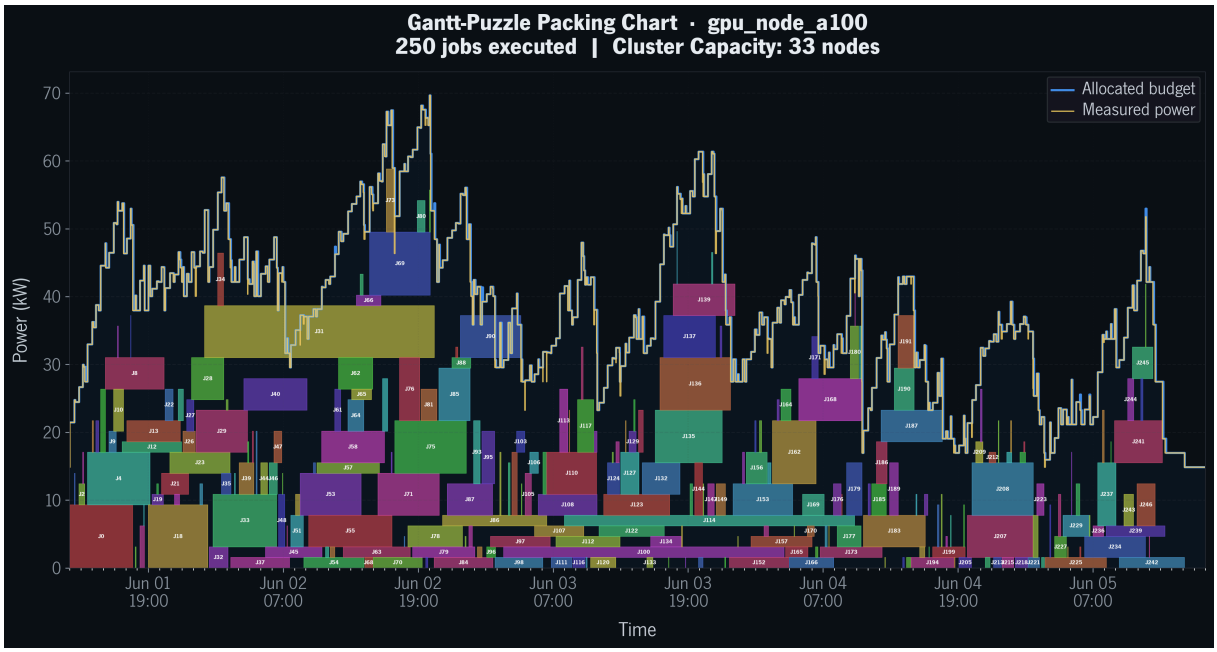


Figure 47: Puzzle execution in the GPU partition with grid supply.

A.7 PV supply

The reference-policy totals are compared in Table 14. Pale green and pale red mark the lowest and highest values in each numerical column. Ties share the highlight, while columns with identical values remain unshaded.

Policy	Mean wait (h)	P95 wait (h)	Makespan (h)	IT energy (kWh)	Grid import (kWh)	CO_2 (kg)	Effective import intensity (gCO_2/kWh)	Misses
FIFO	**2.52**	**9.50**	**108.89**	**24,291.6**	**18,211.5**	**4,636**	**255**	0
Priority	2.25	**8.73**	101.35	23,208.5	**17,065.2**	4,371	**256**	0
EFT	**2.20**	**8.73**	**101.30**	**23,194.7**	17,068.4	**4,362**	**256**	0

Table 14: FIFO, Priority and EFT with PV, each with 2,523 jobs.

A.7.1 FIFO

The recorded budgets, power and energy outcomes appear in Figure 48, Figure 49.

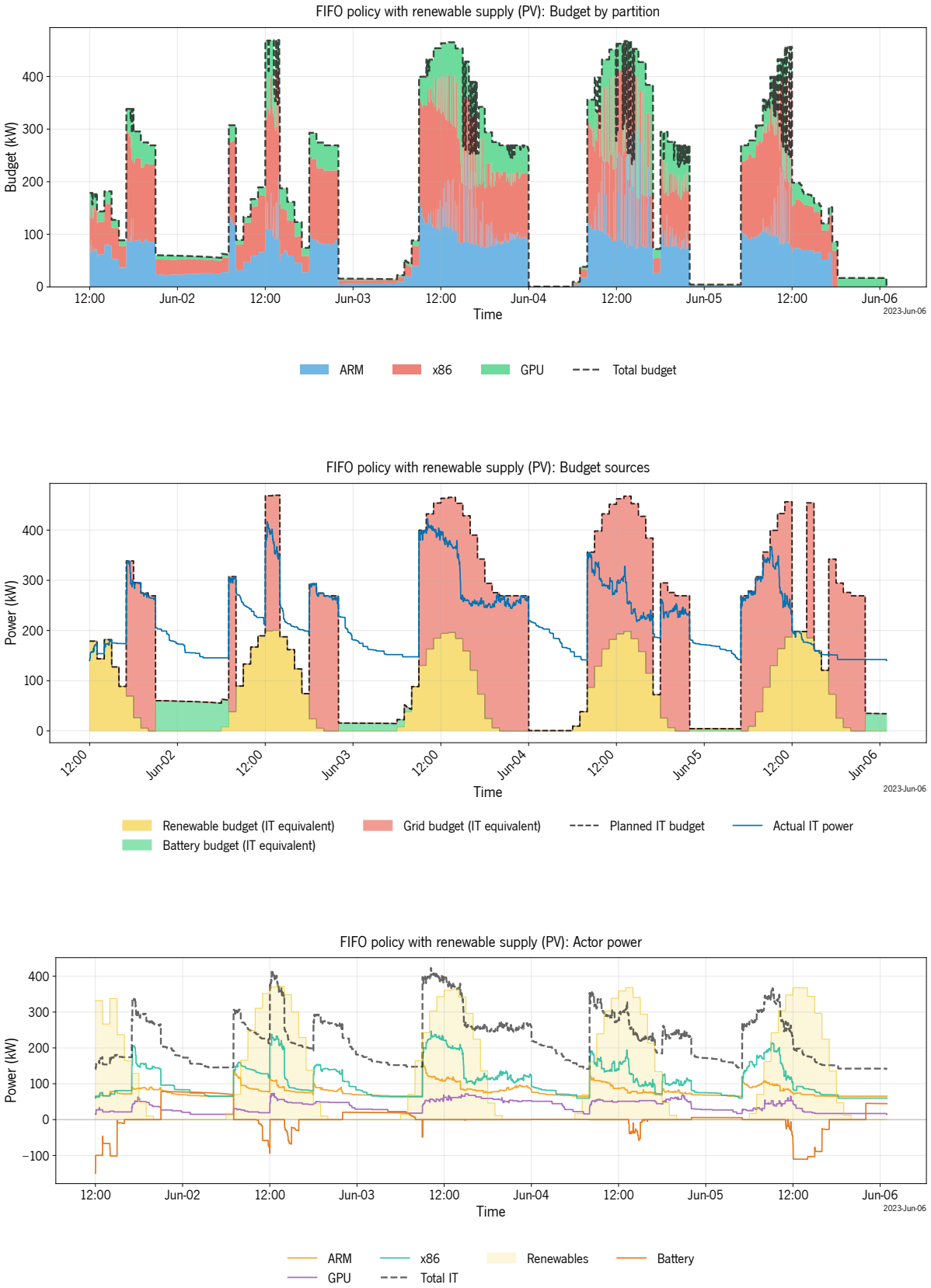


Figure 48: FIFO policy with renewable supply (PV): assigned budgets, planned sources and actor power.

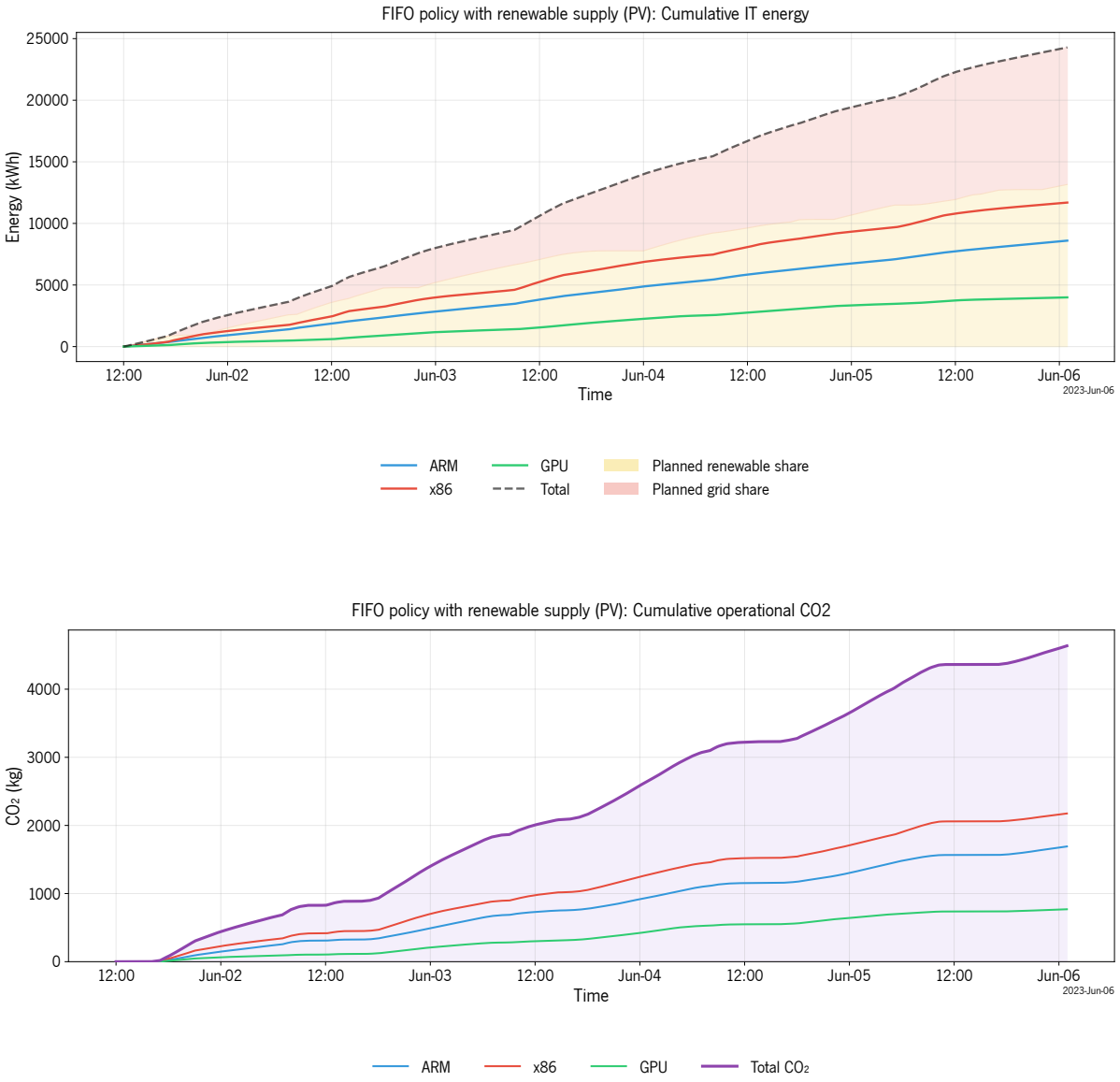


Figure 49: FIFO policy with renewable supply (PV): cumulative IT energy and operational emissions.

A.7.2 Priority

The recorded budgets, power and energy outcomes appear in Figure 50, Figure 51.

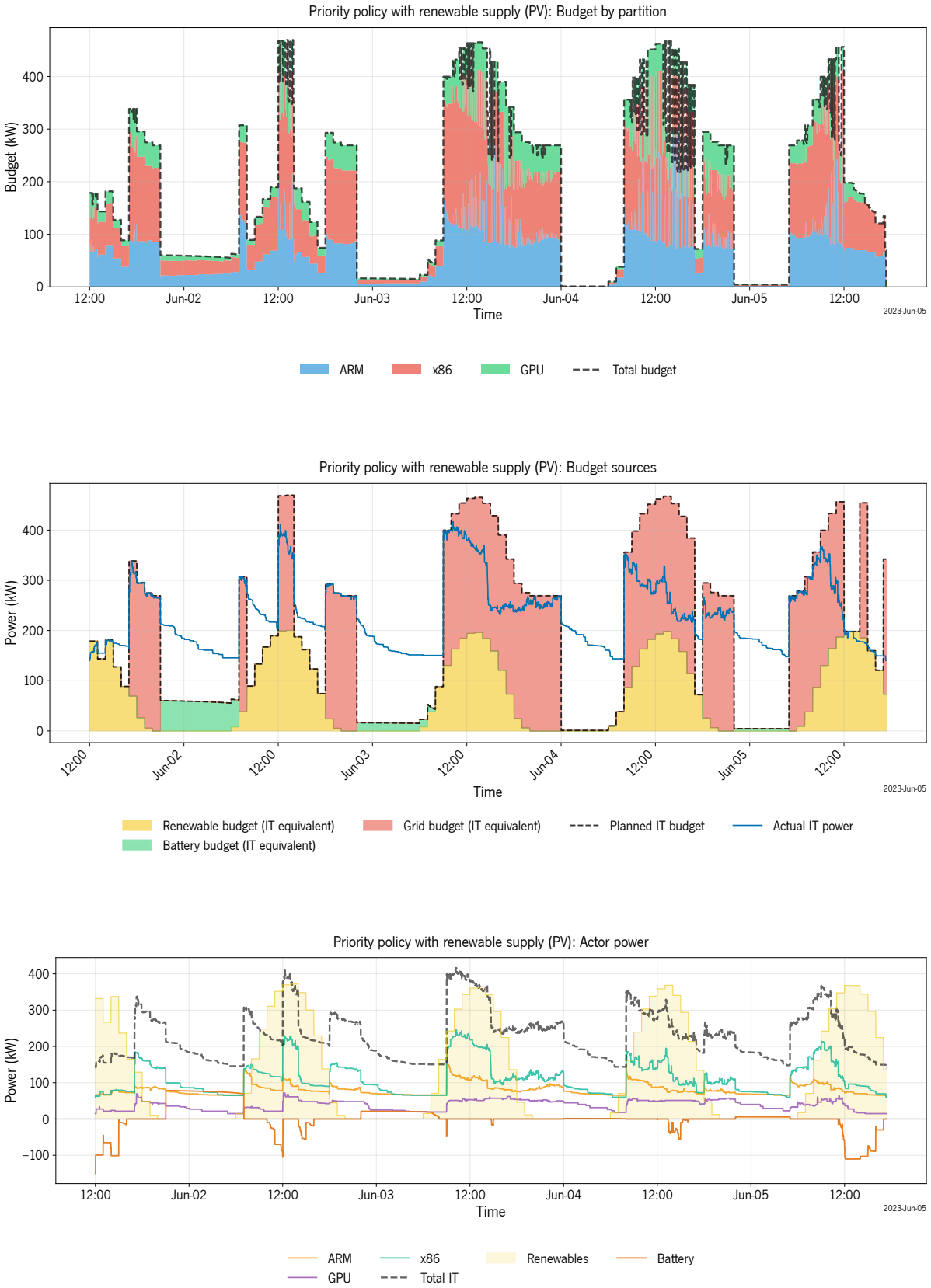


Figure 50: Priority policy with renewable supply (PV): assigned budgets, planned sources and actor power.

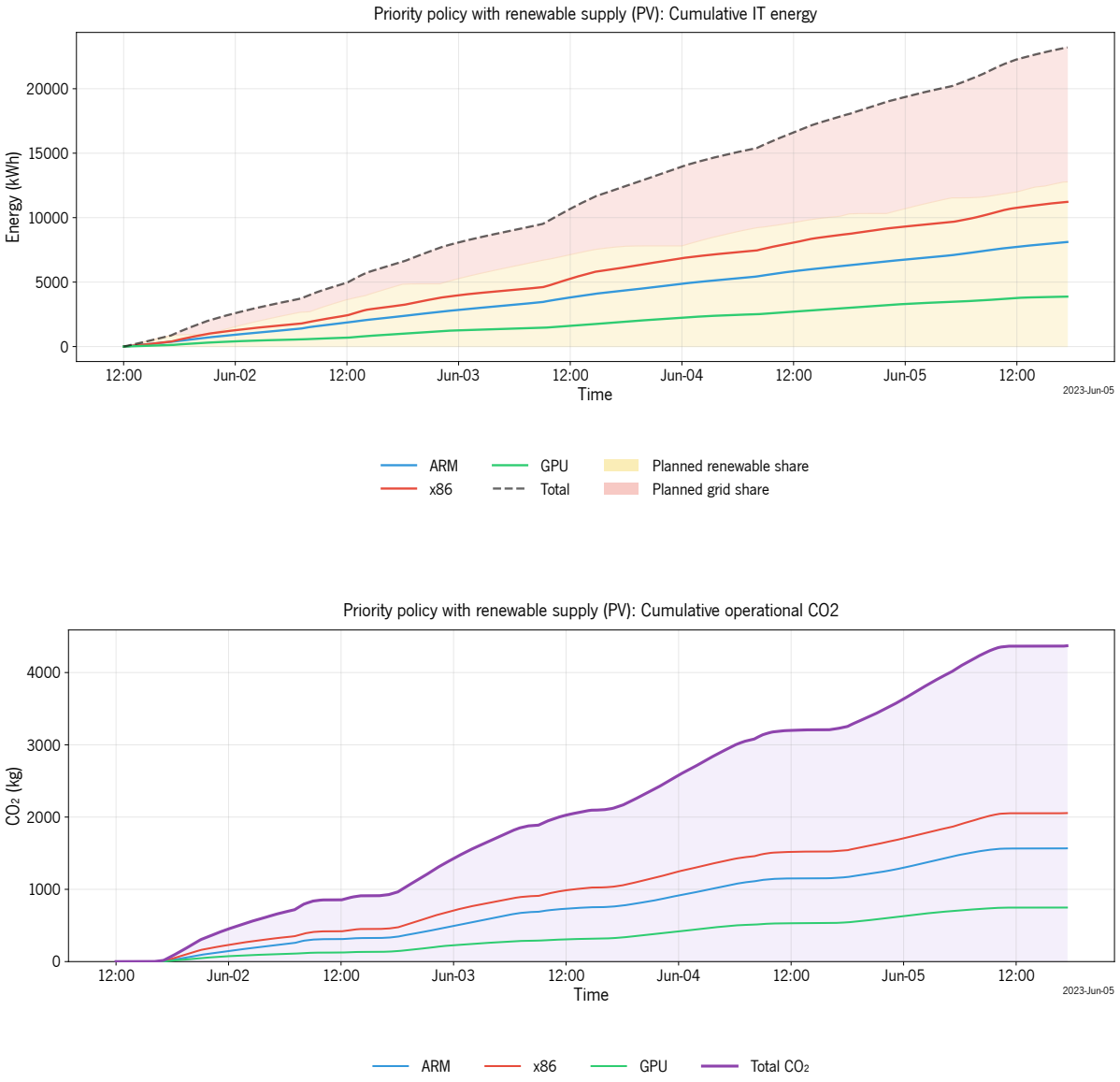


Figure 51: Priority policy with renewable supply (PV): cumulative IT energy and operational emissions.

A.7.3 EFT

The recorded budgets, power and energy outcomes appear in Figure 52, Figure 53.

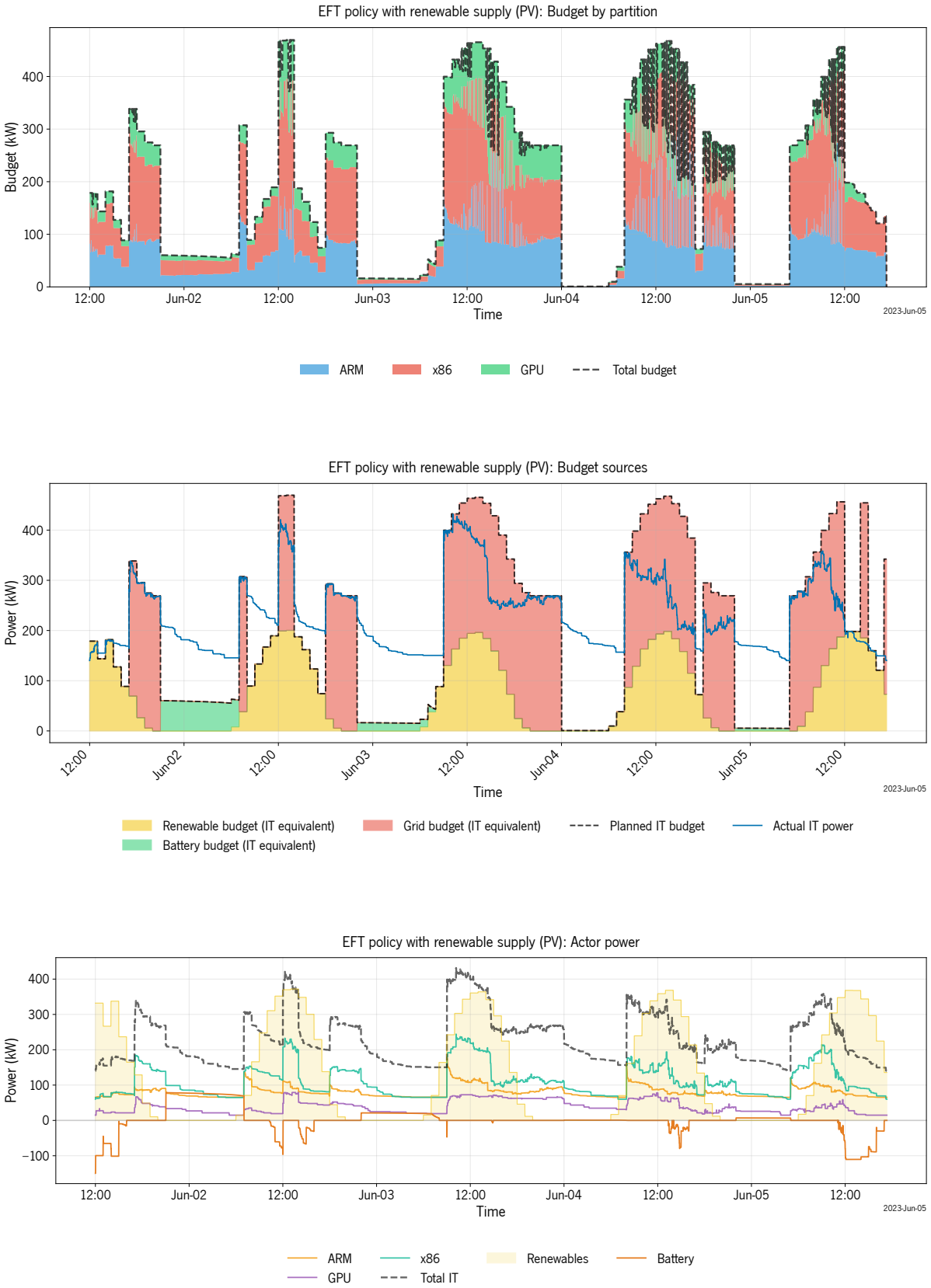


Figure 52: EFT policy with renewable supply (PV): assigned budgets, planned sources and actor power.

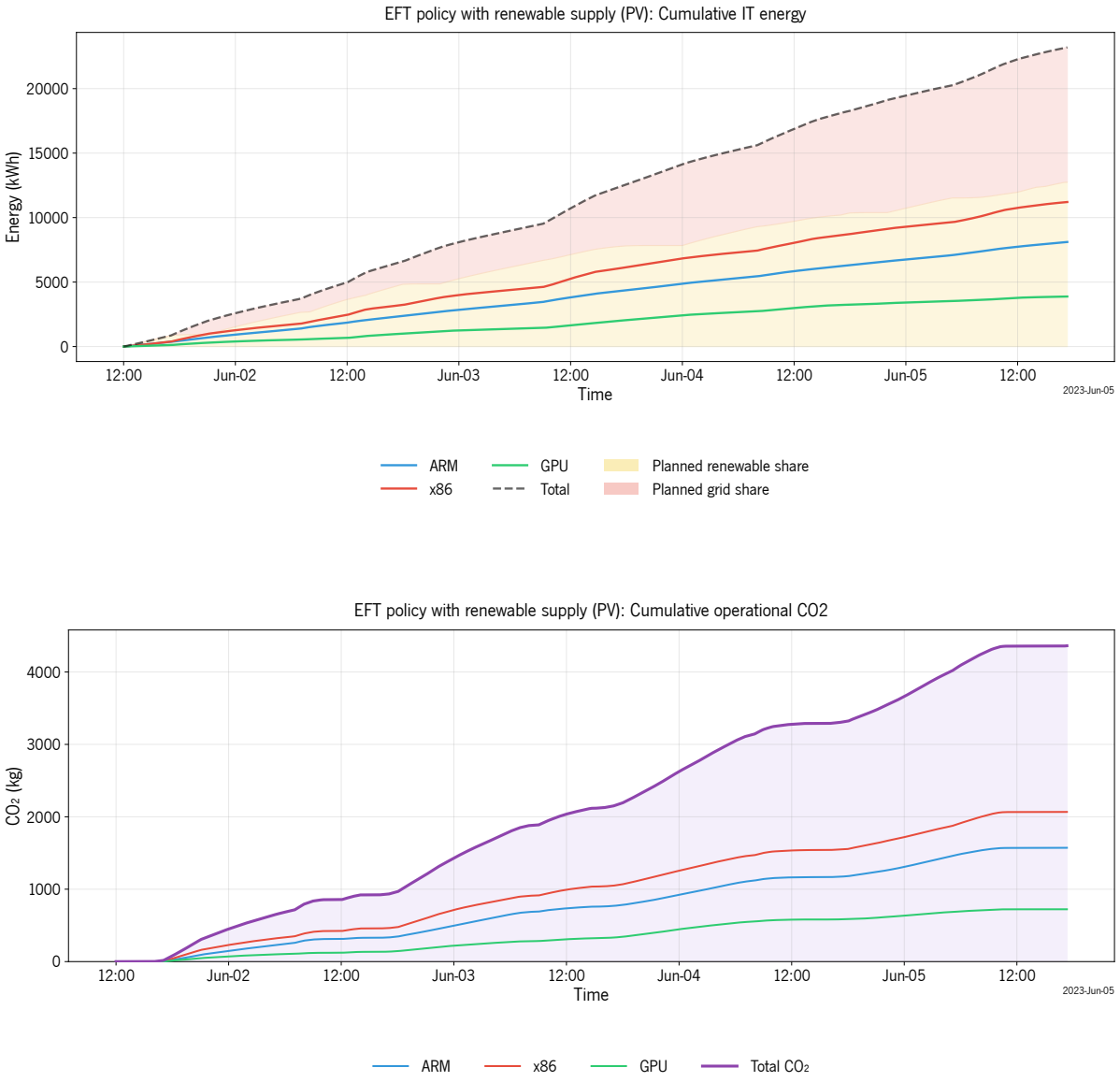


Figure 53: EFT policy with renewable supply (PV): cumulative IT energy and operational emissions.

A.7.4 Puzzle

The recorded budgets, power and energy outcomes appear in Figure 54, Figure 55, Figure 56, Figure 57, Figure 58.

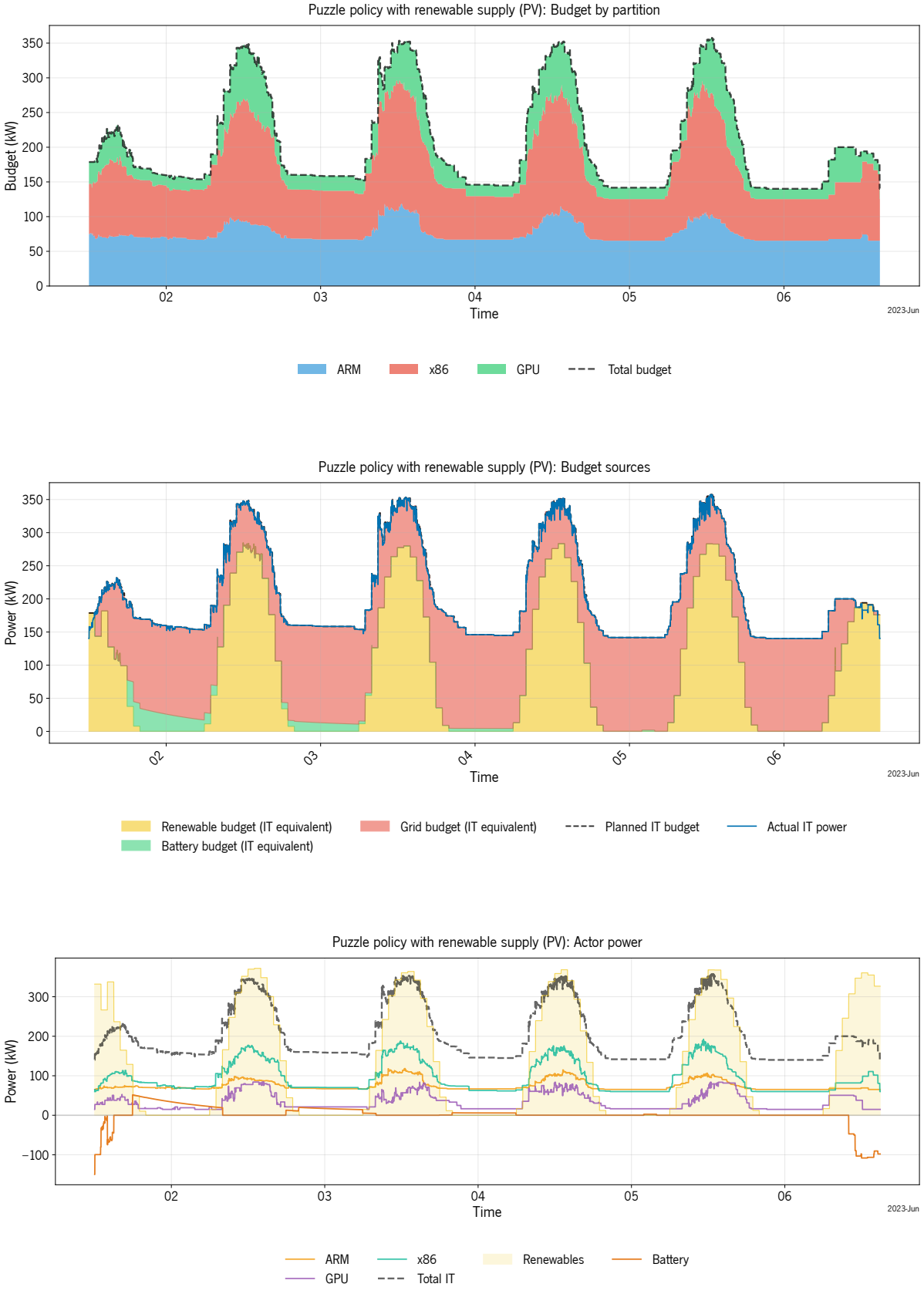


Figure 54: Puzzle with renewable supply (PV): assigned budgets, planned sources and actor power.

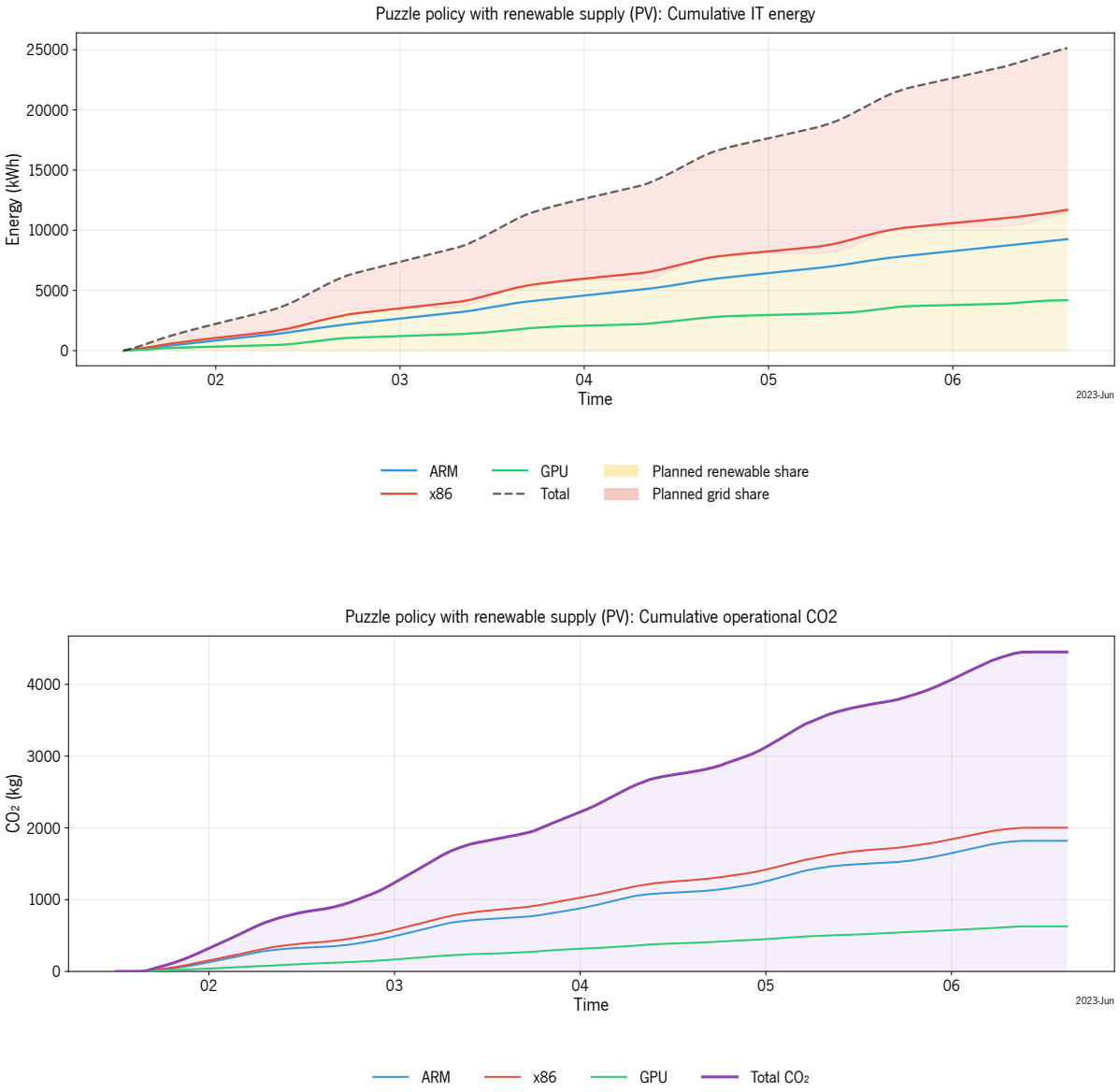


Figure 55: Puzzle with renewable supply (PV): cumulative IT energy and operational emissions.

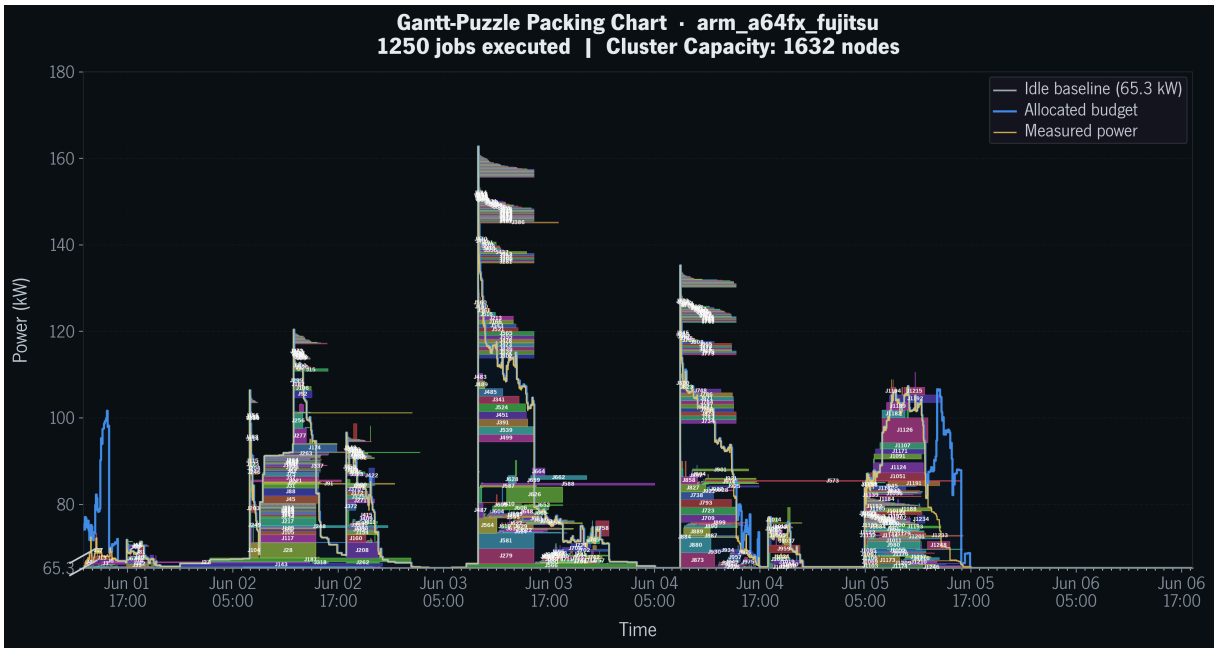


Figure 56: Puzzle execution in the ARM partition with renewable supply (PV). The power axis starts at the idle baseline.

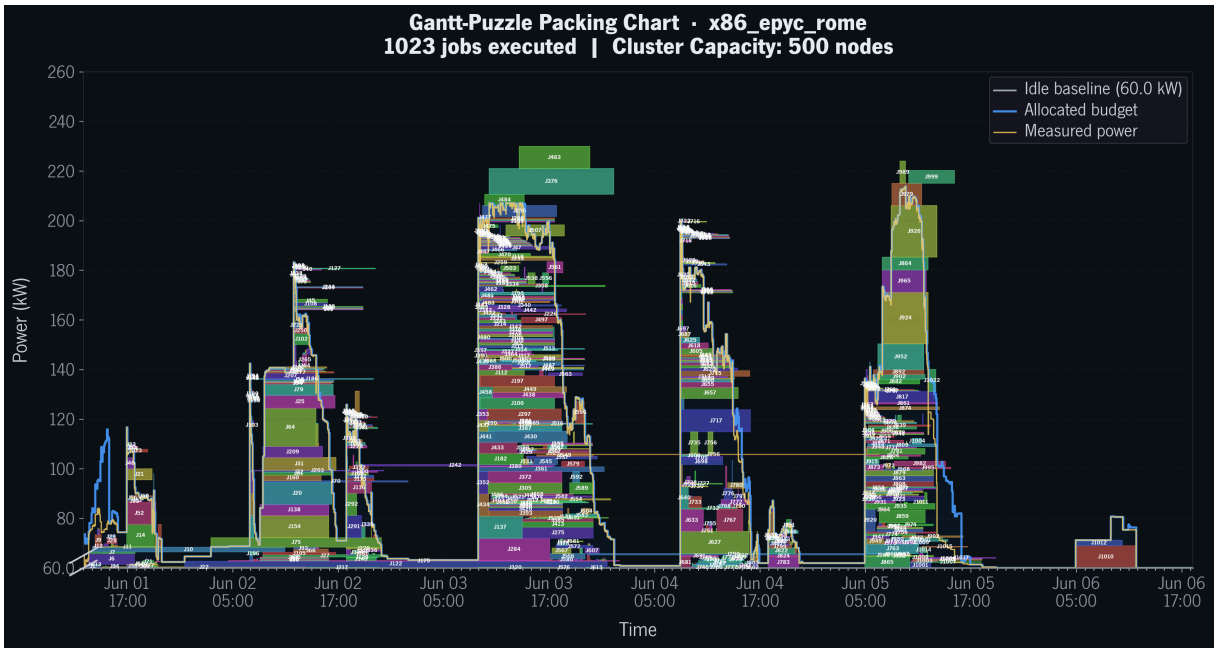


Figure 57: Puzzle execution in the x86 partition with renewable supply (PV). The power axis starts at the idle baseline.

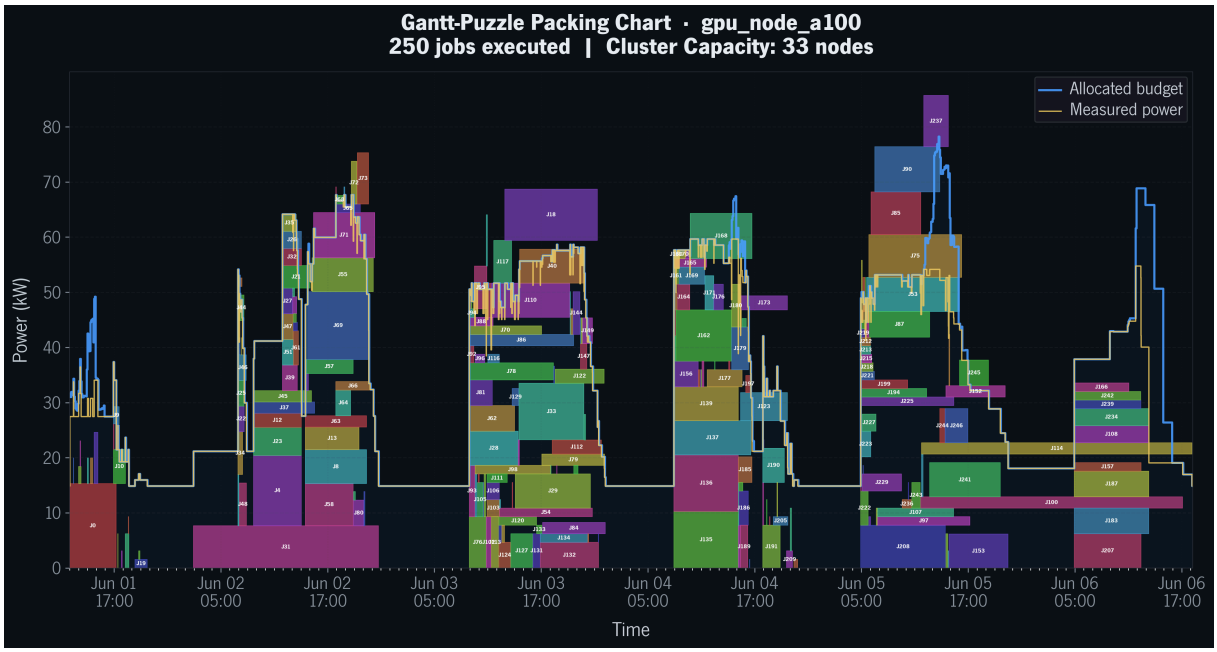


Figure 58: Puzzle execution in the GPU partition with renewable supply (PV).

The GPU, x86 and ARM partitions finished at 126.16 h, 119.80 h and 101.03 h, respectively, leaving a spread of 25.13 h. The extended run is therefore driven primarily by the GPU partition, while the ARM work finishes much earlier.

A.8 PV and wind supply

The reference-policy totals are compared in Table 15. Pale green and pale red mark the lowest and highest values in each numerical column. Ties share the highlight, while columns with identical values remain unshaded.

Policy	Mean wait (h)	P95 wait (h)	Makespan (h)	IT energy (kWh)	Grid import (kWh)	CO_2 (kg)	Effective import intensity (gCO_2/kWh)	Misses
FIFO	**1.90**	**8.48**	**100.87**	23,369.0	6,956.1	1,683	**242**	0
Priority	1.81	8.38	**100.86**	**23,360.9**	**6,862.6**	**1,664**	**242**	0
EFT	**1.80**	**8.37**	**100.86**	**23,383.4**	**7,035.1**	**1,695**	**241**	0

Table 15: FIFO, Priority and EFT with PV and wind, each with 2,523 jobs.

A.8.1 FIFO

The recorded budgets, power and energy outcomes appear in Figure 59, Figure 60.

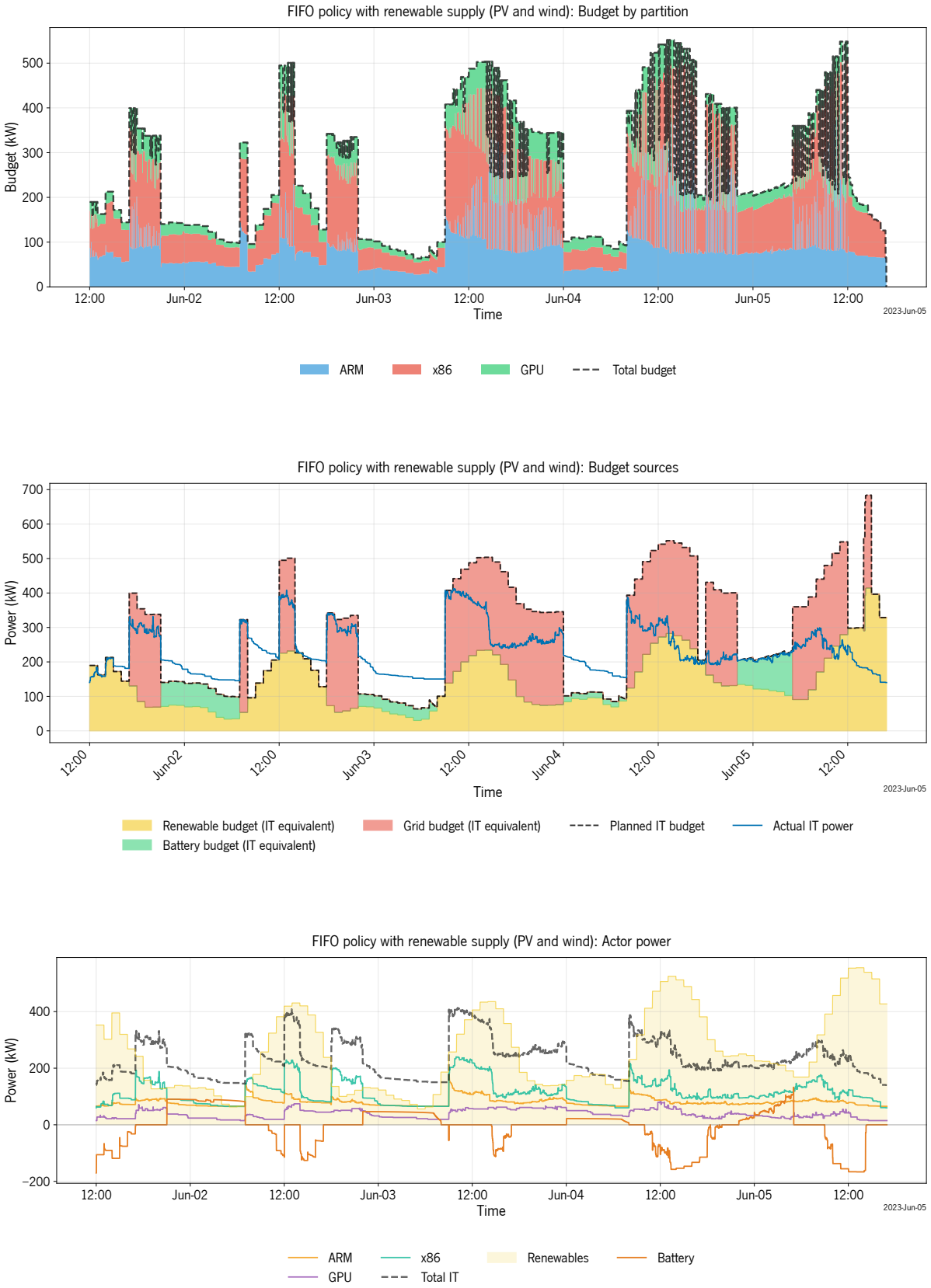


Figure 59: FIFO policy with renewable supply (PV and wind): assigned budgets, planned sources and actor power.

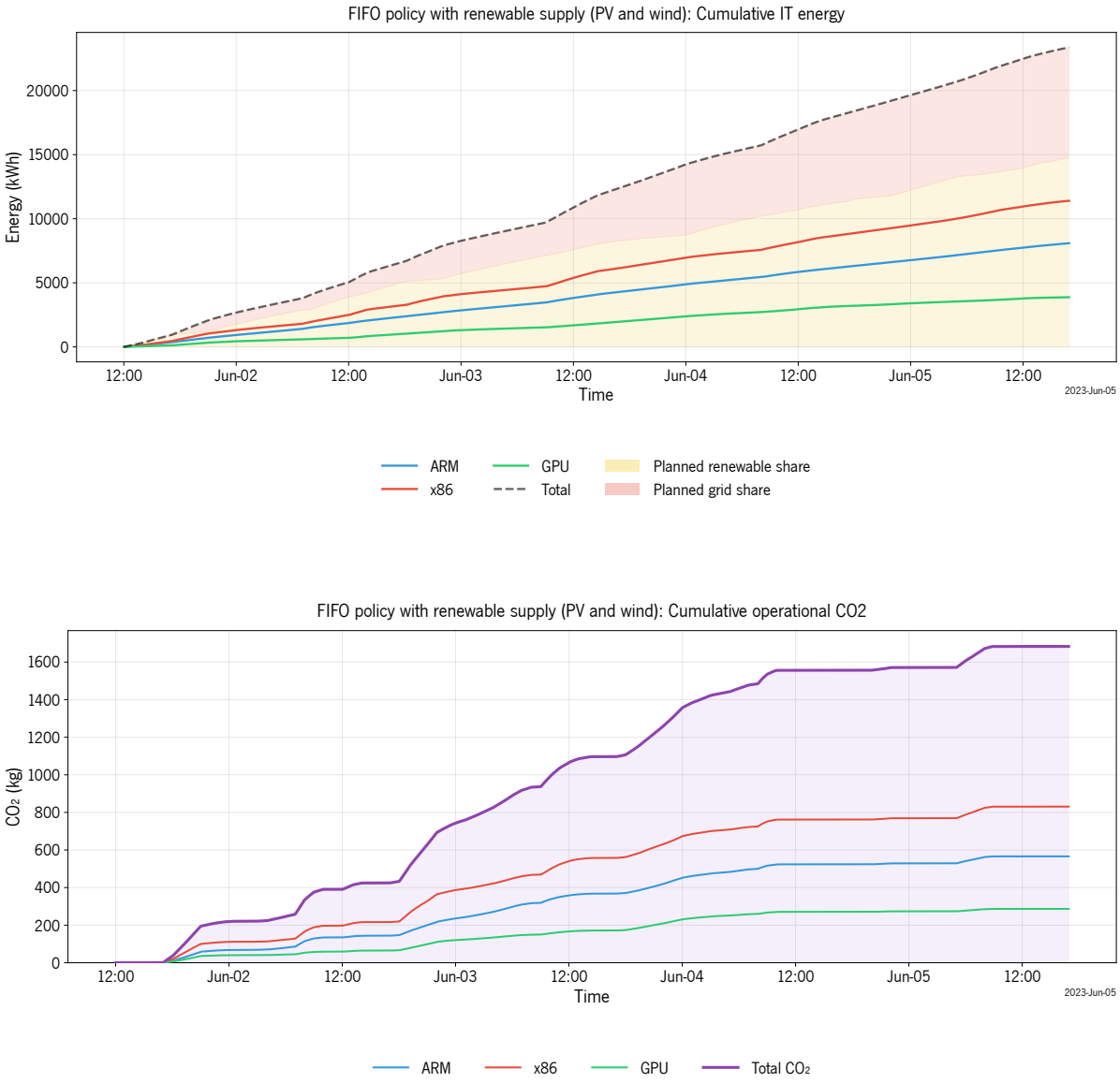


Figure 60: FIFO policy with renewable supply (PV and wind): cumulative IT energy and operational emissions.

A.8.2 Priority

The recorded budgets, power and energy outcomes appear in Figure 61, Figure 62.

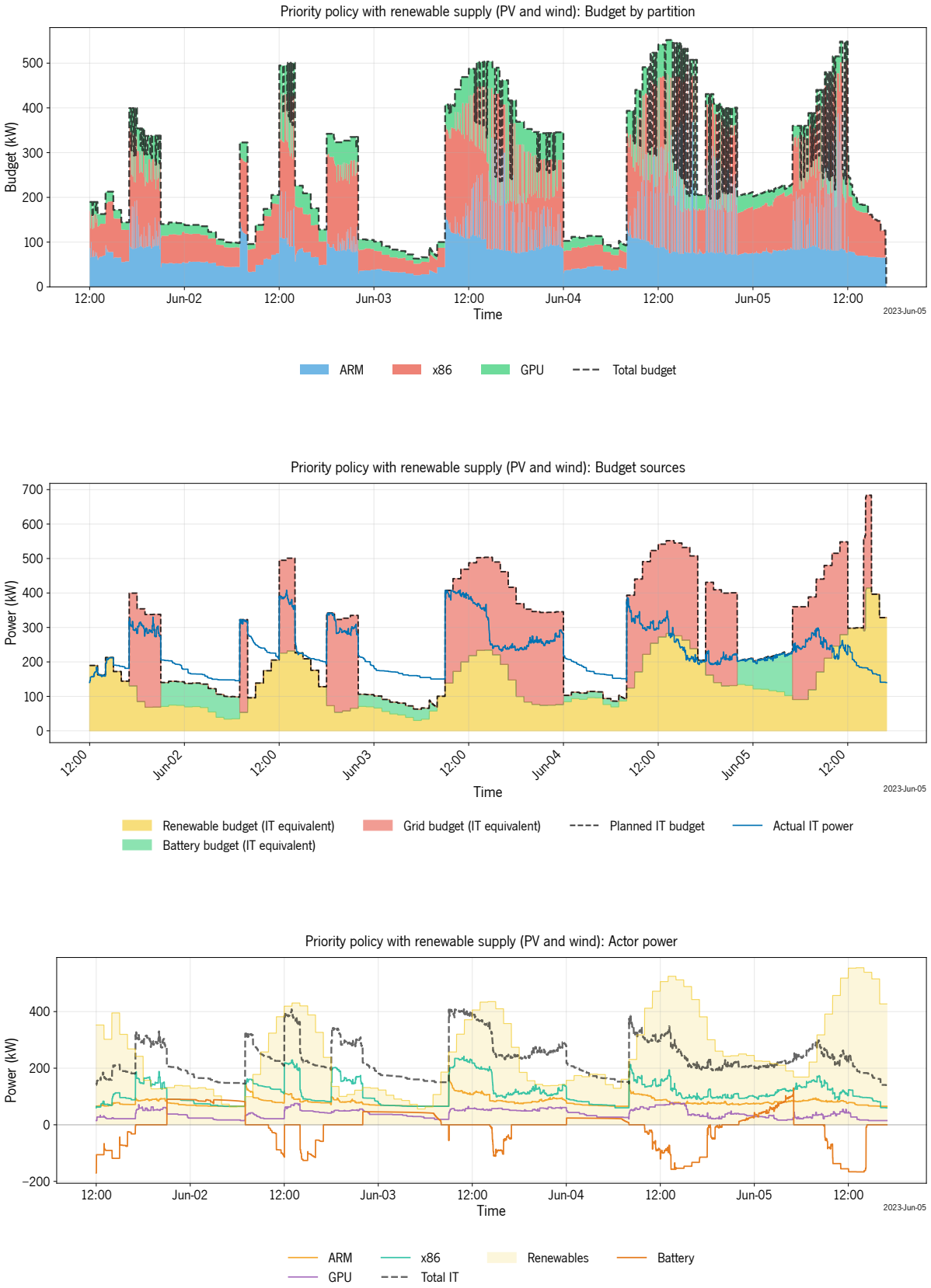


Figure 61: Priority policy with renewable supply (PV and wind): assigned budgets, planned sources and actor power.

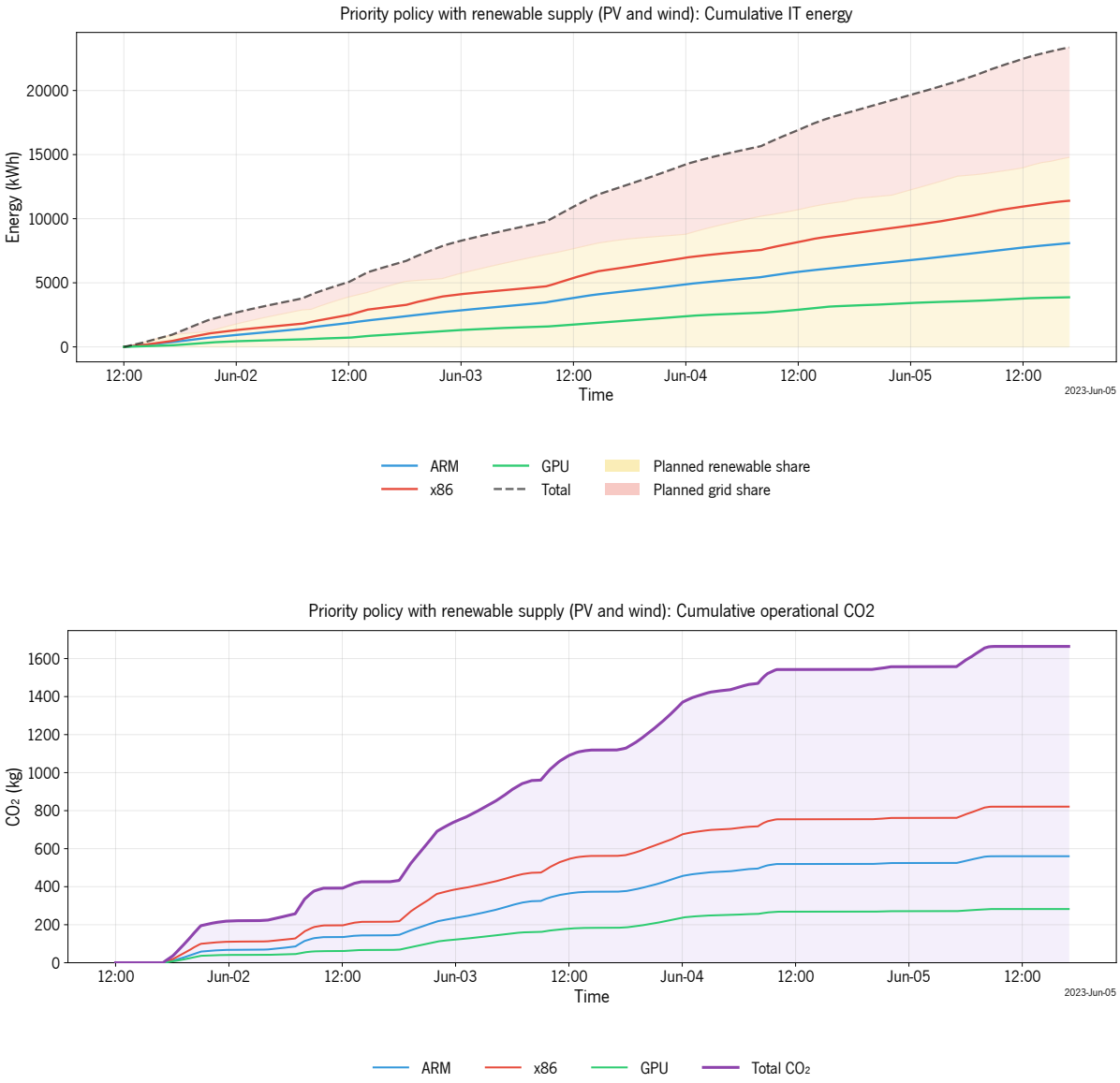


Figure 62: Priority policy with renewable supply (PV and wind): cumulative IT energy and operational emissions.

A.8.3 EFT

The recorded budgets, power and energy outcomes appear in Figure 63, Figure 64.

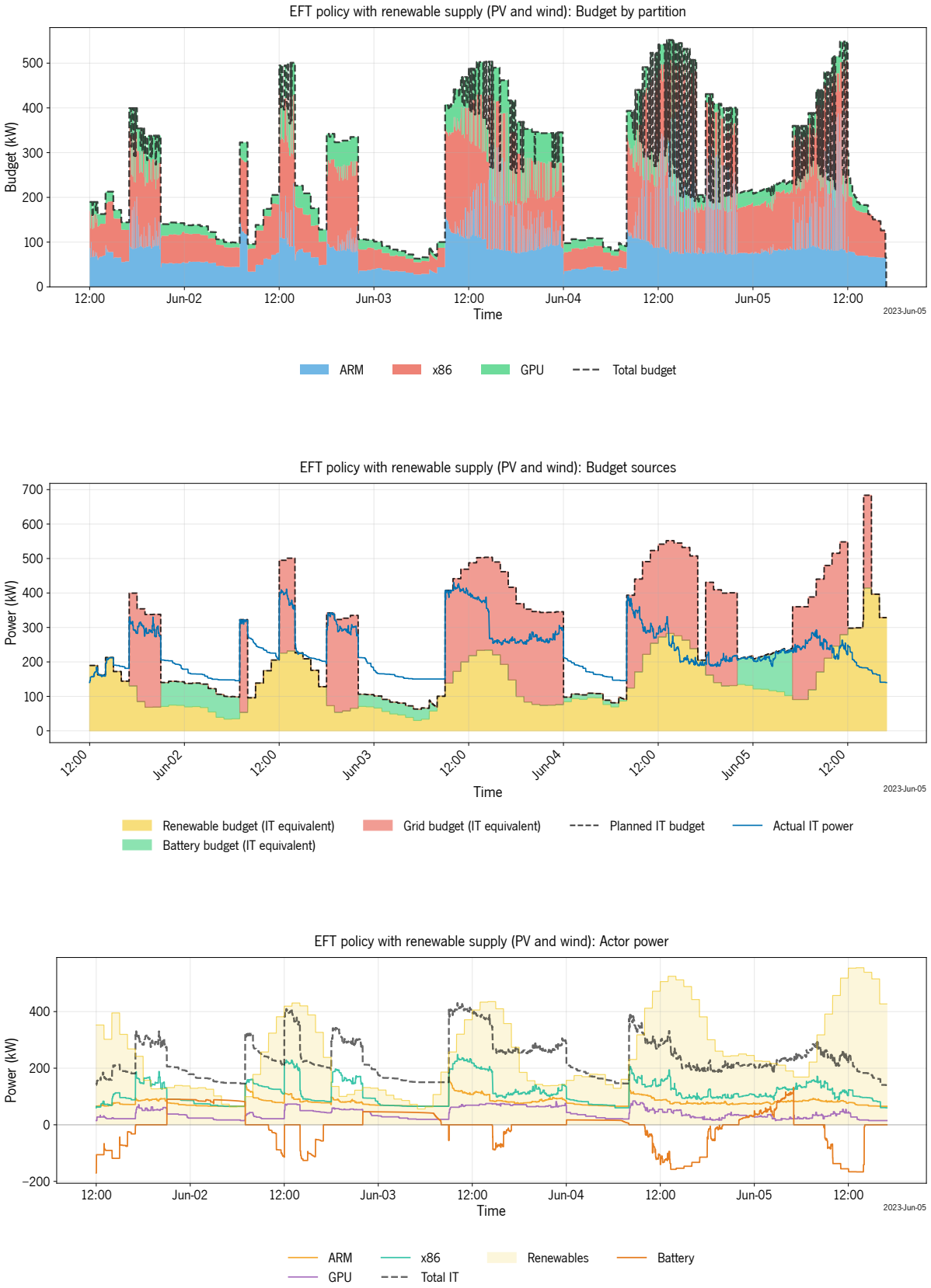


Figure 63: EFT policy with renewable supply (PV and wind): assigned budgets, planned sources and actor power.

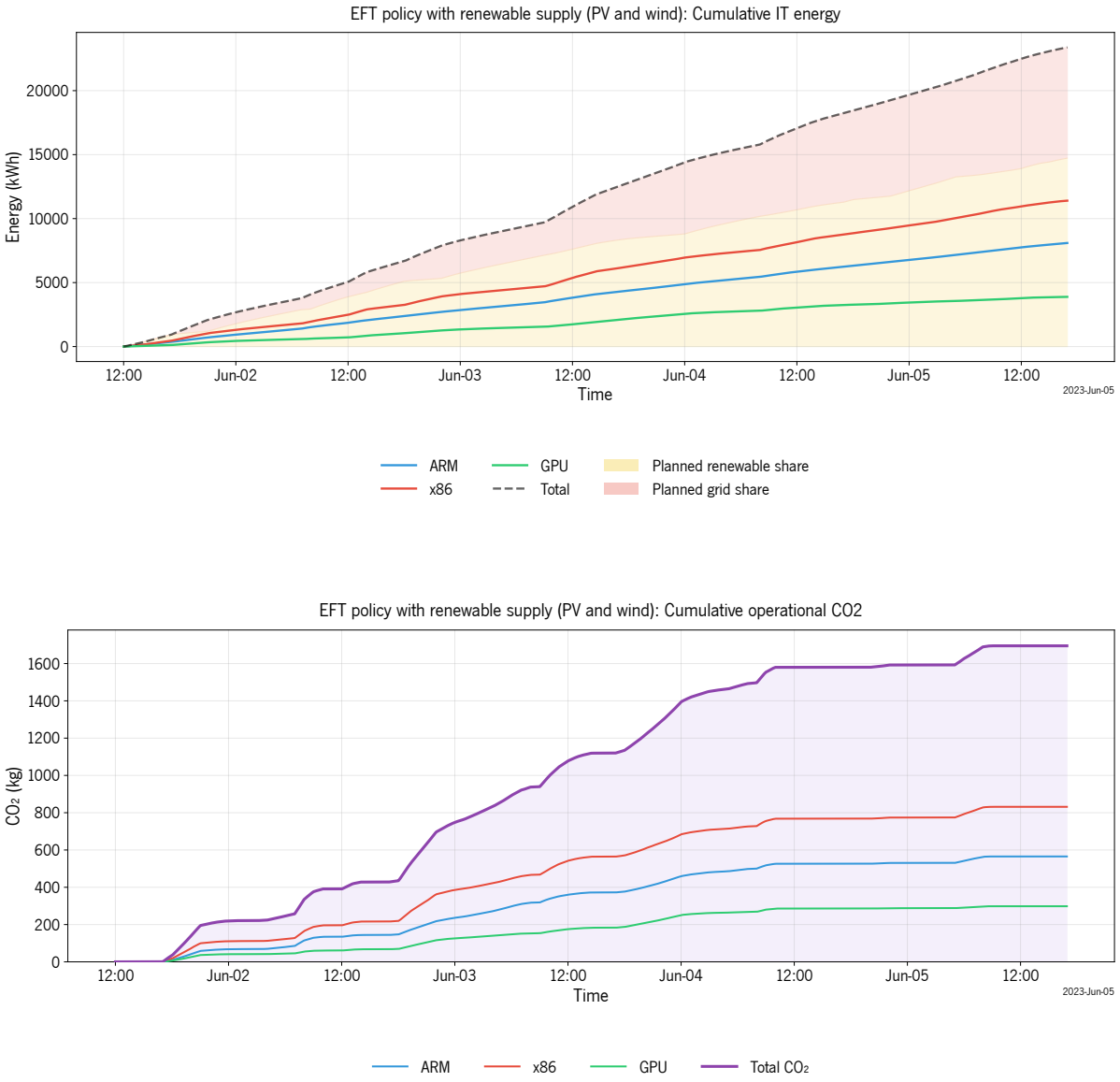


Figure 64: EFT policy with renewable supply (PV and wind): cumulative IT energy and operational emissions.

A.8.4 Puzzle

The recorded budgets, power and energy outcomes appear in Figure 65, Figure 66, Figure 67, Figure 68.

Figure 65: Puzzle with renewable supply (PV and wind): assigned budgets, planned sources and actor power.

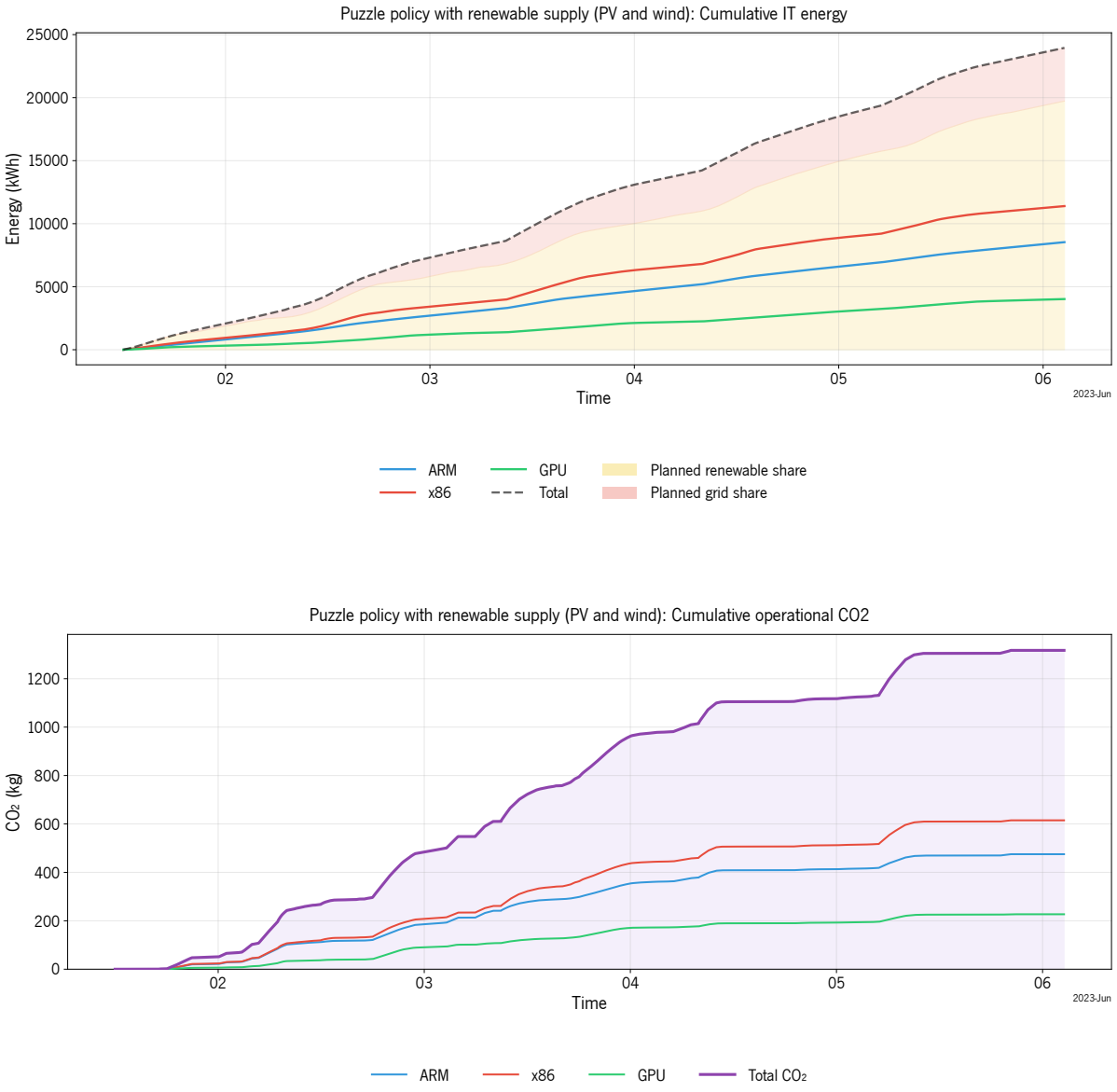


Figure 66: Puzzle with renewable supply (PV and wind): cumulative IT energy and operational emissions.

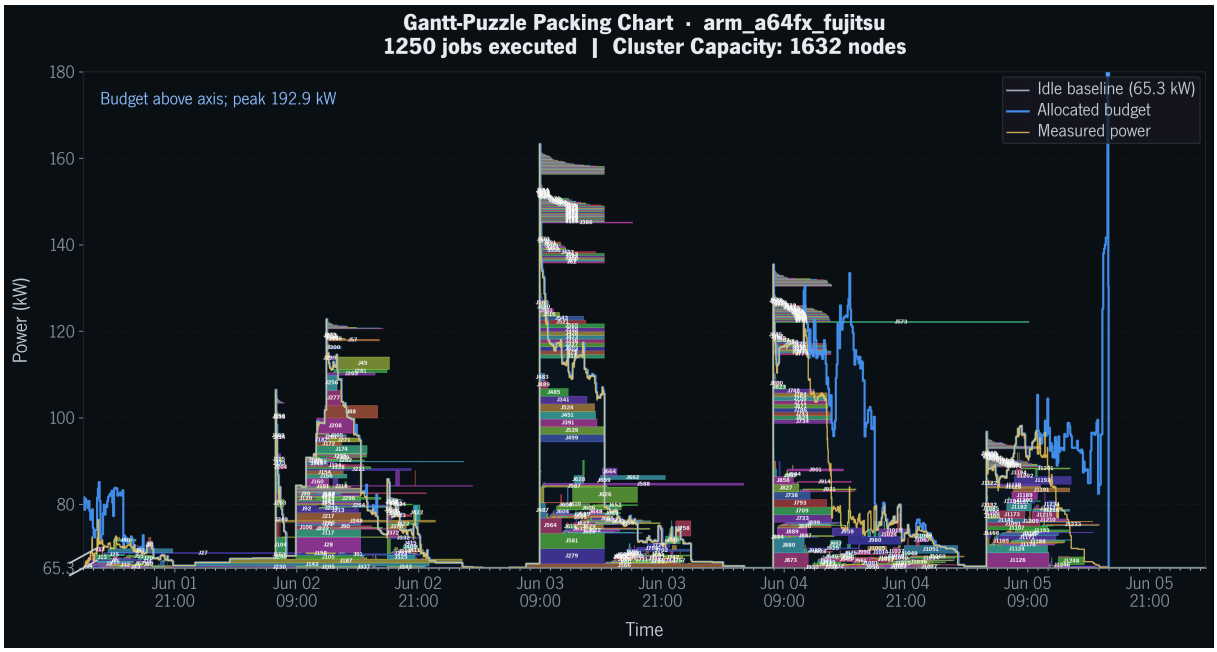


Figure 67: Puzzle execution in the ARM partition with renewable supply (PV and wind). The power axis starts at the idle baseline.

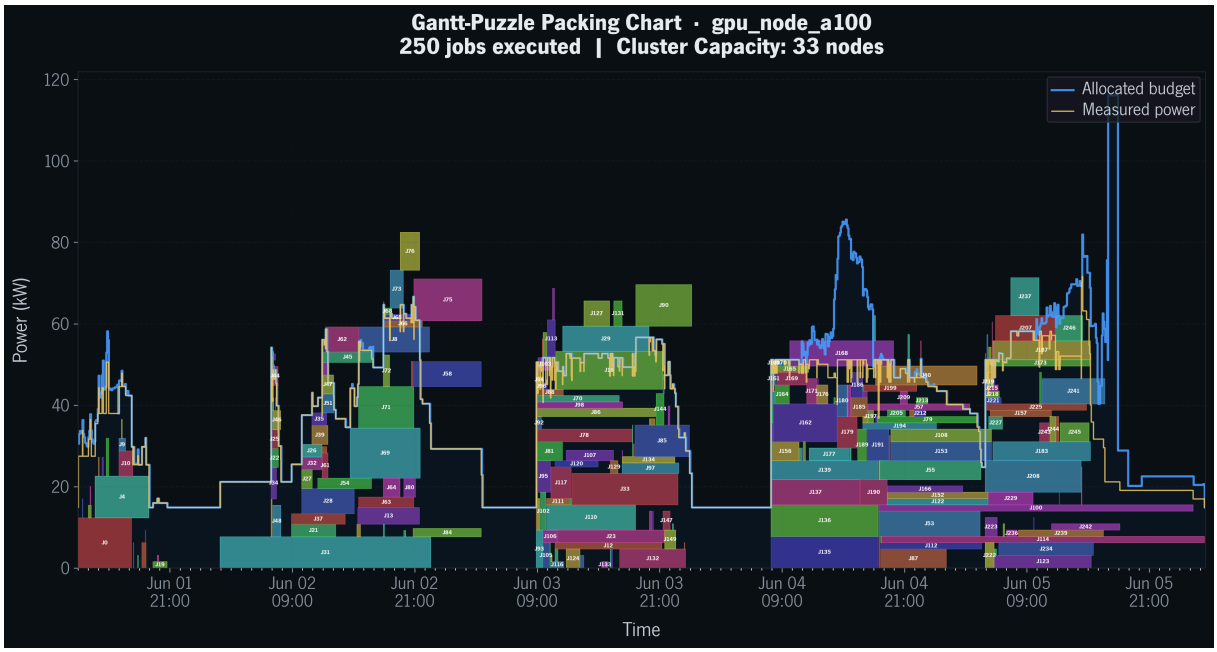


Figure 68: Puzzle execution in the GPU partition with renewable supply (PV and wind).

A.9 Workload-size campaign

Table 16 retains the full metrics behind Table 7. Its final column reports deadline misses followed by monitor samples in which IT power exceeded the policy's own compute budget. Budget excesses are not additional deadline misses: running jobs and idle nodes can continue to draw power after the budget falls.

Puzzle reserves idle demand and ongoing execution in its budget, increasing the grid contribution when needed to cover that demand, while the reference budgets can fall below it. Its zero excess counts therefore partly reflect the budget-generation rule. Since the budgets differ between policies, these counts diagnose each configuration and do not demonstrate better compliance with a common external power cap or lower energy consumption. Figure 69 compares emissions and P95 wait for the small and medium workloads.

Jobs / policy	Mean / P95 wait (h)	Makespan (h)	IT energy (kWh)	Grid import (kWh)	CO_2 (kg)	Effective grid intensity (gCO_2/kWh)	Deadline misses / samples above own budget
500 / FIFO	0.36 / 2.05	100.80	15,850.8	1,250.8	284	227	0 / 769
500 / Priority	0.32 / 1.89	100.80	15,852.8	1,245.1	283	227	0 / 769
500 / EFT	0.32 / 1.90	100.81	15,855.1	1,245.9	283	227	0 / 769
500 / Puzzle	1.65 / 9.76	100.80	15,878.9	777.1	200	258	0 / 0
1,250 / FIFO	1.29 / 7.52	100.82	18,939.6	3,198.4	761	238	0 / 1,810
1,250 / Priority	1.27 / 7.52	100.76	18,935.4	3,192.4	760	238	0 / 1,791
1,250 / EFT	1.27 / 7.52	100.75	18,933.8	3,185.3	758	238	0 / 1,791
1,250 / Puzzle	2.78 / 12.32	100.82	18,851.7	2,510.5	613	244	0 / 0

Table 16: Full metrics for the small and medium workloads, with sampled excesses relative to each policy's own compute budget.

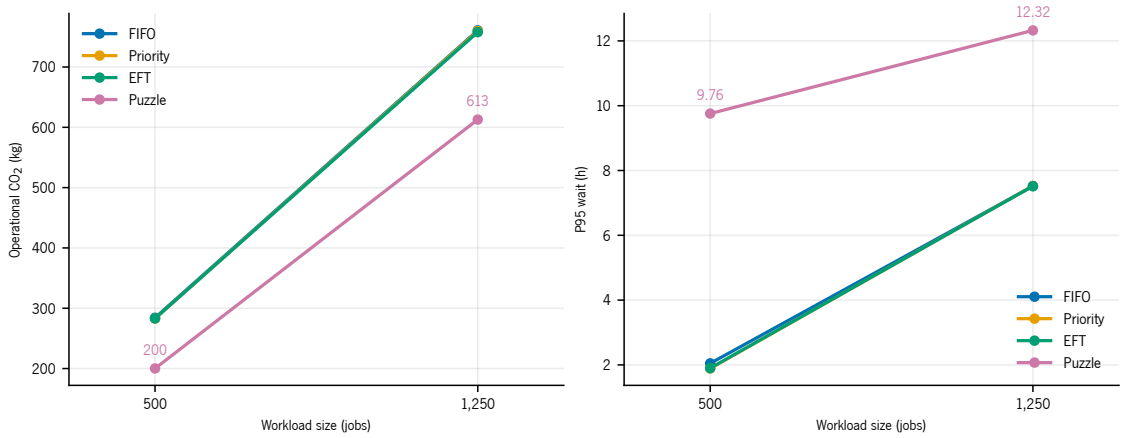


Figure 69: Operational emissions and P95 wait by workload size under PV and wind.

A.9.1 Small workload: FIFO

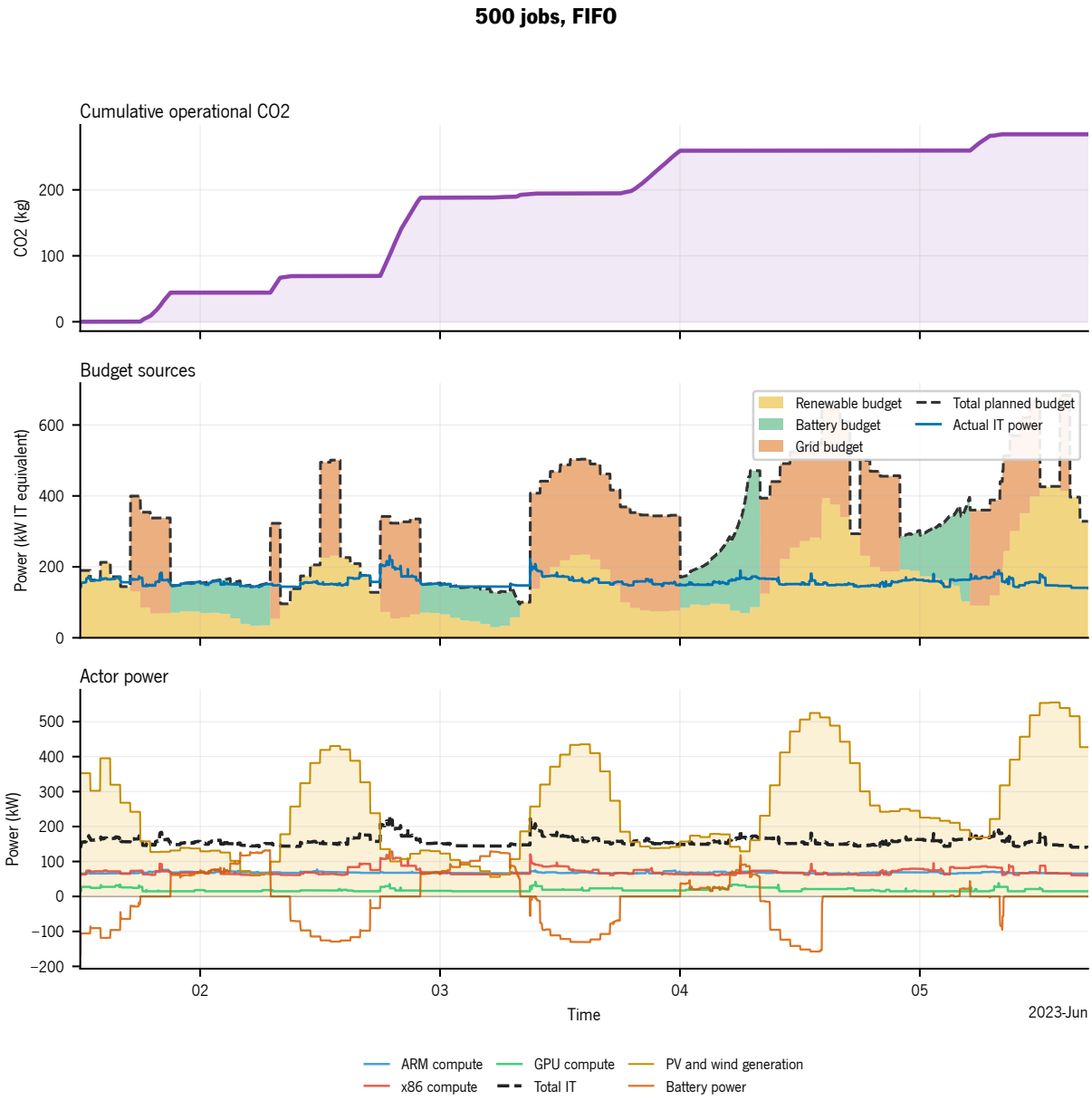


Figure 70: FIFO with the 500-job small workload: carbon emissions, budget sources and actor power.

A.9.2 Small workload: Priority

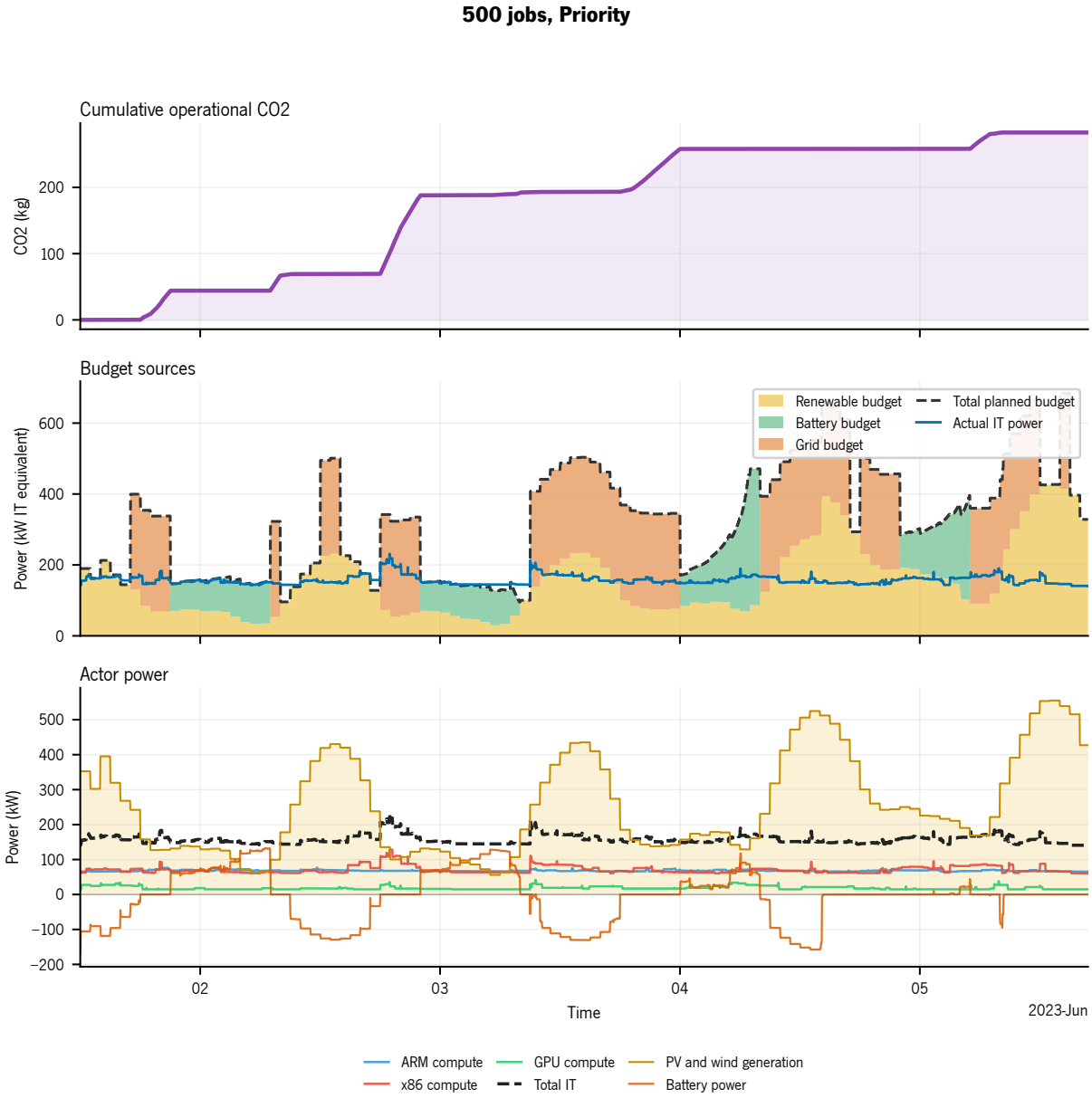


Figure 71: Priority with the 500-job small workload: carbon emissions, budget sources and actor power.

A.9.3 Small workload: EFT

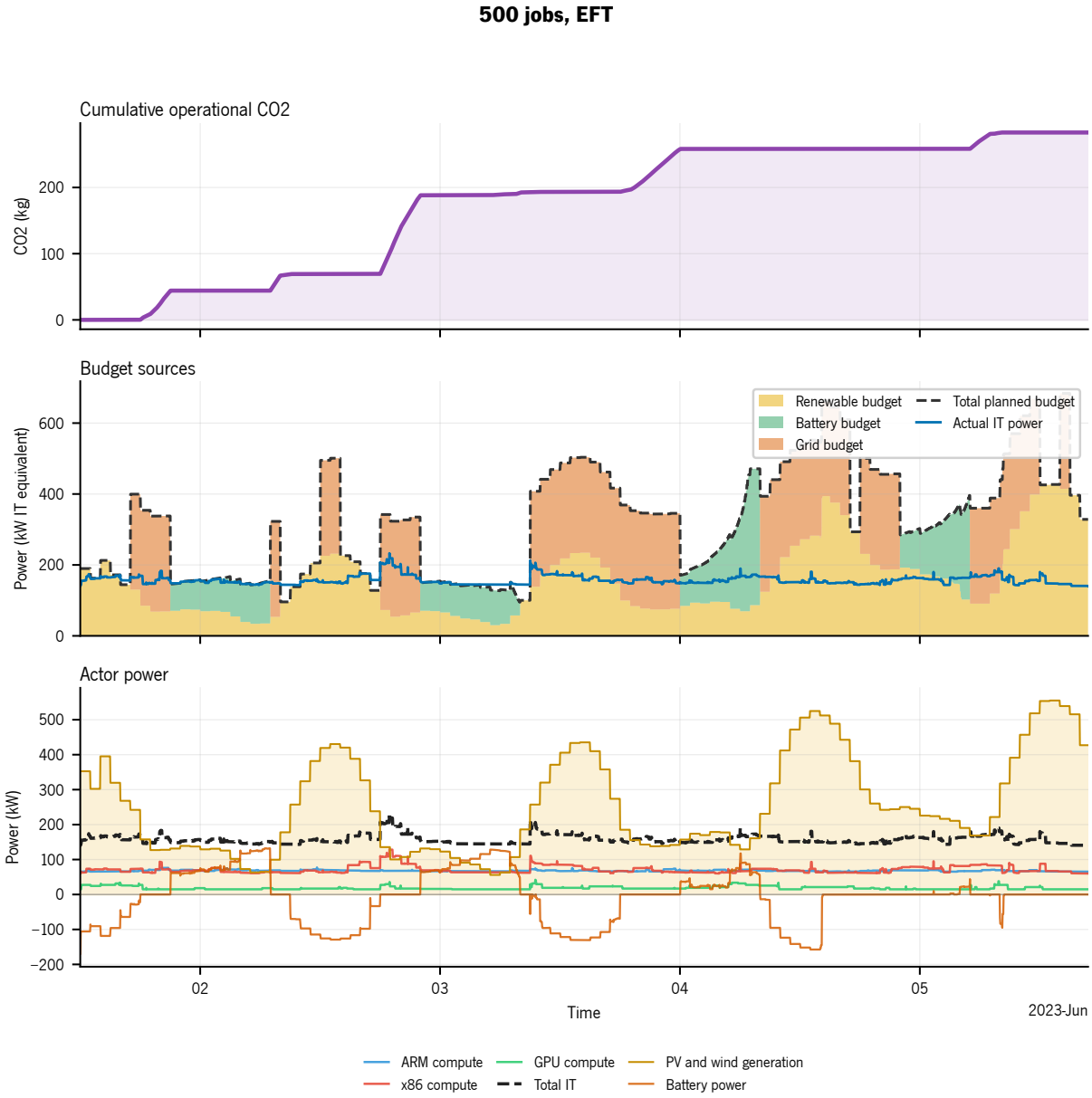


Figure 72: EFT with the 500-job small workload: carbon emissions, budget sources and actor power.

A.9.4 Small workload: Puzzle

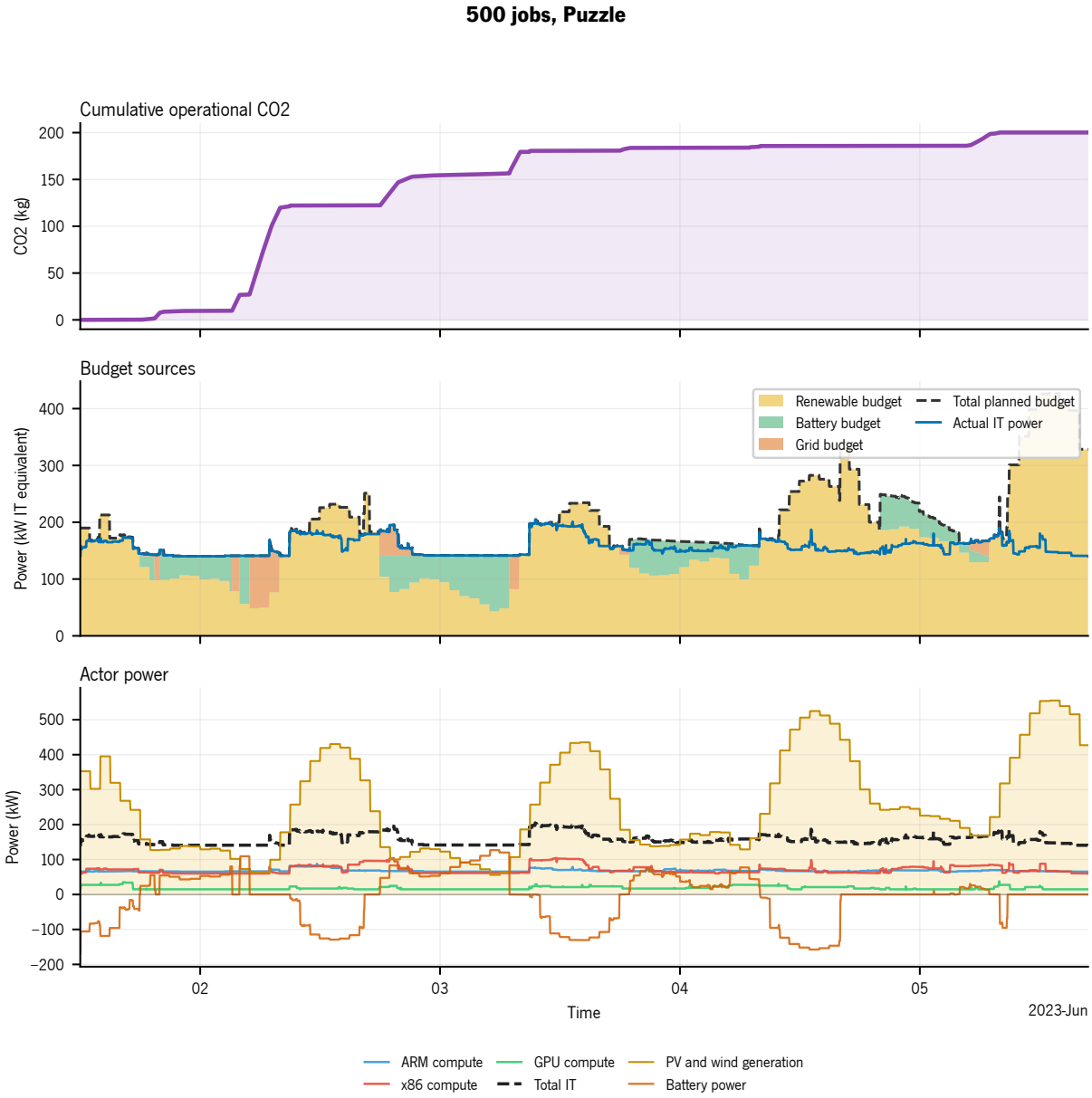


Figure 73: Puzzle with the 500-job small workload: carbon emissions, budget sources and actor power.

A.9.5 Medium workload: FIFO

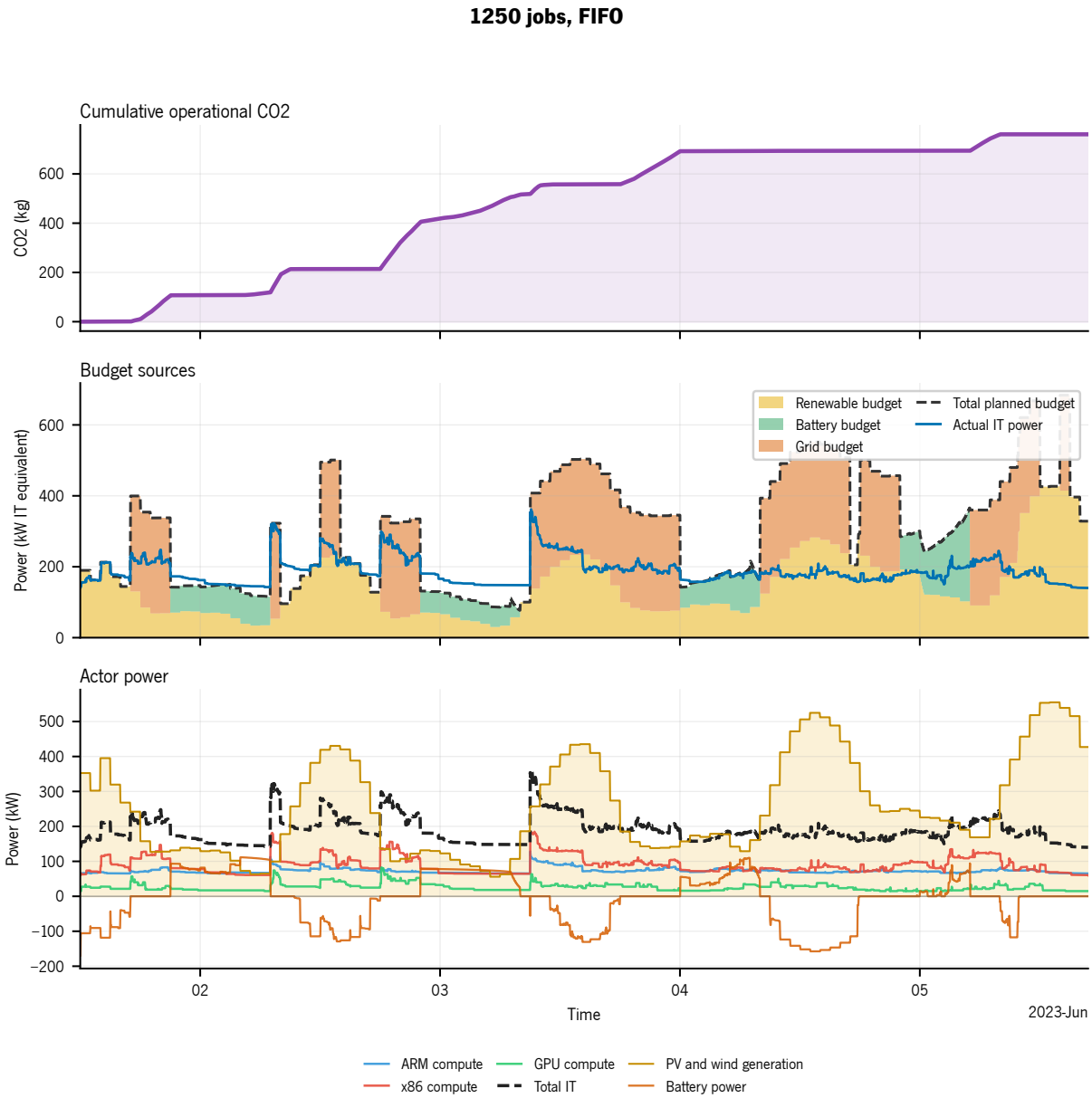


Figure 74: FIFO with the 1,250-job medium workload: carbon emissions, budget sources and actor power.

A.9.6 Medium workload: Priority

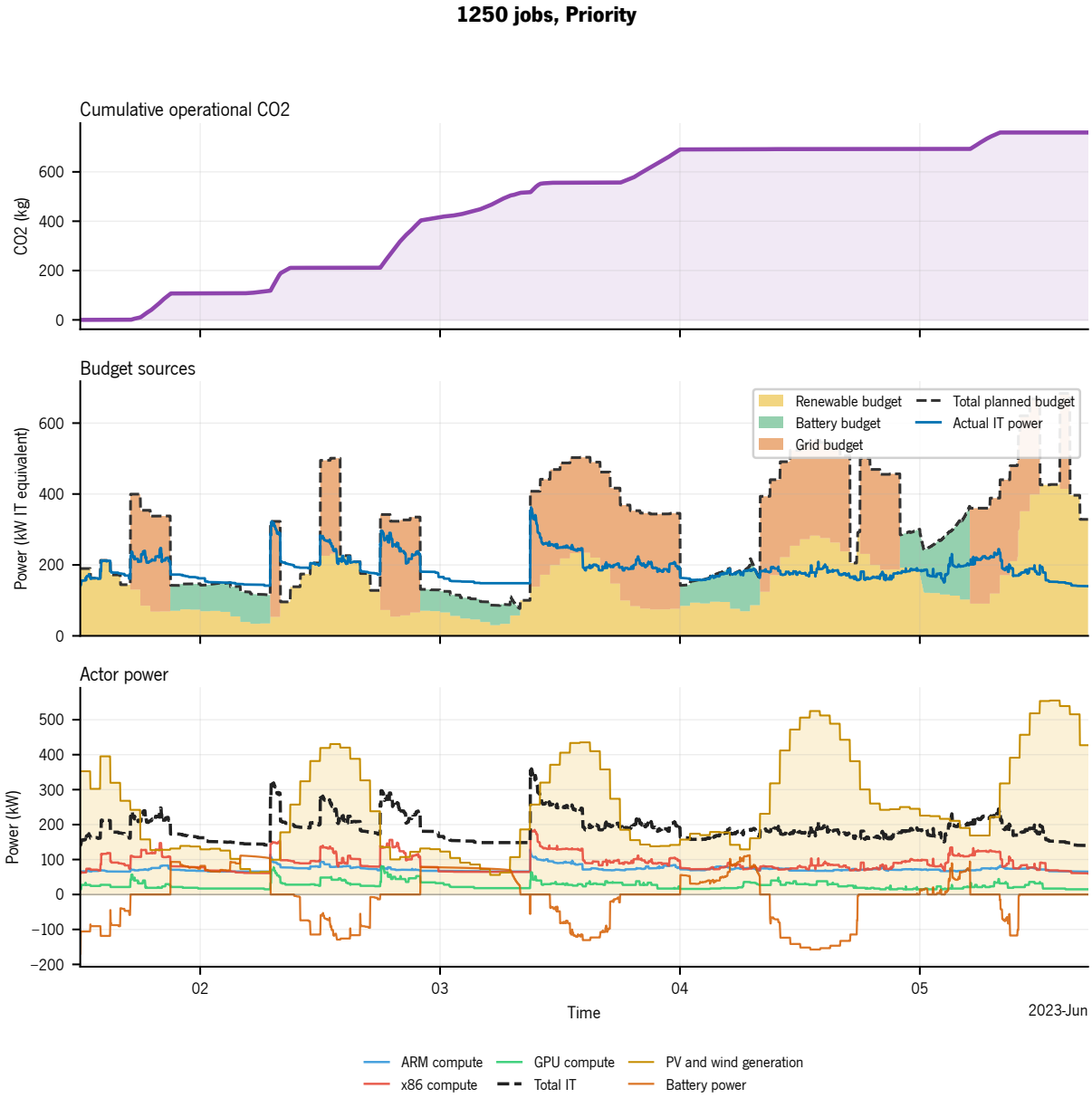


Figure 75: Priority with the 1,250-job medium workload: carbon emissions, budget sources and actor power.

A.9.7 Medium workload: EFT

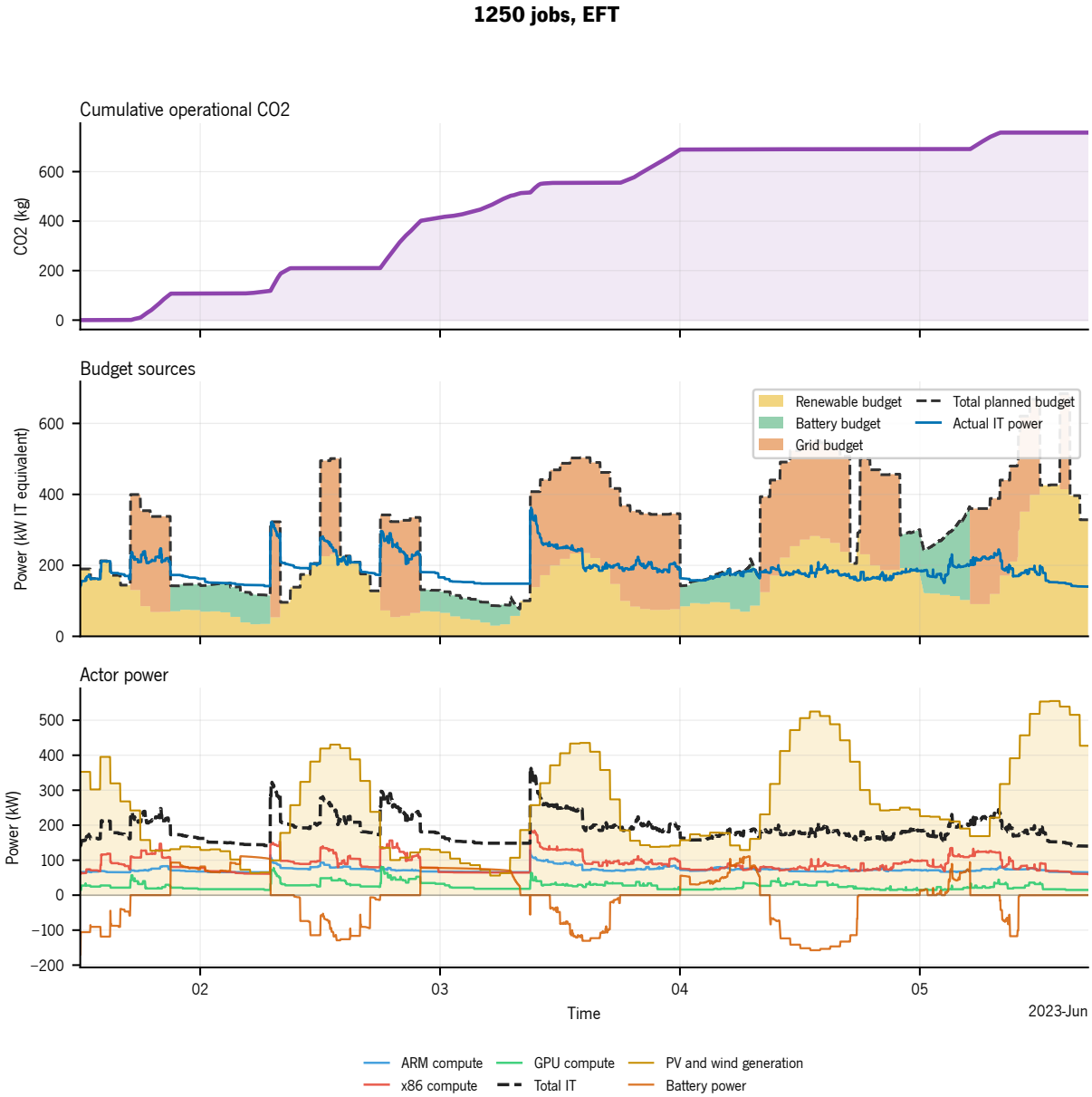


Figure 76: EFT with the 1,250-job medium workload: carbon emissions, budget sources and actor power.

A.9.8 Medium workload: Puzzle

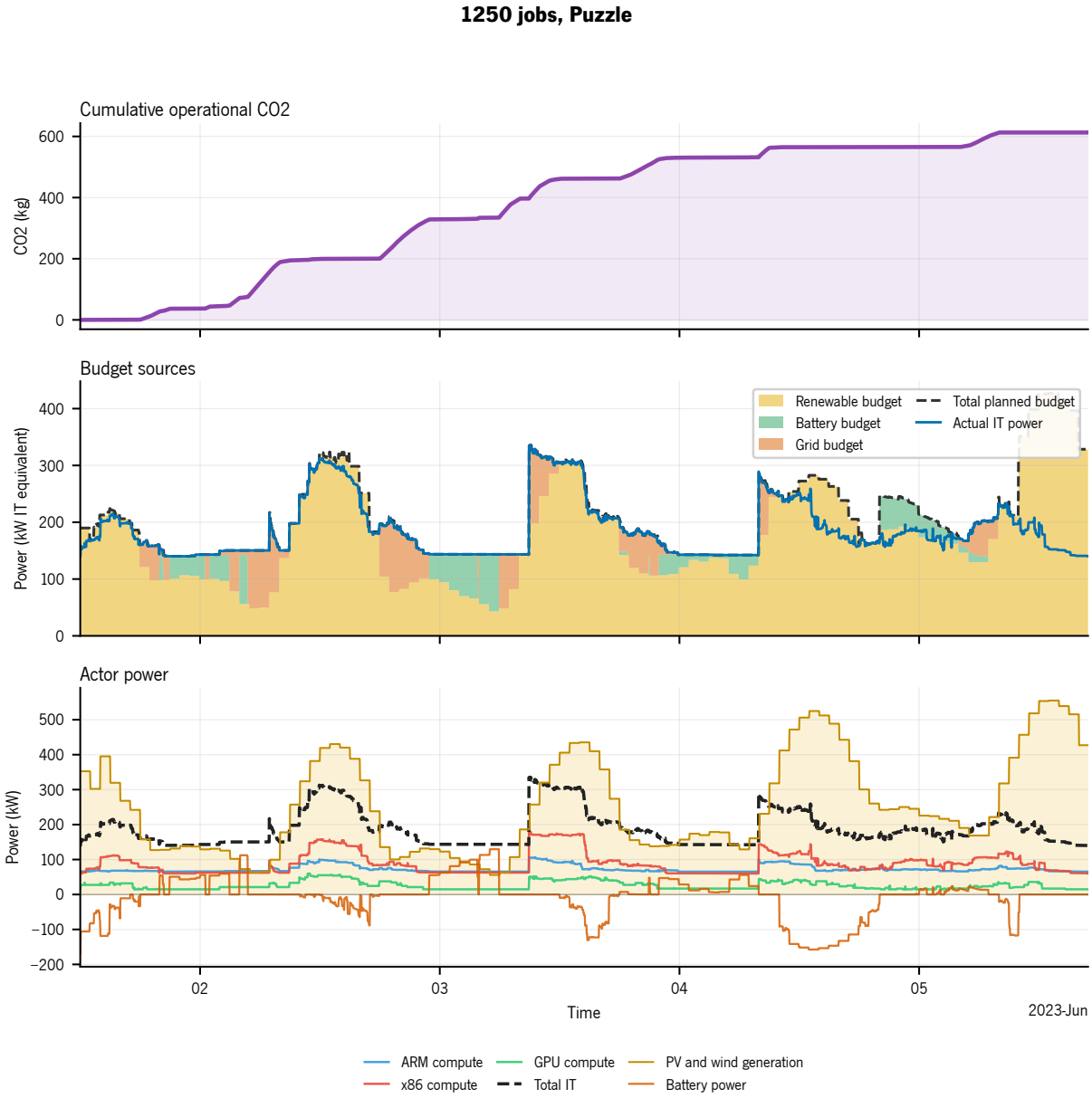


Figure 77: Puzzle with the 1,250-job medium workload: carbon emissions, budget sources and actor power.